

**BAN CƠ YẾU CHÍNH PHỦ
HỌC VIỆN KỸ THUẬT MẬT MÃ**



HOÀNG TRUNG NGUYỄN

**NGHIÊN CỨU ỨNG DỤNG HỌC SÂU TRONG
PHÁT HIỆN TẤN CÔNG THAY ĐỔI
GIAO DIỆN WEB**

ĐỀ ÁN THẠC SĨ

HÀ NỘI – 2024

**BAN CƠ YẾU CHÍNH PHỦ
HỌC VIỆN KỸ THUẬT MẬT MÃ**

HOÀNG TRUNG NGUYỄN

**NGHIÊN CỨU ỨNG DỤNG HỌC SÂU TRONG
PHÁT HIỆN TẤN CÔNG THAY ĐỔI
GIAO DIỆN WEB**

**Chuyên ngành: An toàn thông tin
Mã số: 8480202
Khóa: CHAT09**

ĐỀ ÁN THẠC SĨ

NGƯỜI HƯỚNG DẪN KHOA HỌC:

1. TS. Lại Minh Tuấn – Khoa ATTT, Học viện KTMM

HÀ NỘI – 2024

LỜI CAM ĐOAN

Tôi xin cam đoan rằng, đề án này là kết quả của quá trình nghiên cứu và làm việc nghiêm túc của bản thân tôi dưới sự hướng dẫn của TS. Lại Minh Tuấn. Tất cả các nội dung, số liệu và tài liệu tham khảo trong đề án đều là trung thực và có nguồn gốc rõ ràng, được trích dẫn đầy đủ theo qui định.

Những kết quả nghiên cứu trong luận văn là chưa từng được công bố trong bất kỳ luận văn, công trình nghiên cứu hay bài báo khoa học nào khác. Mọi sự sao chép, tham khảo không đúng quy định nếu có trong luận văn, tôi xin hoàn toàn chịu trách nhiệm trước hội đồng nhà trường.

Tôi xin chân thành cảm ơn sự hướng dẫn, giúp đỡ của thầy Lại Minh Tuấn, gia đình và bạn bè đã ủng hộ và tạo điều kiện cho tôi hoàn thành đề án này.

HỌC VIÊN THỰC HIỆN

Hoàng Trung Nguyên

LỜI MỞ ĐẦU

Trong thời đại kỹ thuật số hiện nay, các trang web đã trở thành nền tảng quan trọng không chỉ cho giao tiếp mà còn cho các hoạt động kinh doanh và dịch vụ trực tuyến. Sự phát triển của công nghệ mạng đã mang lại nhiều tiện ích nhưng cũng kéo theo những mối đe dọa an ninh ngày càng phức tạp và tinh vi. Một trong những phương thức tấn công nguy hiểm nhất là việc thay đổi giao diện của trang web, nhằm làm sai lệch thông tin và lừa đảo người dùng.

Những cuộc tấn công như vậy không chỉ gây thiệt hại về kinh tế mà còn làm tổn hại đến uy tín và hình ảnh của các tổ chức, doanh nghiệp. Trong bối cảnh này, việc nghiên cứu và áp dụng các phương pháp tiên tiến để phát hiện và ngăn chặn những tấn công thay đổi giao diện web trở nên cấp thiết hơn bao giờ hết.

Đã có rất nhiều nghiên cứu đưa ra nhằm phát hiện sớm nhằm ngăn chặn và khắc phục vấn đề này. Tuy nhiên đến nay các phương pháp vẫn luôn có những nhược điểm lớn, còn cảnh báo sai và bỏ sót rất nhiều. Trong những năm gần đây, học sâu ngày càng phát triển và ứng dụng rộng rãi trong nhiều lĩnh vực, các mô hình học sâu mới ra đời với độ chính xác cao và độ trễ giảm đáng kể, cho phép theo dõi trong thời gian thực. Do đó, việc áp dụng bài toán học sâu vào việc phát hiện tấn công thay đổi giao diện web là một hướng nghiên cứu thiết thực, có thể giải quyết hiệu quả các nhược điểm của các phương pháp trước đó. Chính vì lý do đó em đã chọn đề tài ***“Nghiên cứu ứng dụng học sâu trong phát hiện tấn công thay đổi giao diện web”***.

Nội dung đề tài gồm 3 chương:

Chương 1. Cơ sở lý thuyết

Chương này tìm hiểu về tổng quan tình hình về vấn đề tấn công thay đổi giao diện web. Đồng thời tìm hiểu tổng quan về công nghệ học sâu dùng cho nghiên cứu tại chương sau.

Chương 2. Nghiên cứu phương pháp phát hiện tấn công thay đổi giao diện web

Chương này nghiên cứu về các phương pháp và huấn luyện một mô hình học sâu để phát hiện hành vi tấn công thay đổi giao diện web.

Chương 3. Xây dựng hệ thống phát hiện cảnh báo tấn công thay đổi giao diện web

Chương này sẽ xây dựng một hệ thống cảnh báo hoàn chỉnh khi có sự kiện tấn công thay đổi giao diện web.

Vì thời gian, kiến thức có hạn nên báo cáo còn nhiều hạn chế và khó tránh khỏi sai sót, em rất mong nhận được ý kiến đóng góp của thầy cô để báo cáo được hoàn thiện hơn.

Em xin chân thành cảm ơn!

HỌC VIÊN THỰC HIỆN

Hoàng Trung Nguyên

MỤC LỤC

LỜI CAM ĐOAN.....	i
LỜI MỞ ĐẦU.....	ii
MỤC LỤC.....	iv
DANH MỤC KÍ HIỆU VÀ TỪ VIẾT TẮT.....	vi
DANH MỤC CÁC BẢNG BIỂU.....	vii
DANH MỤC HÌNH ẢNH.....	viii
CHƯƠNG 1: CƠ SỞ LÝ THUYẾT.....	1
1.1. Tổng quan tấn công thay đổi giao diện web	1
1.1.1. Tổng quan	1
1.1.2. Một số kỹ thuật tấn công có thể gây thay đổi giao diện web	2
1.1.3. Các phương pháp phát hiện tấn công thay đổi giao diện web.....	6
1.2. Học sâu và bài toán phát hiện tấn công thay đổi giao diện web	9
1.2.1. Học máy.....	9
1.2.2. Học sâu	10
1.3. Mô hình YOLO	17
1.3.1. Giới thiệu.....	17
1.3.2. YOLOv10	29
1.4. Kết luận chương 1	35
CHƯƠNG 2: NGHIÊN CỨU PHƯƠNG PHÁP PHÁT HIỆN TẤN CÔNG THAY ĐỔI GIAO DIỆN WEB.....	36
2.1. Phương pháp tiếp cận.....	36
2.2. Thu thập dữ liệu đầu vào.....	42
2.2.1. Dữ liệu về các trang web bị tấn công thay đổi giao diện	42
2.2.2. Dữ liệu về các trang web bình thường	45
2.3. Huấn luyện mô hình.....	47
2.3.1. Chuẩn bị tập dữ liệu.....	47
2.3.2. Huấn luyện mô hình	49
2.4. Đánh giá mô hình	51

2.4.1. Ma trận nhầm lẫn.....	51
2.4.2. Đường cong điểm F1	52
2.4.3. Đường cong thu hồi chính xác	54
2.4.4. So sánh với các mô hình học sâu khác	55
2.5. Kết luận chương 2	55
CHƯƠNG 3: XÂY DỰNG HỆ THỐNG PHÁT HIỆN VÀ CẢNH BÁO TẤN CÔNG THAY ĐỔI GIAO DIỆN WEB.....	56
3.1. Đặt vấn đề.....	56
3.2. Xây dựng hệ thống cảnh báo.....	57
3.2.1. Các thành phần hệ thống cảnh báo	57
3.2.2. Phân tích thiết kế hệ thống	57
3.2.3. Triển khai hệ thống.....	64
3.3. Triển khai Agent.....	72
3.3.1. Các thành phần hệ thống kết nối Agent	74
3.3.2. Phiên bản cho hệ điều hành Linux	76
3.3.3. Phiên bản cho hệ điều hành Windows.....	78
3.3.4. Giao diện tạo khóa kích hoạt cho Agent	81
3.4. Thực nghiệm đánh giá kết quả	85
3.4.1. Kịch bản 1.....	85
3.4.2. Kịch bản 2.....	89
3.5. Kết luận chương 3	93
KẾT LUẬN.....	94
PHỤ LỤC.....	95
Phụ lục 1: Đóng gói vào Docker	95
TÀI LIỆU THAM KHẢO.....	97

DANH MỤC KÍ HIỆU VÀ TỪ VIẾT TẮT

API	Application Programming Interface
DOM	Document Object Model
EDR	Endpoint Detection and Response
F1	F1 score
FNR	False negative rate
FPR	False positive rate
GPU	Graphics Processing Unit
HTTP	HyperText Transfer Protocol
POC	Proof of Concept
PPV	Positive predictive value
SOC	Security Operation Center
TPR	True positive rate
TPU	Tensor Processing Unit
URL	Uniform Resource Locator
VPN	Virtual Private Network
WAF	Web Application Firewall
YOLO	You Only Look Once
NMS	Non-Maximum Suppression
PAN	Path Aggregation Network
E-ELAN	Extended Efficient Layer Aggregation Networks

DANH MỤC CÁC BẢNG BIỂU

Bảng 1. So sánh hiệu suất YOLOv10 so với các phiên bản trước.....	34
Bảng 2. So sánh YOLOv10 với các mô hình học sâu khác	55

DANH MỤC HÌNH ẢNH

Hình 1.1. Thống kê số lượng trang web tên miền .com bị tấn công giai đoạn năm 2011-2024 (thống kê bởi www.cybergates.org)	1
Hình 1.2. Sơ đồ thuật toán kiểm tra sự thay đổi	6
Hình 1.3. Mối quan hệ giữa AI, Machine Learning và Deep Learning.....	10
Hình 1.4. Confusion matrix (2 nhãn)	15
Hình 1.5. Confusion matrix (3 nhãn)	15
Hình 1.6. Tính giá trị Precision, Recall.....	16
Hình 1.7. Sự phát triển qua các phiên bản YOLO	18
Hình 1.8. Kiến trúc mô hình YOLOv1	19
Hình 1.9. Mô hình mạng Darknet-53.....	21
Hình 1.10. Kiến trúc FPN được áp dụng trong YOLOv3.....	22
Hình 1.11. Khung kiến trúc cơ bản của YOLOv5	23
Hình 1.12. Kiến trúc mạng YOLOv6.....	24
Hình 1.13. Chiến lược gán nhãn kép nhất quán cho huấn luyện không cần NMS	32
Hình 2.1. Dị thường tồn tại ở hai bên của đường cong hình chuông.....	37
Hình 2.2. Các điểm dị thường nằm ngoài lớp cách ly được mô hình xây dựng trong Isolation Forest	37
Hình 2.3. Mô hình nén và tái cấu trúc dữ liệu trong Autoencoder	39
Hình 2.4. Tái cấu trúc chữ số 4 dựa trên mô hình autoencoder đã huấn luyện .	39
Hình 2.5. Tái cấu trúc ảnh con voi dựa trên mô hình autoencoder đã huấn luyện.	40
Hình 2.6. Quy trình phát triển mô hình học sâu.....	41
Hình 2.7. Thông tin cập nhật các trang web đã bị tấn công thay đổi giao diện.	43
Hình 2.8. Dữ liệu từ trang Xatrix.org.....	44
Hình 2.9. Biểu đồ kết quả quá trình huấn luyện	49
Hình 2.10. Chi tiết kết quả quá trình huấn luyện	50
Hình 2.11. Ma trận nhầm lẫn	52
Hình 2.12. F1-Confidence Curve	53
Hình 2.13. Biểu đồ Precision-Recall.....	54

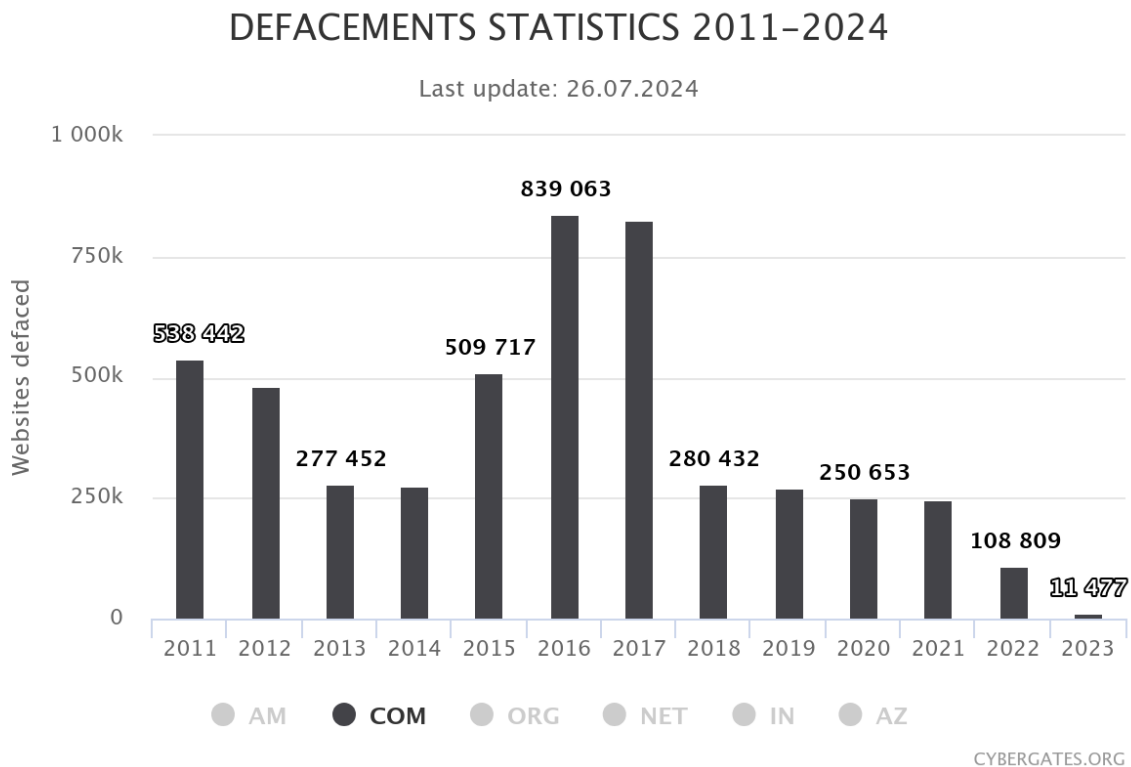
Hình 3.1. Một số giải pháp bảo vệ cho hoạt động ứng dụng web	56
Hình 3.2. Sơ đồ phát hiện và cảnh báo.....	57
Hình 3.4. Giao diện nhận thông tin cấu hình từ người dùng	65
Hình 3.5. Sơ đồ đăng kí theo dõi một URL	65
Hình 3.6. Giao diện đăng kí theo dõi một trang web.....	66
Hình 3.7. Câu lệnh CronTab sau khi đăng kí.....	71
Hình 3.8. Cảnh báo được gửi về email	71
Hình 3.9. Cảnh báo được gửi về tin nhắn Telegram.....	72
Hình 3.10. Các vị trí bị thay đổi giao diện (theo Mirror-h, Zone-Kurd-h).....	73
Hình 3.11. Sơ đồ xử lý sự kiện từ Agent	75
Hình 3.12. Cây thư mục các thành phần của Agent.....	76
Hình 3.13. Sơ đồ đăng kí theo dõi một URL	82
Hình 3.14. Giao diện tạo khóa xác thực Agent.....	83
Hình 3.15. Trạng thái giao diện web tại lần cuối cùng được chụp và thông tin khóa	84
Hình 3.17. Sơ đồ kịch bản 1.....	85
Hình 3.18. Kịch bản 1: Đăng kí cấu hình theo dõi trang web.....	86
Hình 3.19. Kịch bản 1: Giao diện ứng dụng web chứa lỗ hổng khai thác.....	86
Hình 3.20. Kịch bản 1: cảnh báo về Email	88
Hình 3.21. Kịch bản 1: cảnh báo về Telegram.....	89
Hình 3.22. Sơ đồ kịch bản 2.....	90
Hình 3.23. Kịch bản 2: active key được gửi về Email.....	90
Hình 3.25. Kịch bản 2: cảnh báo một tệp tin bị thay đổi giao diện bất thường.	92
Hình 3.26. Kịch bản 2: cảnh báo về telegram.....	92

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT

1.1. Tổng quan tấn công thay đổi giao diện web

1.1.1. Tổng quan

Trong những năm gần đây, với sự gia tăng ứng dụng công nghệ thông tin trong các cơ quan, doanh nghiệp và tổ chức, đặc biệt là trong lĩnh vực giáo dục, các hệ thống quản lý và dịch vụ hành chính công trực tuyến đang được triển khai rộng rãi, từng bước thiết lập chính quyền điện tử. Tuy nhiên, sự phát triển này cũng đồng thời làm gia tăng mối nguy cơ về an toàn và bảo mật đối với các hệ thống thông tin. Đặc biệt, tấn công thay đổi giao diện web là một trong những kết quả của tấn công mạng phổ biến đã nhắm vào các trang thông tin điện tử của các cơ quan giáo dục và tổ chức tại Việt Nam. Các cuộc tấn công này có xu hướng gia tăng, đặt ra nguy cơ thường trực về mất an toàn, an ninh thông tin và uy tín của tổ chức.



Hình 1.1. Thống kê số lượng trang web tên miền .com bị tấn công giai đoạn năm 2011-2024 (thống kê bởi www.cybergates.org)

Tấn công thay đổi giao diện là một kiểu tấn công mạng, trong đó các kẻ tấn công sẽ cố tình để lại dấu vết bằng cách thay đổi hoặc thay thế nội dung của

trang web đó. Việc thay đổi giao diện có rất nhiều lý do cho mỗi chiến dịch tấn công. Thứ nhất là nhằm tuyên truyền các tư tưởng tôn giáo hoặc vấn đề chính trị, việc này thường xảy ra tại các khu vực có tình trạng chính trị bất ổn như chiến tranh, xung đột vũ trang, tranh chấp chủ quyền, xung đột tôn giáo, sắc tộc. Việc thay đổi giao diện trang web có thể coi như là một hình thức tấn công từ chối dịch vụ nhằm ngăn người dân – những người truy cập trang web một cách hợp pháp, tiếp cận được thông tin tin cậy từ phía chính phủ, kẻ tấn công sẽ để lại thông tin hoặc sửa đổi nội dung trang web sai sự thật nhằm đầu độc tư tưởng và nhận thức đúng của người dân. Thứ hai là để khoe chiến tích, rất nhiều chiến dịch tấn công phá hoại chỉ để cho vui hoặc thể hiện trình độ của nhóm và các nhóm tin tặc sẽ để lại thông tin như là tên nhóm thực hiện, một kênh liên lạc với họ như địa chỉ email hoặc đường dẫn telegram để gây tiếng vang cho họ. Thứ ba là để cảnh báo cho quản trị viên biết trang web của họ đang tồn tại lỗ hổng bảo mật/điểm yếu nghiêm trọng, tuy nhiên cho dù có là mục đích tốt đi chăng nữa, việc tấn công thay đổi giao diện như vậy là bất hợp pháp luôn cần lên án.

Dù là mục đích nào, tấn công thay đổi giao diện luôn để lại hậu quả nghiêm trọng, gây ảnh hưởng tới hoạt động kinh doanh của doanh nghiệp, gây mất uy tín cho các tổ chức. Sau khi thay đổi diện mạo, các trang web phải đóng cửa để sửa chữa và xem xét bảo mật, đôi khi việc này phải thực hiện trong một thời gian dài, gây ra chi phí và mất lợi nhuận và giá trị. Nếu theo dõi danh sách các trang web bị tấn công được báo cáo trong mirror-h.org hay zone-h.org, phần lớn các trang web bị tấn công thay đổi giao diện web nằm ở các nước có điểm nóng về chiến tranh, xung đột chính trị. Những cuộc tấn công này phản ánh sự phức tạp và đa diện của chiến tranh hiện đại, nơi mà không gian mạng trở thành một chiến trường mới.

1.1.2. Một số kỹ thuật tấn công có thể gây thay đổi giao diện web

Có rất nhiều kỹ thuật tấn công tác động tới một trang web từ phía lớp ứng dụng cho tới lớp mạng, tuy nhiên để có thể thực hiện thay đổi giao diện được trang web thì các kỹ thuật tấn công này phải tác động được trực tiếp và thực thi mã từ xa lên máy chủ. Các cuộc tấn công trên ứng dụng web có thể kể đến như:

SQL Injection, Command Injection, Path Traversal, Remote File inclusion, Unrestricted File Upload, Weak authentication, XML external entity (XXE) injection, ...

1.1.2.1. SQL Injection

SQL injection (SQLi) là một lỗ hổng bảo mật web cho phép kẻ tấn công chèn vào các truy vấn mà ứng dụng thực hiện đối với cơ sở dữ liệu của ứng dụng. Điều này có thể cho phép kẻ tấn công xem các dữ liệu nhạy cảm mà ứng dụng bình thường không hiển thị. Trong nhiều trường hợp, kẻ tấn công có thể thêm, sửa hoặc xóa dữ liệu, thậm chí là tải lên mã độc hại, chiếm quyền điều khiển máy chủ cơ sở dữ liệu. Nó cũng có thể cho phép chúng thực hiện các cuộc tấn công từ chối dịch vụ.

SQL Injection có nhiều hình thức tấn công như Error-Based SQL Injection, kẻ tấn công khai thác lợi dụng các thông báo lỗi không được xử lý kỹ gửi về từ phía máy chủ; Boolean-based SQL Injection, mặc dù một số chức năng không trả về thông báo lỗi, tuy nhiên kẻ tấn công có thể theo dõi các thể hiện của trang web khi xử lý một câu lệnh SQL (do kẻ tấn công chèn vào) không hợp lệ có thể xác định được lỗi. Để khai thác lỗi này, kẻ tấn công sẽ dựa vào các câu điều kiện Boolean trong SQL để khai thác; Time-based SQL Injection, cũng giống như Boolean-based SQL Injection, khi xử lý câu truy vấn không lệ máy chủ sẽ không thể hiện thông báo lỗi, tuy nhiên dựa vào thời gian phản hồi của trang web, kẻ tấn công có thể xác định được lỗi và khai thác nó.

Khi trang web có lỗ hổng SQL injection, thông qua câu truy vấn “INTO OUTFILE”, kẻ tấn công có thể tải được web shell lên máy chủ và thực thi mã tùy ý trên đó.

1.1.2.2. Command Injection

Đây là kỹ thuật tấn công phép chèn các câu lệnh thực thi trên máy chủ. Điều này thường xảy ra khi một ứng dụng nhận dữ liệu từ người dùng mà không được làm sạch đúng cách, và sau đó truyền dữ liệu này vào hệ thống và thực thi chúng. Ví dụ đoạn mã nguồn ứng dụng web sau:

```
<?php
if (isset($_POST['filename'])) {
    $filename = $_POST['filename'];
    $output = shell_exec("cat " . $filename);
    echo "<pre>$output</pre>";
}
?>

<form method="post">
    File      Name:      <input      type="text"
name="filename">
    <input type="submit" value="Submit">
</form>
```

Mã nguồn trên thực hiện lệnh `cat` để đọc nội dung tệp tin mà người dùng chỉ định. Tuy nhiên, không có bước kiểm tra hay xác minh nào đối với biến `$filename`, do đó kẻ tấn công có thể chèn các lệnh độc hại vào đây một chuỗi như `file.txt; ls`, kết quả là lệnh sau sẽ được thực thi trên máy chủ và hiển thị danh sách các tệp tin trên thư mục.

1.1.2.3. Path Traversal

Đây là lỗ hổng này cho phép kẻ tấn công đọc các tệp tin tùy ý trên máy chủ đang chạy ứng dụng. Điều này có thể bao gồm:

- Mã nguồn ứng dụng và dữ liệu nhạy cảm
- Các thông tin xác thực
- Các tệp tin cấu hình hệ điều hành

Khi có được các thông tin nhạy cảm này của máy chủ, kẻ tấn công dễ dàng thực hiện các bước khai thác tiếp theo để chiếm quyền điều khiển máy chủ.

1.1.2.4. Remote File inclusion

Remote File Inclusion (RFI) là một lỗ hổng bảo mật trên web cho phép tin tặc độc hại buộc ứng dụng “include” các tệp tin mã tùy ý được nhập từ một vị trí khác, ví dụ: máy chủ do kẻ tấn công kiểm soát, ứng dụng web có thể bị lừa đưa vào các lệnh độc hại và thực thi nó.

Nhiều ngôn ngữ lập trình, bao gồm cả những ngôn ngữ được sử dụng ở phía máy chủ để phát triển ứng dụng web, cho phép nhà phát triển đưa mã

nguồn từ các tệp tin khác vào. Một số hàm được sử dụng để đưa mã nguồn vào chỉ cho phép đưa các tệp cục bộ vào, nhưng ở một số ngôn ngữ, chúng cũng cho phép đưa vào theo URL. Do vậy, nếu kẻ tấn công thao túng được đầu vào, buộc máy chủ bao gồm một mã nguồn từ xa do kẻ tấn công kiểm soát. Khi thực thi, kẻ tấn công hoàn toàn có thể thực thi mã từ xa trên máy chủ.

1.1.2.5. Local File inclusion

Local file inclusion (LFI) là một cơ chế xử lý các cơ chế bao gồm tệp tin trong ứng dụng web, các tệp tin này đã có sẵn trên máy chủ. Lỗi hỏng này xảy ra khi một trang nhận đường dẫn đến tệp làm đầu vào để thực thi tệp tin đó, đầu vào này không được làm sạch đúng cách, kẻ tấn công hoàn toàn có thể thực thi một tệp tin độc hại bất kỳ trên máy chủ cục bộ. Lỗi hỏng này thường kết hợp với lỗi hỏng Unrestricted File Upload để tải tệp tin độc hại lên và thực thi nó chiếm quyền điều khiển máy chủ.

1.1.2.6. Unrestricted File Upload

Đây là một lỗi hỏng nhằm vào chức năng tải tệp tin lên máy chủ từ phía khách hàng. Một số ứng dụng yêu cầu chức năng tải ảnh hay tệp tin lên máy chủ cho chức năng xác định, nếu ứng dụng không kiểm soát đầu vào tốt, kẻ tấn công có thể tải mã độc hay web shell và thực thi chúng trên phía máy chủ.

1.1.2.7. Weak authentication

Đây là một lỗi hỏng tác động tới các phương thức xác thực yếu trong hệ thống. Khi mà người quản trị đặt các thông tin đăng nhập của các giao thức, dịch vụ là mặc định, dễ đoán hoặc là không có hay không có cơ chế giới hạn số lần đăng nhập sai. Kẻ tấn công có thể đoán được mật khẩu và truy cập chiếm quyền điều khiển của máy chủ.

1.1.2.8. XML external entity (XXE) injection

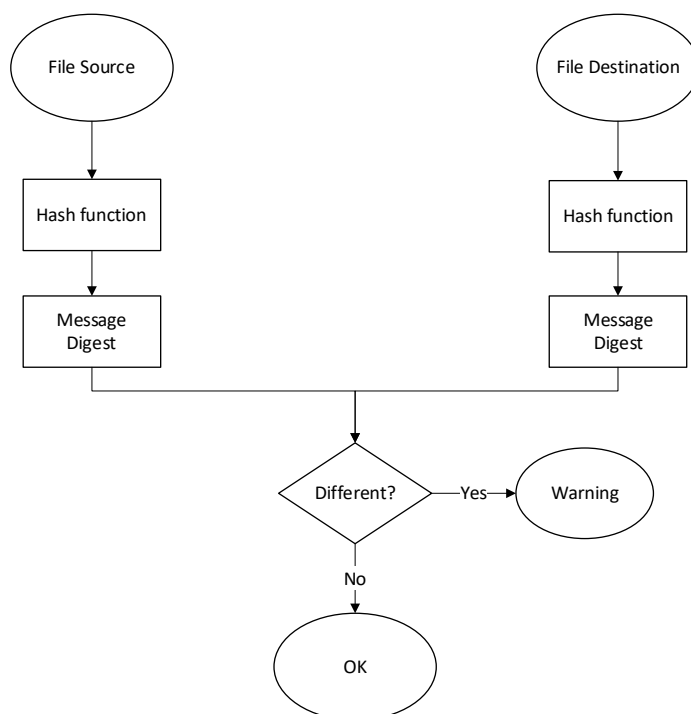
XXE (XML External Entity) Injection là một loại lỗi hỏng bảo mật xảy ra khi một ứng dụng xử lý các dữ liệu XML không an toàn, cho phép kẻ tấn công chèn hoặc thay đổi các thực thể bên ngoài (external entity) trong tài liệu XML. Điều này có thể dẫn đến việc tiết lộ thông tin nhạy cảm, truy cập vào tài nguyên nội bộ của hệ thống hoặc thậm chí thực thi mã độc hại từ xa.

XXE thường xảy ra khi ứng dụng cho phép phân tích và xử lý các thực thể bên ngoài trong tài liệu XML mà không kiểm tra kỹ. Các thực thể này là các tham chiếu đến tài nguyên bên ngoài, như tệp tin hoặc URL. Ví dụ, kẻ tấn công có thể định nghĩa một thực thể trỏ đến tệp tin nội bộ trong hệ thống (như /etc/passwd trên Linux) và dùng thực thể đó để lấy dữ liệu nhạy cảm từ server.

1.1.3. Các phương pháp phát hiện tấn công thay đổi giao diện web

1.1.3.1. Phương pháp so sánh mã băm

Đây là một phương pháp tạo ra các chữ kí tin cậy cho bộ mã nguồn bằng hàm băm, nhằm kiểm tra các thay đổi, thêm bớt của tệp tin sau một khoảng thời gian nhất định, như minh họa trong sơ đồ thuật toán ở hình 1.2.



Hình 1.2. Sơ đồ thuật toán kiểm tra sự thay đổi

Đầu tiên có thể nói việc sử dụng hàm băm là một phương pháp đơn giản nhất để phát hiện sự thay đổi trong trang web. Thứ nhất, mã băm của nội dung trang web được tính bằng thuật toán băm, chẳng hạn như MD5 hoặc SHA1 và được lưu trữ. Thứ hai, sẽ cần tính toán một mã băm mới sau một khoảng thời gian và so sánh với mã băm gốc. Nếu hai giá trị khác nhau, một cảnh báo được đưa ra. Kỹ thuật này trông có vẻ ổn đối với các trang web tĩnh. Tuy nhiên, kỹ thuật này không áp dụng được đối với các trang web động do nội dung của chúng thay đổi thường xuyên.

1.1.3.2. Phương pháp sử dụng chữ kí

Cách tiếp cận phát hiện dựa trên chữ ký trước tiên tạo ra một tập hợp các chữ ký tấn công đã biết từ các trang web bị tấn công thay đổi giao diện. Chữ ký của kẻ tấn công thường được mã hóa dưới dạng các tập luật, hoặc chuỗi kí tự. Sau đó, phương pháp này sẽ tìm kiếm các chữ ký của kẻ tấn công trong các trang web được giám sát. Nếu một mẫu được tìm thấy, một cuộc tấn công thay đổi giao diện web được phát hiện. Ưu điểm của cách tiếp cận này nhanh chóng và hiệu quả để phát hiện các cuộc tấn công đã biết, và 100% không có cảnh báo sai. Tuy nhiên, nó không thể phát hiện các cuộc tấn công dạng mới hoặc không xác định.

Một cách tiếp cận khác đối với phương pháp sử dụng chữ kí đó là xác định chữ kí của các trang web được theo dõi. Từ các trang web ở trạng thái bình thường, thu thập các kí tự được coi là đặc trưng và rất hiếm khi thay đổi lưu nó lại làm “chữ kí” của trang web. Sau một khoảng thời gian hệ thống sẽ duyệt lại trang web và tìm kiếm lại các chữ kí đã xác định trước đó có còn xuất hiện hay không, tùy ngưỡng thiết lập (ví dụ: bị mất 2 trên 10 từ chữ kí) thì hệ thống sẽ đưa ra cảnh báo. Phương pháp này ưu điểm là có thể phát hiện được các cuộc tấn công chưa biết, nhưng sẽ gây nhiều cảnh báo giả khi trang web có sự thay đổi lớn, hoặc kẻ tấn công không xóa quá nhiều chữ kí trên trang web.

1.1.3.3. Phương pháp so sánh sự khác biệt

Mặt khác, phương pháp phát hiện dựa trên sự bất thường trước tiên sẽ xây dựng một “hồ sơ” từ thông tin của các trang được giám sát của một trang web đang ở trong điều kiện hoạt động bình thường. Sau đó, các trang được theo dõi để trích xuất thông tin và sau đó so sánh với hồ sơ để tìm kiếm sự khác biệt. Một cuộc tấn công thay đổi giao diện web được phát hiện nếu tìm thấy bất kỳ sự khác biệt đáng chú ý nào và cảnh báo tấn công được nâng lên. Ưu điểm chính của cách tiếp cận này là nó có khả năng phát hiện các cuộc tấn công mới hoặc chưa biết. Tuy nhiên, rất khó để quyết định ngưỡng phát hiện giữa trang web được giám sát và cấu hình vì nội dung của các trang web động thay đổi thường xuyên.

Phương pháp so sánh sự khác biệt sử dụng công cụ DIFF thường có sẵn trên môi trường Linux và UNIX. DIFF được sử dụng để so sánh nội dung hiện tại của trang web và nội dung của nó được lưu trữ trong hồ sơ để tìm các thay đổi. Điều khó khăn nhất là xác định một ngưỡng bất thường làm đầu vào cho quá trình giám sát mỗi trang web. Tóm lại, kỹ thuật so sánh chênh lệch tương đối hiệu quả đối với hầu hết các trang web động nếu ngưỡng bất thường được chọn chính xác.

DOM là một API xác định cấu trúc của các trang web hoặc cấu trúc HTML. DOM có thể được sử dụng để quét và phân tích cấu trúc trang web. Phân tích DOM tree có thể được sử dụng để phát hiện thay đổi cấu trúc trang web, thay vì những thay đổi trong nội dung trang web. Đầu tiên, cấu trúc trang web được trích xuất từ trang web trong điều kiện làm việc bình thường và được lưu trữ trong “hồ sơ”. Sau đó, trích xuất cấu trúc trang của trang được giám sát và đối sánh với cấu trúc trang trong hồ sơ để tìm kiếm sự khác biệt. Nếu một sự khác biệt đáng chú ý được tìm thấy giữa các cấu trúc trang, một cảnh báo sẽ được đẩy ra. Nói chung, kỹ thuật này hoạt động tốt cho các trang web có cấu trúc ổn định. Tuy nhiên nó không thể phát hiện sửa đổi trái phép nội dung trong các trang web.

1.1.3.4. Phương pháp ứng dụng công nghệ học sâu

Các công nghệ học sâu hiện nay đã có một bước phát triển lớn và ứng dụng được rất nhiều trong đời sống. Các thuật toán phát hiện sự bất thường được ngày càng có độ tin cậy cao, đây đang là một trong những phương pháp mới cho giải pháp phát hiện và dự đoán các trang web bị tấn công thay đổi giao trong thời điểm hiện nay. Cốt lõi của hướng tiếp cận này là áp dụng các bài toán phát hiện bất thường/ngoại lệ hay phân loại ảnh số trong học sâu để phân tích giao diện của trang web.

Ưu điểm của phương pháp này là có thể phát hiện ra được các cuộc tấn công mới, bất kể đó là trang web động hay tĩnh, và có thể phát hiện sửa đổi trái phép nội dung. Nhưng nhược điểm là cần có hệ thống cấu hình cao cho việc huấn luyện mô hình học máy, các thuật toán luôn có tỉ lệ bỏ sót nhất định, và rất

khó để xác định xác định một ngưỡng bất thường làm đầu vào cho quá trình giám sát mỗi trang web.

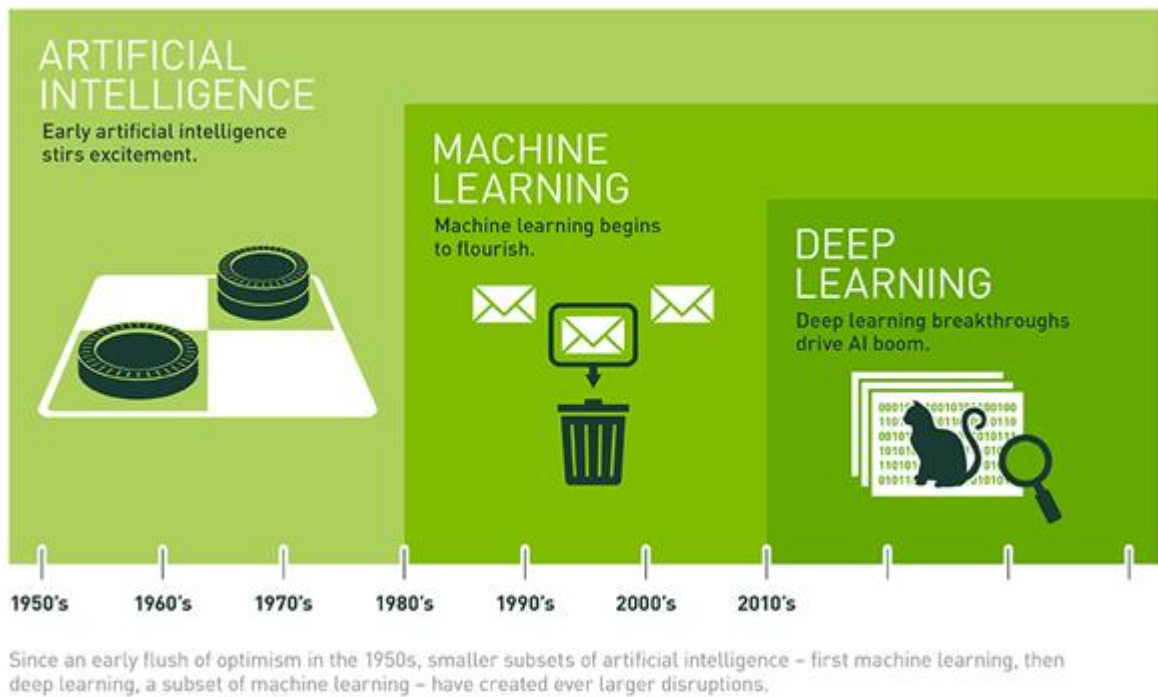
1.2. Học sâu và bài toán phát hiện tấn công thay đổi giao diện web

1.2.1. Học máy

Học máy (Machine Learning) là một nhánh của trí tuệ nhân tạo (AI) tập trung vào việc xây dựng các hệ thống có khả năng học hỏi từ dữ liệu và cải thiện hiệu suất theo thời gian mà không cần lập trình cụ thể. Được phát triển từ giữa thế kỷ 20, học máy đã trải qua nhiều giai đoạn phát triển quan trọng, bắt đầu từ các thuật toán cơ bản như hồi quy tuyến tính, phân loại và cụm dữ liệu, đến các mô hình phức tạp hơn như mạng nơ-ron nhân tạo và các phương pháp học không giám sát.

Một bước tiến lớn trong lĩnh vực học máy là sự ra đời của học sâu (Deep Learning). Học sâu là một bộ con của học máy, sử dụng các mạng nơ-ron sâu có nhiều tầng để mô phỏng cách con người học hỏi từ dữ liệu. Mạng nơ-ron sâu cho phép hệ thống nhận dạng và trích xuất các đặc trưng từ dữ liệu một cách hiệu quả, đặc biệt là trong các tác vụ như nhận dạng hình ảnh, xử lý ngôn ngữ tự nhiên và chẩn đoán y tế. Các mô hình học sâu nổi bật bao gồm CNN (mạng nơ-ron tích chập) dành cho xử lý hình ảnh và RNN (mạng nơ-ron tái phản hồi) dành cho xử lý chuỗi thời gian.

Sự phát triển của học máy và học sâu được thúc đẩy mạnh mẽ bởi sự gia tăng của dữ liệu lớn (Big Data) và cải tiến về phần cứng, đặc biệt là trong công nghệ GPU giúp tăng tốc độ xử lý. Các công ty công nghệ hàng đầu như Google, Facebook, Amazon và Microsoft đều đầu tư mạnh vào nghiên cứu và ứng dụng AI, tạo ra những đột phá mang tính cách mạng. Các ứng dụng của học máy và học sâu không chỉ giới hạn trong lĩnh vực công nghệ, mà còn mở rộng sang y tế, tài chính, tiếp thị, và thậm chí cả nghệ thuật.



Hình 1.3. Mối quan hệ giữa AI, Machine Learning và Deep Learning

1.2.2. Học sâu

1.2.2.1. Giới thiệu

Học sâu là một nhánh của học máy tập trung vào việc sử dụng các mạng nơ-ron nhân tạo có nhiều lớp để mô hình hóa và hiểu các mẫu dữ liệu phức tạp, được gọi là mạng nơ-ron sâu, để mô phỏng khả năng ra quyết định phức tạp của não người. Một trong những ưu điểm nổi bật của học sâu là khả năng tự động học và trích xuất các đặc trưng từ dữ liệu thô mà không cần phải trải qua quá trình tiền xử lý phức tạp do con người thực hiện. Học sâu đã và đang được ứng dụng rộng rãi trong nhiều lĩnh vực, bao gồm nhận dạng giọng nói, thị giác máy tính, xử lý ngôn ngữ tự nhiên, xe tự lái, và nhiều ngành công nghiệp khác.

1.2.2.2. Một số mô hình học sâu nổi tiếng

- AlexNet

AlexNet, do Alex Krizhevsky, Ilya Sutskever và Geoffrey Hinton giới thiệu vào năm 2012, đánh dấu một bước ngoặt trong thị giác máy tính khi giành chiến thắng thuyết phục trong cuộc thi ImageNet. AlexNet bao gồm 5 lớp tích chập xen kẽ với các lớp pooling, kết hợp cùng 3 lớp kết nối đầy đủ, đã trở thành chuẩn mực nhờ khả năng nhận diện hình ảnh mạnh mẽ. Nhờ sử dụng hàm kích

hoạt ReLU và kỹ thuật dropout, AlexNet giảm thiểu hiện tượng quá khớp, cải thiện hiệu quả và tốc độ huấn luyện. Đây chính là mạng tiên phong mở đầu cho sự phát triển vượt bậc của các mô hình Deep Learning trong phân loại và nhận dạng ảnh.

- VGGNet

Mạng VGG, phát triển bởi Visual Geometry Group tại Đại học Oxford, nhấn mạnh vào việc mở rộng chiều sâu của mạng bằng cách sử dụng các lớp tích chập nhỏ kích thước 3×3 . VGGNet, đặc biệt là hai kiến trúc nổi tiếng VGG16 và VGG19, đã đạt được độ chính xác cao trong phân loại ảnh mà vẫn đảm bảo tính dễ hiểu và ổn định. Độ sâu của VGGNet giúp mạng có khả năng học các đặc trưng phức tạp hơn trong hình ảnh, từ đó tạo ra kết quả chính xác và chi tiết. Mô hình VGGNet đã trở thành nền tảng quan trọng trong các ứng dụng nhận diện ảnh, phân đoạn và phát hiện đối tượng.

- ResNet

Residual Network (ResNet), phát triển bởi Microsoft Research vào năm 2015, là một bước tiến lớn trong việc giải quyết vấn đề gradient tiêu hao (vanishing gradient) thường gặp ở các mạng rất sâu. ResNet giới thiệu kiến trúc “residual block,” trong đó thông tin được truyền thẳng qua nhiều lớp mà không bị mất mát. Cấu trúc này giúp ResNet có thể đạt độ sâu lớn mà không cần phải tăng số lượng tham số phức tạp. Các phiên bản của ResNet như ResNet50 và ResNet101 trở nên phổ biến nhờ khả năng hoạt động mạnh mẽ và ổn định, đồng thời đóng vai trò nền tảng cho các mô hình tiên tiến như Mask R-CNN và YOLO.

- Inception (GoogLeNet)

Được Google giới thiệu, mạng Inception (hay GoogLeNet) là một trong những mạng đầu tiên sử dụng mô-đun tích hợp các lớp tích chập kích thước khác nhau trong cùng một khối, giúp mạng có thể học các đặc trưng ở nhiều kích cỡ khác nhau. Mạng Inception đã cải thiện độ chính xác trong các bài toán

phân loại và nhận dạng hình ảnh, trong khi vẫn duy trì được hiệu suất tính toán cao nhờ kỹ thuật tối ưu hóa. Các phiên bản như InceptionV3 và InceptionV4 đã tối ưu hơn nữa, cho phép ứng dụng rộng rãi trong phân loại và nhận diện đối tượng với quy mô lớn.

- MobileNet

MobileNet được thiết kế đặc biệt để tối ưu hóa cho các thiết bị di động và các hệ thống hạn chế tài nguyên tính toán. MobileNet sử dụng kỹ thuật tích chập có thể tách theo chiều sâu, giảm thiểu đáng kể số phép tính so với các mạng thông thường mà không làm giảm đáng kể độ chính xác. Nhờ vào tính nhẹ và hiệu quả, MobileNet được triển khai rộng rãi trong các ứng dụng di động và hệ thống nhúng, giúp xử lý các tác vụ như phân loại ảnh và nhận diện đối tượng trong thời gian thực.

- DenseNet

DenseNet, hay Densely Connected Convolutional Network, là mạng được thiết kế để tối ưu khả năng truyền gradient bằng cách kết nối mỗi lớp với tất cả các lớp trước đó. Kiến trúc này giúp mạng tận dụng tối đa các đặc trưng từ nhiều lớp trước, từ đó cải thiện khả năng học mà không cần phải tăng độ sâu đáng kể. DenseNet phù hợp với các bài toán yêu cầu độ chính xác cao nhưng lại tiết kiệm tài nguyên, là lựa chọn hàng đầu cho các ứng dụng phân loại ảnh và nhận diện đối tượng.

- RNN và LSTM

RNN (Recurrent Neural Networks) là kiến trúc mạng lý tưởng cho dữ liệu chuỗi thời gian, như văn bản hoặc tín hiệu, nhờ khả năng ghi nhớ trạng thái trước. Tuy nhiên, RNN gặp phải vấn đề gradient tiêu hao khi chuỗi quá dài. Để khắc phục điều này, LSTM (Long Short-Term Memory) đã ra đời, bổ sung thêm các cơ chế cổng để điều chỉnh thông tin quan trọng cần lưu trữ hoặc loại bỏ. Cả RNN và LSTM đều có đóng góp lớn trong xử lý ngôn ngữ tự nhiên, dịch máy, và phân tích chuỗi thời gian như dự đoán giá cổ phiếu.

- GAN

Generative Adversarial Networks (GAN) được giới thiệu bởi Ian Goodfellow vào năm 2014, gồm hai mạng cạnh tranh nhau là Generator (mạng tạo dữ liệu giả) và Discriminator (mạng phân biệt thật - giả). Nhờ vào cơ chế cạnh tranh, GAN có khả năng tạo ra dữ liệu giả rất gần giống với dữ liệu thật. GAN được ứng dụng rộng rãi trong tạo hình ảnh và video giả, nâng cao chất lượng ảnh, đồng thời có khả năng tạo ra các nội dung giả mạo (deepfake) hay hỗ trợ trong lĩnh vực sáng tạo nội dung.

Những mạng Deep Learning này đã góp phần tạo nên nền tảng cho nhiều ứng dụng tiên tiến trong AI và học máy hiện đại. Với cấu trúc và mục đích sử dụng riêng, các mạng này đáp ứng được nhu cầu đa dạng trong nhận diện hình ảnh, xử lý ngôn ngữ, học tăng cường và nhiều lĩnh vực khác. Chúng không chỉ là minh chứng cho sự phát triển vượt bậc của trí tuệ nhân tạo mà còn mở ra những hướng phát triển mới cho tương lai.

1.2.2.3. Bài toán phát hiện tấn công thay đổi giao diện web

Cách tiếp cận dễ dàng nhất của phương pháp ứng dụng học sâu đó là phân tích ảnh giao diện của trang web. Để làm được công việc này, có thể ứng dụng một số kỹ thuật toán trong học sâu như phát hiện bất thường/ngoại lệ và kỹ thuật phân loại ảnh.

Đối với các thuật toán phát hiện bất thường/ngoại lệ, từ các bức ảnh giao diện của trang web được theo dõi, nghiên cứu sẽ huấn luyện một mô hình để trích xuất các đặc trưng trên giao diện của trang web ở trạng thái bình thường. Sau đó dựa vào mô hình được huấn luyện, hệ thống sẽ theo dõi xem giao diện có sự thay đổi bất thường nào không và đưa ra các mức cảnh báo. Ưu điểm của phương pháp này là không cần phân tích dữ liệu về trang web đã bị tấn công. Nhưng nhược điểm là dễ có cảnh báo sai đối với các thay đổi hợp lệ, và khó xác định một ngưỡng thay đổi để cảnh báo.

Đối với thuật toán phân loại ảnh, nghiên cứu sẽ thu thập một lượng lớn dữ liệu có gán nhãn của trang web ở trạng thái bình thường và trang web bị tấn

công thay đổi giao diện, sau đó sẽ huấn luyện cho mô hình có thể phân loại được tập dữ liệu này, dựa vào mô hình được huấn luyện nghiên cứu sẽ theo dõi xem giao diện của trang web đang ở trạng thái nào và đưa ra cảnh báo.

1.2.2.4. Các phương pháp đánh giá mô hình phân lớp

Đánh giá là yếu tố chính trong việc đo hiệu suất phân loại và hướng dẫn mô hình hóa phân loại. Có ba giai đoạn chính của quá trình phân loại đó là giai đoạn huấn luyện, giai đoạn xác nhận và giai đoạn thử nghiệm. Để huấn luyện mô hình lập trình viên sử dụng các mẫu đầu vào được gọi là dữ liệu huấn luyện. Trong giai đoạn này, các tham số của mô hình phân loại được điều chỉnh. Mục tiêu của các thuật toán học máy là học từ dữ liệu huấn luyện để dự đoán nhãn cho dữ liệu chưa biết tới. Các lỗi huấn luyện (training error) đo lường mức độ phù hợp với mô hình được huấn luyện, tức là training error càng nhỏ, xác suất dự đoán nhãn của dữ liệu mới càng chính xác.

Dựa theo số lượng các lớp, có thể chia thành hai mô hình phân lớp, đó là phân lớp nhị phân chỉ có hai lớp và phân loại nhiều lớp trong đó số lượng lớp lớn hơn hai. Giả sử với vấn đề phân loại nhị phân - với nhãn P đại diện cho lớp dương tính (positive) và nhãn N đại diện cho lớp âm tính (negative). Mô hình phân loại đã được huấn luyện sử dụng để dự đoán một mẫu chưa biết, phân loại chúng thành Positive hoặc Negative. Mô hình phân loại này tạo ra đầu ra liên tục hoặc rời rạc.

Hình 1.4 thể hiện bốn đầu ra đại diện cho các khả năng dự đoán của ma trận nhầm lẫn (confusion matrix) kích thước 2×2 . Nếu mẫu là Positive và được phân loại là Positive, tức là mẫu được phân loại chính xác thì mẫu được tính là dương tính thật (True Positive - TP), nếu mẫu là Negative nhưng không được phân loại là âm tính, nó được coi là lỗi âm tính giả (False Negative - FN). Nếu mẫu Negative và được phân loại là âm tính thì nó được coi là âm tính thật (True Negative - TN), nếu nó được phân loại là Positive, nó được tính là dương tính giả (False Positive - FP).

		Actual Values	
		Positive (1)	Negative (0)
Predicted Values	Positive (1)	TP	FP
	Negative (0)	FN	TN

Hình 1.4. Confusion matrix (2 nhãn)

Trong bài toán phân loại đa lớp, giả sử ba lớp (A, B và C). Hình 1.5 hiển thị ma trận nhầm lẫn, với TP_A là số mẫu dương tính thật trong lớp A, nghĩa là mẫu được phân loại chính xác là loại A và EAB nghĩa là các mẫu loại A được phân loại thành là loại B, tức là các mẫu được xác định sai. Do đó, âm tính giả trong lớp A (FNA) là tổng của EAB và EAC ($FNA = EAB + EAC$) hay tổng của tất cả các mẫu lớp A được phân loại không chính xác. Trong khi đó dương tính giả cho bất kỳ dự đoán nào nằm trong một hàng sẽ phản ánh tổng của tất cả các lỗi trong hàng đó ($FPA = EBA + ECA$). Với ma trận nhầm lẫn kích thước $m \times m$ có m điểm phân loại đúng và $m^2 - m$ điểm phân loại sai.

		Lớp thực tế		
		A	B	C
Lớp dự đoán	A	TP_A	E_{BA}	E_{CA}
	B	E_{AB}	TP_B	E_{CB}
	C	E_{AC}	E_{BC}	TP_C

Hình 1.5. Confusion matrix (3 nhãn)

Độ chính xác (accuracy) là một trong những phương pháp được sử dụng phổ biến nhất tính hiệu suất phân loại và được định nghĩa là tỷ lệ giữa các mẫu được phân loại chính xác với tổng số mẫu với công thức:

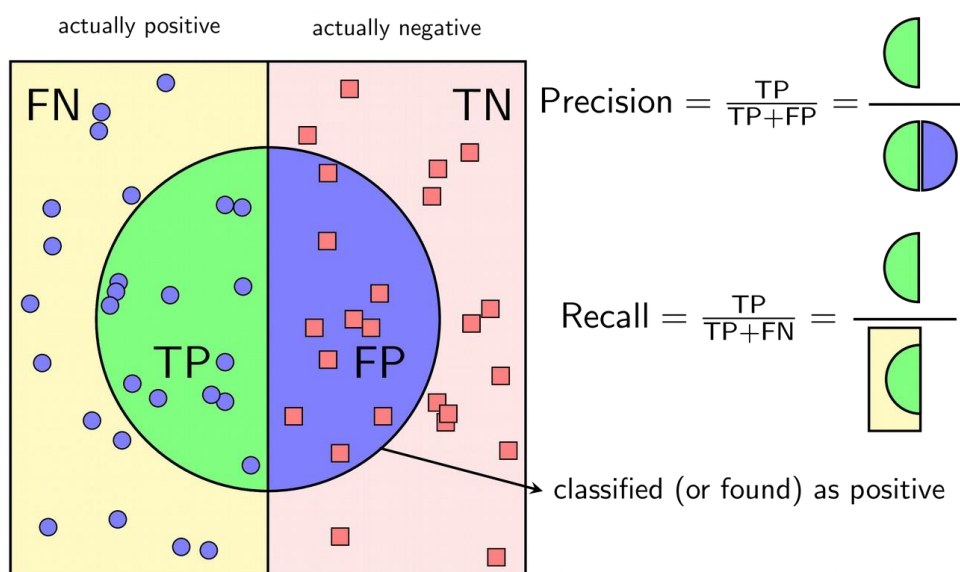
$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Trong đó P và N chỉ ra số lượng mẫu dương tính và âm tính tương ứng. Ngoài ra, tỷ lệ lỗi (Error rate - ERR) hoặc tỷ lệ phân loại sai đại diện cho số lượng mẫu được xác định sai từ cả hai loại dương tính và âm tính và nó được tính toán:

$$ERR = 1 - Accuracy = \frac{FP + FN}{TP + TN + FP + FN}$$

Số liệu về độ chính xác và tỷ lệ lỗi đều dễ bị ảnh hưởng bởi dữ liệu mất cân bằng, tức là khi phân bố dữ liệu giữa các lớp không đồng đều. Nếu chỉ dựa vào chỉ số độ chính xác, ta chỉ biết được có bao nhiêu phần trăm dữ liệu được phân loại đúng mà không thể xác định cụ thể cách mỗi loại được phân loại ra sao, lớp nào được phân loại chính xác nhiều nhất và dữ liệu của lớp nào thường bị nhầm sang lớp khác. Trong các bài toán phân loại với tập dữ liệu có sự chênh lệch lớn giữa các lớp, chỉ số độ chính xác không đủ công bằng để đánh giá. Khi đó, cần sử dụng các phương pháp đánh giá hiệu quả khác như Precision và Recall.

Với bài toán phân loại nhị phân, coi một trong hai lớp là dương tính, lớp còn lại là âm tính. Tỷ lệ cảnh báo đúng (precision) được định nghĩa là tỷ lệ số điểm xác định đúng thực là dương tính trong số tất cả các điểm được phân loại là dương tính còn tỷ lệ phát hiện (recall) được định nghĩa là tỷ lệ số điểm xác định đúng là dương tính trong số những điểm thực sự là dương tính.



Hình 1.6. Tính giá trị Precision, Recall

Tỉ lệ phát hiện và tỉ lệ cảnh báo đúng đều nằm trong khoảng từ 0 đến 1. Tỉ lệ cảnh báo đúng cao thể hiện độ chính xác cao của các điểm được tìm thấy. Tỉ lệ phát hiện cao cho biết tỷ lệ phân loại đúng cao, tức là tỷ lệ bỏ sót các điểm thực sự dương tính là thấp. Khi Precision = 1, mọi điểm được tìm thấy đều là dương tính, nghĩa là không có điểm âm tính nào lẫn vào. Tuy nhiên, Precision = 1 không đảm bảo rằng mô hình là tốt, bởi vì điều này không chắc chắn rằng mô hình đã tìm được tất cả các điểm dương tính hay chưa. Nếu một mô hình chỉ tìm đúng một điểm dương tính mà nó chắc chắn nhất thì không thể coi đó là mô hình tốt. Khi Recall = 1, mọi điểm dương tính đều được tìm thấy, nhưng điều này không đo lường có bao nhiêu điểm âm tính bị nhầm lẫn vào. Nếu mô hình phân loại tất cả các điểm là dương tính thì chắc chắn Recall = 1, nhưng đây là một mô hình rất kém. Một mô hình phân loại tốt là mô hình có cả tỉ lệ phát hiện và tỉ lệ cảnh báo đúng đều cao, tức là cả hai chỉ số này càng gần 1 càng tốt.

Ngoài ra, trong nhiều trường hợp, chỉ số F1 được xem là chỉ số đánh giá mô hình một cách hiệu quả nhất. Chỉ số F1 là trung bình điều hòa của Precision và Recall, với điều kiện cả hai đại lượng này đều khác không:

$$\frac{2}{f1} = \frac{1}{Precision} + \frac{1}{Recall} \text{ hay } f1 = 2 * \frac{Precision * Recall}{Precision + Recall}$$

Chỉ số f1 có giá trị nằm trong nửa khoảng (0,1]. F1 càng cao, mô hình càng ổn định. Như vậy F1 được dùng khi cần đánh giá sự đồng điệu của cả tỉ lệ phát hiện và tỉ lệ cảnh báo đúng, nói cách khác mô hình huấn luyện vừa nhạy, vừa có độ chính xác cao.

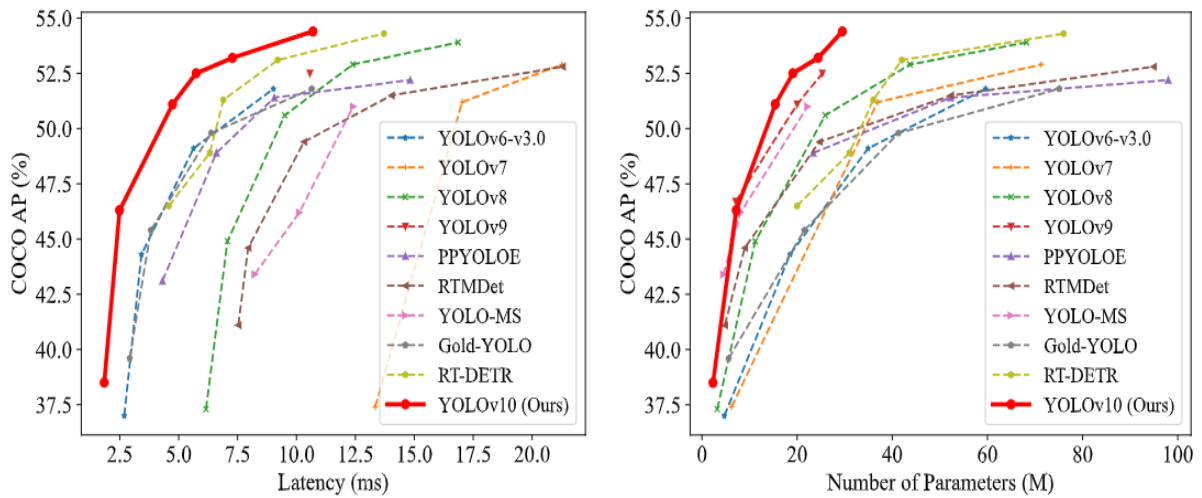
1.3. Mô hình YOLO

1.3.1. Giới thiệu

Phát hiện đối tượng thời gian thực đã trở thành yếu tố quan trọng trong nhiều ứng dụng khác nhau, từ xe tự lái, robot, giám sát video, cho đến thực tế tăng cường. Trong số các thuật toán phát hiện đối tượng khác nhau, hệ thống YOLO (You Only Look Once) nổi bật nhờ sự cân bằng tuyệt vời giữa tốc độ và độ chính xác, cho phép nhận diện nhanh chóng và đáng tin cậy các đối tượng trong hình ảnh. Kể từ khi ra đời, dòng thuật toán YOLO đã phát triển qua nhiều

phiên bản, mỗi phiên bản đều xây dựng dựa trên các phiên bản trước đó để khắc phục hạn chế và nâng cao hiệu suất (Hình 1.7).

YOLO chia hình ảnh đầu vào thành một lưới (grid) và dự đoán bounding box cũng như xác suất lớp cho mỗi ô trong lưới. Điểm mạnh của YOLO là khả năng nhận diện đối tượng nhanh chóng và chính xác trong thời gian thực, phù hợp cho các ứng dụng yêu cầu hiệu suất cao như giám sát giao thông, hệ thống an ninh, và xe tự lái.

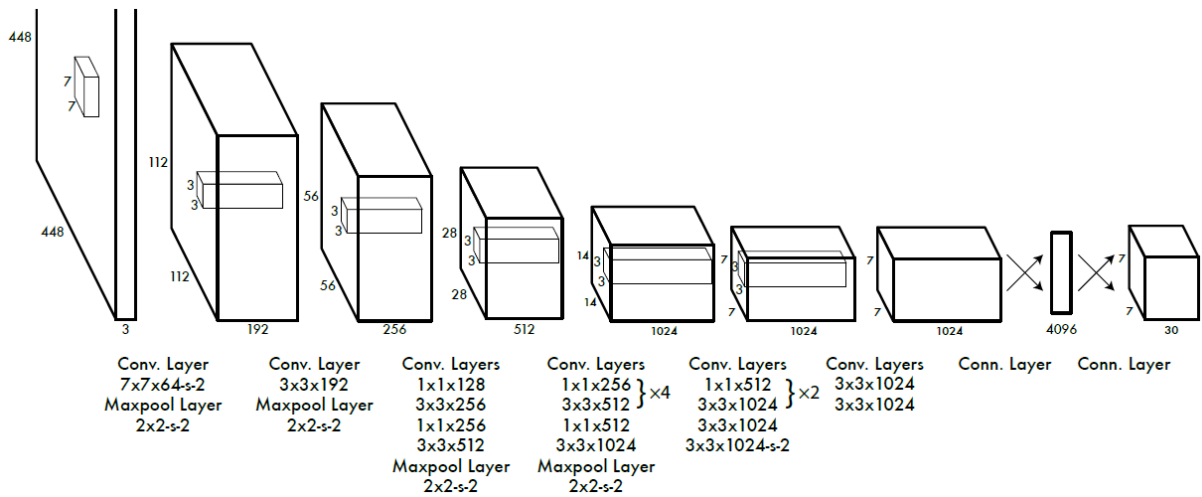


Hình 1.7. Sự phát triển qua các phiên bản YOLO

1.3.1.1. Sự phát triển của YOLO từ v1 đến v10

1) YOLOv1 (2015)

YOLOv1 đánh dấu bước khởi đầu của YOLO với một kiến trúc mạng nơ-ron tích chập đơn giản gồm 24 lớp tích chập và 2 kết nối đầy đủ. Mô hình này chia hình ảnh thành một lưới và dự đoán bounding boxes cùng xác suất các lớp C cho mỗi ô trong lưới. Mặc dù đạt được tốc độ nhanh chóng vượt trội so với các phương pháp phát hiện đối tượng trước đó, YOLOv1 gặp khó khăn trong việc phát hiện các đối tượng nhỏ và các đối tượng gần nhau, do bị giới hạn bởi độ phân giải chia lưới.



Hình 1.8. Kiến trúc mô hình YOLOv1

Kiến trúc mạng YOLOv1 được lấy ý tưởng từ mô hình GoogLeNet cho phân loại ảnh. Nó gồm có 24 lớp tích chập dùng để trích xuất các features từ bức ảnh, theo sau bởi 2 lớp kết nối đầy đủ để dự đoán đầu ra xác suất và tọa độ. Thay vì sử dụng mô-đun khởi động trong GoogLeNet, YOLO chỉ sử dụng lớp giảm có kích thước 1 x 1 theo sau bởi lớp tích chập có kích thước 3 x 3. Lớp cuối cùng của mạng là các dự đoán ở dạng tensor với kích thước 7 x 7 x 30. Các lớp tích chập được huấn luyện trước trên tập dữ liệu ImageNet cho việc phân loại ảnh với độ phân giải 224 x 224, rồi sau đó gấp đôi độ phân giải cho việc detection.

YOLOv1 áp dụng hàm thất thoát SSE (Sum of Square Error – Tổng bình phương sai số), dễ dàng để tối ưu nhưng tồn tại một số nhược điểm:

- SSE tính toán lỗi vị trí và lỗi phân loại như nhau, mặc dù bản chất của chúng khác nhau, dẫn đến kết quả không chính xác.
- Trong một hình ảnh, rất nhiều ô lưới không chứa đối tượng, khiến điểm tin cậy của các ô này về gần 0. Số lượng lớn các ô lưới không chứa đối tượng sẽ làm giảm độ dốc từ những ô chứa đối tượng, gây ra sự không ổn định và kết thúc sớm quá trình huấn luyện mô hình.
- SSE không phân biệt sai số giữa các hộp lớn và nhỏ, trong khi sự sai lệch nhỏ ở hộp lớn ít tác động hơn so với hộp nhỏ.

Để khắc phục những điểm yếu này, YOLO đưa ra các giải pháp cải tiến:

- Tăng trọng số lỗi cho dự đoán tọa độ hộp giới hạn bằng cách sử dụng tham số $\lambda_{coord} = 5$, nhằm nâng mức độ chính xác của dự đoán tọa độ.
- Giảm trọng số lỗi cho dự đoán độ tin cậy của các hộp giới hạn không chứa vật thể bằng tham số $\lambda_{noobj} = 5$, để giảm sai số từ các ô lưới không có đối tượng.
- Thay vì dự đoán trực tiếp chiều rộng và chiều cao của hộp giới hạn, mô hình dự đoán căn bậc hai của các giá trị này để cải thiện độ chính xác.

Một khía cạnh quan trọng khác, YOLO dự đoán nhiều hộp giới hạn cho mỗi ô lưới. Trong khi huấn luyện, chỉ một hộp giới hạn sẽ chịu trách nhiệm cho việc dự đoán đối tượng trong ô đó, được chọn dựa trên giá trị IOU cao nhất với hộp chân thực (ground truth box). Điều này giúp chuyên môn hóa các bộ dự đoán hộp, mỗi bộ sẽ trở nên tốt hơn trong việc dự đoán kích thước, tỷ lệ khung hình và lớp đối tượng cụ thể, nâng cao độ chính xác tổng thể.

2) YOLOv2 (2016)

Năm 2017, Joseph Redmon (Sinh viên sau đại học tại Đại học Washington) và Ali Farhadi (trưởng nhóm PRIOR tại Viện Allen về AI) đã công bố bài báo YOLO9000: Tốt hơn, Nhanh hơn, Mạnh hơn tại hội nghị CVPR. Các tác giả đã đề xuất hai biến thể YOLO tiên tiến trong bài báo này: YOLOv2 và YOLO9000; cả hai đều giống hệt nhau nhưng khác nhau về chiến lược đào tạo.

YOLOv2 sử dụng mạng dựa trên Darknet-19, một mạng lưới với 19 tầng tích chập. Các kỹ thuật nổi bật được bổ sung bao gồm chuẩn hóa hàng loạt để cải thiện độ ổn định trong quá trình huấn luyện và sử dụng các hộp neo (anchor boxes) như trong Faster R-CNN để phát hiện các đối tượng có kích thước khác nhau hiệu quả hơn. Phiên bản này cũng áp dụng kỹ thuật phân loại độ phân giải cao, tức là huấn luyện mô hình trên hình ảnh có độ phân giải cao hơn để tăng độ chính xác. Kết quả là, YOLOv2 tăng cường khả năng tổng quát hóa và độ chính xác so với YOLOv1, nhưng vẫn giữ được tốc độ suy luận vượt trội.

3) YOLOv3 (2018)

YOLOv3 tiếp tục lộ trình phát triển bằng cách sử dụng Darknet-53, gồm 53 lớp tích chập sâu hơn để trích xuất các đặc trưng phức tạp hơn.

	Type	Filters	Size	Output
	Convolutional	32	3×3	256×256
	Convolutional	64	$3 \times 3 / 2$	128×128
1x	Convolutional	32	1×1	
	Convolutional	64	3×3	
	Residual			128×128
	Convolutional	128	$3 \times 3 / 2$	64×64
2x	Convolutional	64	1×1	
	Convolutional	128	3×3	
	Residual			64×64
	Convolutional	256	$3 \times 3 / 2$	32×32
8x	Convolutional	128	1×1	
	Convolutional	256	3×3	
	Residual			32×32
	Convolutional	512	$3 \times 3 / 2$	16×16
8x	Convolutional	256	1×1	
	Convolutional	512	3×3	
	Residual			16×16
	Convolutional	1024	$3 \times 3 / 2$	8×8
4x	Convolutional	512	1×1	
	Convolutional	1024	3×3	
	Residual			8×8
	Avgpool		Global	
	Connected		1000	
	Softmax			

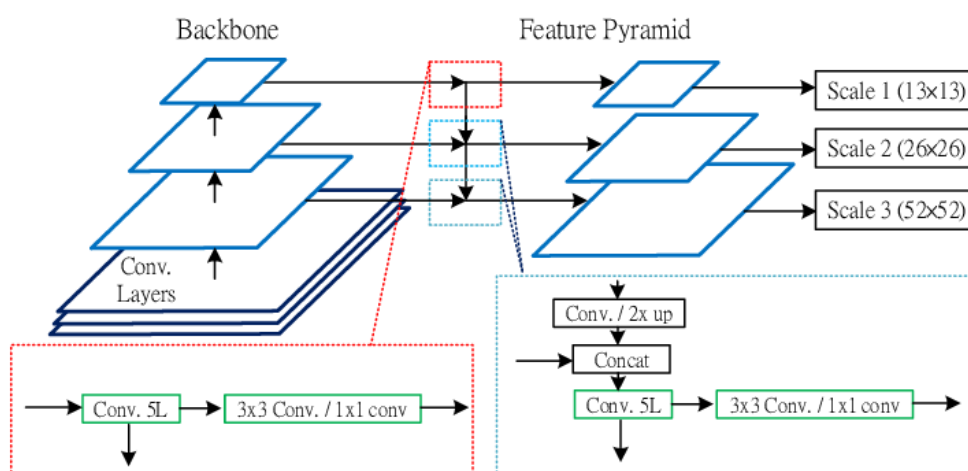
Hình 1.9. Mô hình mạng Darknet-53

Darknet-53 đóng vai trò nền tảng trong YOLOv3, hoạt động như xương sống để trích xuất các đặc trưng mạnh mẽ. Với kiến trúc sâu và khả năng trích xuất tính năng vượt trội, Darknet-53 cung cấp những đặc trưng quan trọng, giúp YOLOv3 phát hiện đối tượng một cách chính xác và nhanh chóng ngay cả trong các tình huống thời gian thực.

Bên cạnh đó, YOLOv3 giới thiệu mô hình "Mạng kim tự tháp tính năng" (FPN), một kiến trúc CNN tiên tiến được thiết kế để phát hiện các đối tượng ở nhiều thang đo khác nhau. FPN xây dựng một kim tự tháp phân cấp của các bản đồ đặc trưng, cho phép mô hình xác định các đối tượng với quy mô khác nhau đồng thời. Điều này đặc biệt hữu ích trong việc phát hiện các đối tượng nhỏ, vì mô hình có thể phân tích và nhận diện các đối tượng này trên nhiều cấp độ thang đo.

Sự kết hợp của Darknet-53 và FPN mang lại sức mạnh ấn tượng cho YOLOv3, cải thiện đáng kể hiệu suất phát hiện đối tượng, đặc biệt là với các đối

tượng nhỏ và ở nhiều quy mô khác nhau. Nhờ vào sự nâng cao này, YOLOv3 không chỉ duy trì được tốc độ xử lý nhanh, mà còn tăng cường độ chính xác trong việc phát hiện đối tượng, phù hợp với các ứng dụng thời gian thực phức tạp..



Hình 1.10. Kiến trúc FPN được áp dụng trong YOLOv3

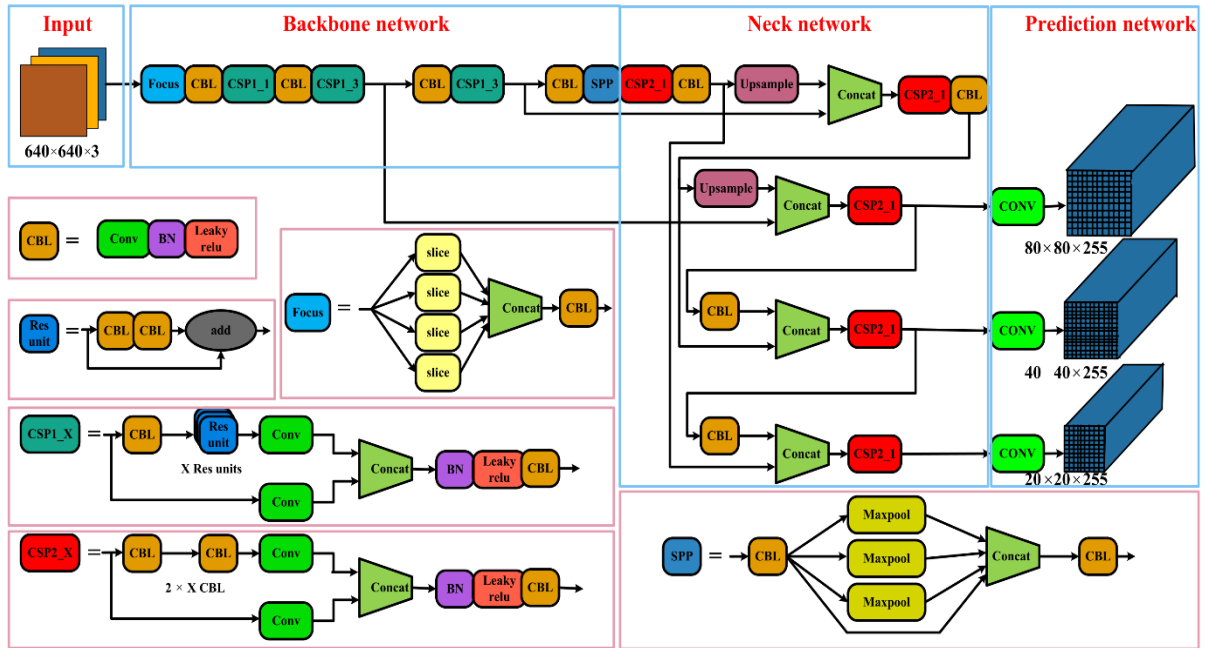
4) YOLOv4 (2020)

YOLOv4 giới thiệu CSPDarknet53, kết hợp CSPNet (Cross-Stage Partial Network) với Darknet53, giúp mô hình hiệu quả hơn trong việc trích xuất và tổng hợp đặc trưng. Hàm kích hoạt Mish được sử dụng thay thế ReLU, giúp tối ưu gradient flow và tăng độ chính xác. Các mô-dun nâng cao như Spatial Pyramid Pooling (SPP) và Path Aggregation Network (PANet) được tích hợp để cải thiện hiệu suất trích xuất đặc trưng đa tỷ lệ. GIoU loss được giới thiệu để tối ưu hóa việc dự đoán bounding boxes chính xác hơn. Những cải tiến này giúp YOLOv4 đạt được độ chính xác cao hơn đồng thời giữ được tốc độ suy luận nhanh.

5) YOLOv5 (2020)

YOLOv5, mặc dù không chính thức từ nhóm nghiên cứu ban đầu, đã được phát triển và duy trì bởi Ultralytics. Viết bằng PyTorch, YOLOv5 cung cấp môi trường triển khai và tinh chỉnh dễ dàng hơn. Điều này bao gồm các biến thể khác nhau (nano, small, medium, large) để đáp ứng các yêu cầu hiệu suất khác nhau. Các kỹ thuật tối ưu hóa tiên tiến như Mosaic và MixUp được sử dụng để cải thiện khả năng tổng quát hoá của mô hình. YOLOv5 đã trở thành một lựa chọn

phổ biến nhờ tính dễ sử dụng, triển khai, và cải thiện độ chính xác so với các phiên bản trước.

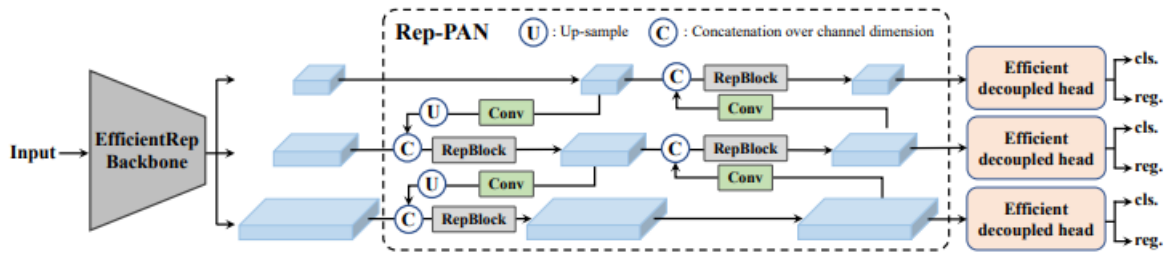


Hình 1.11. Khung kiến trúc cơ bản của YOLOv5

Mô-dun đầu vào trước hết thực hiện việc tăng cường dữ liệu Mosaic trên hình ảnh đầu vào, bao gồm thay đổi tỷ lệ và cắt ngẫu nhiên. Ngay sau đó, hộp neo thích ứng được tính toán để xác định giá trị tối ưu cho hộp neo. Cuối cùng, hình ảnh đầu vào được thay đổi tỷ lệ một cách thích ứng. Mạng xương sống chủ yếu bao gồm cấu trúc focus, cấu trúc CSP và cấu trúc SPP. Cấu trúc focus chia tách hình ảnh đầu vào, khối CSP chủ yếu được sử dụng trong phần trích xuất đặc điểm của CNN để tăng tốc quá trình huấn luyện và dự đoán của mô hình, và cấu trúc SPP mở rộng trường tiếp nhận của mạng. Mạng neck sử dụng cấu trúc của FPN và PAN, nâng cao khả năng hợp nhất đặc điểm của mạng. Mạng dự đoán phát hiện các đối tượng trong hình ảnh và dự đoán danh mục cũng như vị trí của chúng.

6) YOLOv6

YOLOv6 tiếp tục phát triển với mục tiêu tối ưu hóa cả tốc độ và độ chính xác.



Hình 1.12. Kiến trúc mạng YOLOv6

YOLOv6 sử dụng một kiến trúc xương sống hiệu suất cao được gọi là EfficientRep (Hình 1.12), được thiết kế với mục tiêu cải thiện hiệu năng và tốc độ:

- RepBlock: Lớp RepBlock giúp giảm số lượng tham số và tính toán không cần thiết mà vẫn bảo toàn chất lượng của đặc trưng học được.
- NeXtStage: Được tối ưu hóa để vừa duy trì đại diện mạnh mẽ của đối tượng, vừa tối thiểu hóa độ phức tạp tính toán.

Mô hình thực hiện sự tách biệt giữa dòng chảy tính năng (feature flow) và kim tự tháp tính năng (feature pyramid):

- Dòng chảy tính năng giúp đảm bảo thông tin mạnh mẽ xuyên suốt các tầng của mô hình, tăng khả năng hội tụ và đồng thời tối ưu hóa việc chia sẻ thông tin.
- Phương pháp kim tự tháp tính năng giúp cải thiện khả năng nhận diện các đối tượng ở nhiều tỉ lệ khác nhau, tăng hiệu suất phát hiện đối tượng nhỏ và lớn.

Mô-dun tăng cường được giới thiệu để khai thác tối đa thông tin từ các lớp tính năng thấp, giúp cải thiện độ rõ nét của đặc trưng. Mô-dun này tập trung vào việc tối ưu hóa và làm sạch tín hiệu từ các tầng thấp, giúp mạng nơ-ron dễ dàng phân biệt các đối tượng phức tạp hơn. Đồng thời cũng loại bỏ sự cần thiết của anchors. Những cải tiến này của YOLOv6 không chỉ giúp mô hình thực hiện tốt hơn trong nhiệm vụ phát hiện đối tượng mà còn cải thiện đáng kể về tốc độ và

hiệu quả tính toán. Điều này làm cho YOLOv6 trở thành một lựa chọn mạnh mẽ trong các ứng dụng thời gian thực và đòi hỏi sự chính xác cao.

7) YOLOv7

YOLOv7 đã đánh dấu một cuộc cách mạng trong lĩnh vực này với cải tiến vượt bậc cả về độ chính xác lẫn tốc độ, từ 5 FPS lên đến 160 FPS. Những tiến bộ này tập trung vào việc nâng cao hiệu quả và tính mở rộng, nhờ vào sự tích hợp của mạng tổng hợp lớp hiệu quả mở rộng (E-ELAN) và kiến trúc nối kết dễ mở rộng. E-ELAN đóng vai trò quan trọng trong việc quản lý đường dẫn gradient, giúp tăng cường khả năng học tập và hội tụ của mô hình. Kỹ thuật này linh hoạt và có thể áp dụng cho các mô hình với khối tính toán chồng, khéo léo xáo trộn và hợp nhất các tính năng từ nhiều nhóm khác nhau mà vẫn giữ nguyên đường dẫn gradient.

Không giống như các bộ phát hiện đối tượng thời gian thực truyền thống tập trung vào tối ưu hóa kiến trúc, YOLOv7 tập trung vào tối ưu hóa quy trình đào tạo. Điều này bao gồm các mô-đun và phương pháp tối ưu hóa được thiết kế để cải thiện độ chính xác của việc phát hiện đối tượng mà không làm tăng chi phí suy luận, một khái niệm được gọi là Trainable Bag-of-Freebies (BoF). BoF tập trung vào việc tối ưu hóa mô hình mà không tăng chi phí tính toán trong quá trình suy luận. BoF giúp mô hình học được từ các dữ liệu có độ phân giải khác nhau và tích hợp các cơ chế chú ý có thể thay thế vào nhiều vị trí trong mạng. Các cơ chế chú ý như SE (Squeeze-and-Excitation), CBAM (Convolutional Block Attention Module) được tích hợp một cách linh hoạt vào mô hình để tăng cường khả năng tập trung vào các vùng quan trọng của ảnh. Một trong những kỹ thuật này là khái niệm tích chập được tham số hóa lại, được sử dụng trong YOLOv6 và cải tiến hơn bởi YOLOv7 thông qua phương pháp RepConvN để giải quyết các vấn đề với kết nối danh tính trong RepConv 54 và nối DenseNet 55.

Ngoài ra, YOLOv7 còn áp dụng phương pháp gán nhãn thô cho đầu phụ và gán nhãn tinh cho đầu chính. Đầu phụ góp phần vào quá trình đào tạo, trong khi đầu chính tạo ra đầu ra cuối cùng. Một phương pháp khác là chuẩn hóa lô, kết

a) Backbone

YOLOv8 sử dụng xương sống mạng nơ-ron tích chập (CNN) tinh vi được thiết kế để trích xuất các đặc điểm đa tỷ lệ từ hình ảnh đầu vào. Được tối ưu hóa cho cả tốc độ và độ chính xác, xương sống này có thể là phiên bản nâng cao của CSPDarknet hoặc một kiến trúc hiệu quả khác, nắm bắt các bản đồ đặc điểm phân cấp, biểu diễn cả kết cấu cấp thấp và thông tin ngữ nghĩa cấp cao, quan trọng cho việc phát hiện đối tượng chính xác. Nó kết hợp các phép tích chập có thể tách theo chiều sâu hoặc các lớp hiệu quả khác để giảm thiểu chi phí tính toán trong khi giữ nguyên sức mạnh biểu diễn.

b) Neck

Mô-đun neck của YOLOv8 tinh chỉnh và hợp nhất các đặc điểm đa thang đo được trích xuất bởi mạng xương sống. Nó tận dụng phiên bản tối ưu của Path Aggregation Network (PANet), được cải tiến để cải thiện luồng thông tin qua các cấp tính năng khác nhau. Việc tích hợp các đặc điểm đa thang đo này rất quan trọng để phát hiện các đối tượng có kích thước và tỷ lệ khác nhau. Thiết kế PANet nâng cao trong YOLOv8 có thể bao gồm các sửa đổi đối với PANet gốc để tối ưu hóa hơn nữa việc sử dụng bộ nhớ và hiệu quả tính toán.

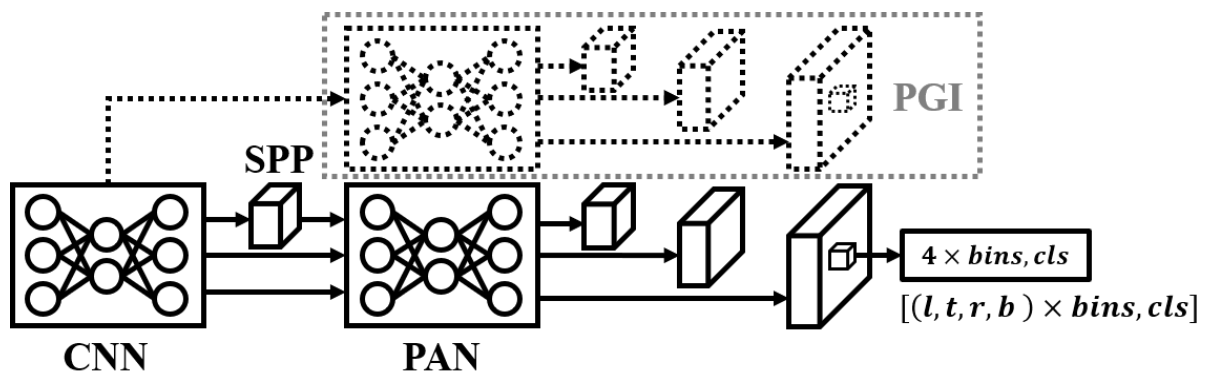
c) Head

Mô-đun head chịu trách nhiệm tạo ra các dự đoán cuối cùng, bao gồm tọa độ hộp giới hạn, điểm tin cậy của đối tượng, và nhãn lớp, từ các đặc điểm đã được tinh chỉnh. YOLOv8 giới thiệu phương pháp không có neo để dự đoán hộp giới hạn, tránh xa các phương pháp dựa trên neo được sử dụng trong các phiên bản YOLO trước đó. Phương pháp không có neo này đơn giản hóa quá trình dự đoán, giảm số lượng siêu tham số và cải thiện khả năng thích ứng của mô hình với các đối tượng có tỷ lệ khung hình và tỷ lệ khác nhau.

Với việc tích hợp những cải tiến kiến trúc này, YOLOv8 nâng cao hiệu suất trong các tác vụ phát hiện đối tượng, mang lại độ chính xác, tốc độ, và tính linh hoạt được cải thiện rõ rệt.

9) YOLOv9

YOLOv9 mang đến những cải tiến vượt bậc so với các phiên bản trước đó, với việc tối ưu hóa toàn bộ kiến trúc mạng nhằm nâng cao cả tính năng và hiệu suất. Thành phần Backbone của YOLOv9 được tinh chỉnh và cải thiện với các mạng nơ-ron tích chập (CNN) mới hoặc công nghệ Vision Transformers, giúp trích xuất đặc điểm đa tỷ lệ từ hình ảnh đầu vào một cách hiệu quả hơn. Mô-đun Neck sử dụng phiên bản nâng cao của Path Aggregation Network (PANet), cải thiện đáng kể luồng thông tin giữa các cấp độ đặc điểm và tích hợp cơ chế attention để tăng cường khả năng hợp nhất thông tin. Đặc biệt, YOLOv9 áp dụng phương pháp không tạo khung trong mô-đun Head, giúp đơn giản hóa quá trình dự đoán và tăng cường độ chính xác. Các kỹ thuật đào tạo mới, bao gồm augmentation và chiến lược tối ưu hóa tiên tiến, cùng với các hàm mất mát mới, đã giảm thiểu quá khớp và cải thiện hiệu năng mô hình. Các cải tiến này không những cải thiện độ chính xác và tính linh hoạt trong việc phát hiện đối tượng ở nhiều kích thước và tỷ lệ khác nhau, mà còn giảm chi phí tính toán và tăng tốc độ xử lý, mang lại hiệu suất vượt trội trong các ứng dụng thực tế.



Hình 1.14. Kiến trúc của YOLOv9

YOLOv9 đã giới thiệu một công nghệ tiên tiến và đáng tin cậy – Programmable Gradient Information (PGI), với kiến trúc được minh họa trong Hình 14. Kiến trúc này tăng cường khả năng diễn giải, độ mạnh mẽ và tính linh hoạt của mô hình. PGI được thiết kế dựa trên các nguyên lý của kiến trúc đảo ngược và thông tin đa cấp, nhằm tối đa hóa lượng dữ liệu gốc mà mô hình có thể

giữ lại, cũng như thông tin cần thiết để hoàn thành các nhiệm vụ mục tiêu. YOLOv9 đã mở rộng ELAN thành G-ELAN và sử dụng nó để chứng minh cách thức PGI có thể đạt được độ chính xác, độ ổn định và tốc độ suy luận vượt trội trên các mô hình có số lượng tham số thấp. Các tính năng nổi bật của YOLOv9 được mô tả dưới đây:

PGI khai thác các đặc điểm của kiến trúc đảo ngược để giải quyết vấn đề tắc nghẽn thông tin trong các mạng nơ-ron sâu. Điều này hoàn toàn khác biệt với các kiến trúc đảo ngược mục đích chung, vốn chỉ tập trung vào việc tối đa hóa thông tin được lưu giữ. Thay vào đó, PGI chia sẻ thông tin được lưu giữ bởi kiến trúc đảo ngược với nhánh chính dưới dạng thông tin phụ trợ, giúp giữ lại tối đa lượng thông tin cần thiết từ dữ liệu gốc cho các nhiệm vụ mục tiêu.

Thông tin phụ trợ đa cấp: PGI đề xuất khái niệm thông tin phụ trợ đa cấp (Multi-level Auxiliary Information), đảm bảo mỗi lớp của nhánh chính giữ lại được lượng thông tin cần thiết cho tất cả các mục tiêu nhiệm vụ càng nhiều càng tốt. Điều này giúp tránh tình trạng mất mát thông tin quan trọng ở các tầng nông, một vấn đề thường gặp ở các phương pháp trước đây, dẫn đến việc thu thập không đủ thông tin ở các tầng sâu hơn. Nhờ khả năng tối ưu hóa việc lưu giữ thông tin dữ liệu gốc, các mô hình được huấn luyện với PGI đạt được hiệu suất mạnh mẽ hơn trong các tập dữ liệu nhỏ, học chuyển giao, học đa nhiệm và dễ dàng thích ứng với các nhiệm vụ mới phát sinh từ nguồn dữ liệu khác.

Tính linh hoạt qua nhiều kiến trúc khác nhau: PGI không chỉ giới hạn trong YOLOv9; nó còn có thể áp dụng cho nhiều kiến trúc khác, bao gồm CNN thông thường, CNN tích chập theo chiều sâu, và bộ biến đổi. Khả năng này còn mở rộng ra các phương pháp xử lý thị giác máy tính khác nhau như anchor-based, anchor-free, và không cần xử lý hậu kỳ, giúp PGI có tính linh hoạt và ứng dụng rộng rãi trong nhiều lĩnh vực khác nhau.

1.3.2. YOLOv10

YOLOv10, ra mắt vào tháng 5 năm 2024 bởi các tác giả thuộc Đại học Thanh Hoa, đại diện cho một bước tiến mới trong lĩnh vực phát hiện đối tượng

theo thời gian thực. Dựa trên những cải tiến từ YOLOv9, YOLOv10 giải quyết những thách thức quan trọng như sự phụ thuộc vào phương pháp ức chế không tối đa (NMS) và sự dư thừa tính toán, những yếu tố trước đây đã giới hạn hiệu suất và hiệu quả của các mô hình YOLO trước đó. Quy trình phát hiện của YOLO gồm hai phần: quá trình mô hình tiến tới và xử lý hậu kỳ bằng NMS, nhưng cả hai vẫn còn nhiều điểm thiếu sót. YOLO sử dụng chiến lược gán nhãn một-đối-nhiều trong huấn luyện, đòi hỏi NMS để chọn lựa dự đoán dương tính tốt nhất, làm chậm tốc độ suy luận và khiến hiệu suất nhạy cảm với các siêu tham số của NMS. Các kiến trúc DETR và chiến lược gán nhãn một-đối-một đã được đề xuất để giải quyết vấn đề này, nhưng chúng thường gây thêm độ trễ suy luận hoặc đạt hiệu suất chưa tối ưu.

Thiết kế kiến trúc mô hình vẫn là một thách thức quan trọng cho YOLO, ảnh hưởng lớn đến độ chính xác và tốc độ. Các chiến lược thiết kế như DarkNet, CSPNet, EfficientRep và ELAN cho phần backbone, và PAN, BiC, GD và RepGFPN cho phần neck, cùng với các kỹ thuật mở rộng mô hình và tái tham số hóa, đã đạt được tiến bộ đáng kể. Tuy nhiên, kiểm tra toàn diện các thành phần của YOLO từ cả góc độ hiệu quả và độ chính xác vẫn còn thiếu, dẫn đến dư thừa tính toán và sử dụng tham số không hiệu quả, làm giảm hiệu suất và hiệu quả của mô hình.

Để giải quyết vấn đề này, các tác giả tập trung cải thiện cả quá trình xử lý hậu kỳ và thiết kế kiến trúc mô hình cho YOLO nhằm nâng cao độ chính xác và tốc độ. Các tác giả giới thiệu chiến lược gán nhãn kép nhất quán để loại bỏ nhu cầu sử dụng NMS, giúp tăng hiệu suất và hiệu quả. Ngoài ra, các tác giả cũng đề xuất chiến lược thiết kế mô hình toàn diện để giảm sự dư thừa tính toán và tăng cường khả năng của mô hình, bao gồm đầu phân loại nhẹ, giảm mẫu tách kênh không gian và các mô-dun tự chú ý.

Kết quả là thế hệ mới của các bộ phát hiện từ đầu đến cuối thời gian thực với nhiều quy mô mô hình khác nhau, YOLOv10-N/S/M/B/L/X. Thí nghiệm

trên COCO cho thấy YOLOv10 vượt trội về cả hiệu suất và hiệu quả so với các mô hình trước đó. Cụ thể:

- YOLOv10-S/X nhanh hơn $1.8\times$ và $1.3\times$ so với RT-DETR-R18/R101,
- YOLOv10-B giảm 46% độ trễ so với YOLOv9-C với cùng hiệu suất. YOLOv10-L/X hiệu quả hơn về sử dụng tham số so với YOLOv8-L/X
- YOLOv10-M ít tham số hơn 23% / 31% so với YOLOv9-M/YOLOv9-MS mà vẫn đạt hiệu suất tương đương.

1.3.2.1. Chiến lược gán nhãn kép nhất quán cho huấn luyện không cần NMS

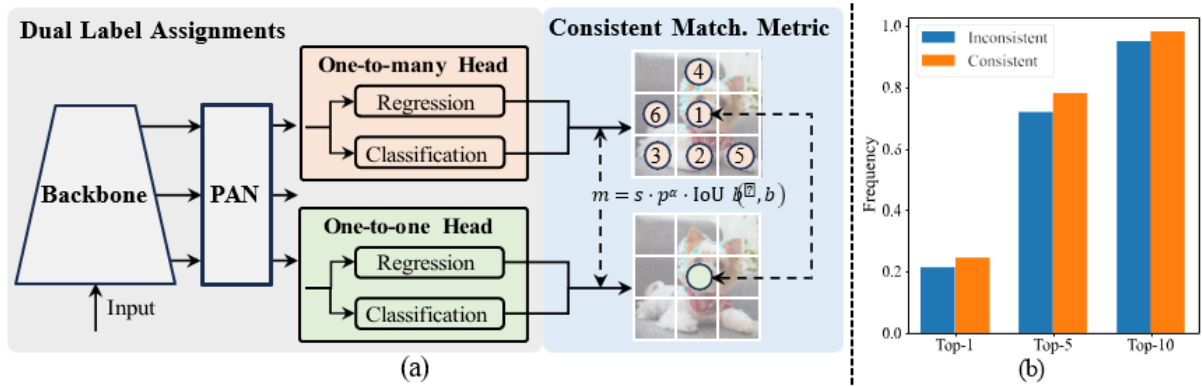
Trong quá trình huấn luyện, các phiên bản YOLO thường sử dụng TAL để phân bổ nhiều mẫu dương tính cho mỗi đối tượng. Việc áp dụng chiến lược gán nhãn một-đối-nhiều mang lại nhiều tín hiệu giám sát phong phú, giúp tối ưu hóa mô hình và đạt hiệu suất vượt trội. Tuy nhiên, điều này khiến YOLO phải dựa vào NMS trong xử lý hậu kỳ, dẫn đến hiệu suất suy luận chưa tối ưu khi triển khai. Trong khi các công trình trước đây đã khám phá việc so khớp một-đối-một để giảm dự đoán dư thừa, thường chúng dẫn đến chi phí suy luận bổ sung hoặc hiệu suất chưa tối ưu. Trong phiên bản mới nhất, YOLOv10 được giới thiệu chiến lược huấn luyện không cần NMS với gán nhãn kép và chỉ số so khớp nhất quán, đạt được cả hiệu suất cao và hiệu quả cạnh tranh.

Gán nhãn kép

Khác với gán nhãn một-đối-nhiều, so khớp một-đối-một chỉ gán một dự đoán cho mỗi đối tượng thực, tránh được NMS trong xử lý hậu kỳ. Tuy nhiên, điều này dẫn đến giám sát yếu, gây giảm độ chính xác và tốc độ hội tụ. May mắn thay, điểm yếu này có thể được bù đắp bằng chiến lược gán nhãn một-đối-nhiều. Để đạt được điều này, các tác giả giới thiệu chiến lược gán nhãn kép cho YOLOs để kết hợp ưu điểm của cả hai phương pháp.

Cụ thể, như minh họa trong Hình 1.15.(a), các tác giả thêm một nhánh so khớp một-đối-một cho YOLOs. Nhánh này giữ cấu trúc giống hệt và áp dụng các mục tiêu tối ưu hóa tương tự như nhánh gán nhãn một-đối-nhiều ban đầu

nhưng sử dụng so khớp một-đối-một để gán nhãn. Trong quá trình huấn luyện, hai nhánh được tối ưu hóa đồng bộ với mô hình, cho phép phần backbone và neck nhận được sự giám sát phong phú từ chiến lược gán nhãn một-đối-nhiều. Trong suy luận, các tác giả bỏ nhánh gán nhãn một-đối-nhiều và sử dụng nhánh một-đối-một để thực hiện dự đoán. Điều này cho phép YOLOs triển khai từ đầu đến cuối mà không tăng thêm chi phí suy luận. Hơn nữa, trong so khớp một-đối-một, các tác giả áp dụng lựa chọn top một, đạt được hiệu suất tương đương với thuật toán Hungarian matching nhưng với thời gian huấn luyện ít hơn.



Hình 1.15. Chiến lược gán nhãn kép nhất quán cho huấn luyện không cần NMS

Trong hình 1.15.(b), tần suất của gán nhãn một-đối-một trong Top-1/5/10 kết quả gán nhãn một-đối-nhiều đối với YOLOv8-S khi áp dụng $\alpha_{2m}=0.5$ và $\beta_{2m}=6$ theo mặc định. Để đảm bảo nhất quán, áp dụng $\alpha_{o2o}=0.5$; $\beta_{o2o}=6$. Để không nhất quán, áp dụng $\alpha_{o2o}=0.5$; $\beta_{o2o}=2$.

Kiến trúc của YOLOv10 được xây dựng dựa trên những ưu điểm của các mô hình YOLO trước đó và đồng thời giới thiệu một số cải tiến quan trọng. Kiến trúc của mô hình bao gồm các thành phần chính sau:

- **Backbone:** Chịu trách nhiệm trích xuất đặc trưng, phần backbone trong YOLOv10 sử dụng phiên bản nâng cao của CSPNet (Cross Stage Partial Network) nhằm cải thiện luồng thông tin và sử dụng ít năng lượng tính toán hơn.
- **Neck:** Neck được thiết kế để kết hợp các đặc điểm từ các cấp độ backbone khác nhau và truyền chúng đến đầu. Nó kết hợp hiệu quả các đặc điểm từ nhiều thang đo bằng cách sử dụng các lớp PAN.

- **One-to-Many Head:** Trong quá trình huấn luyện, phần này tạo ra nhiều dự đoán cho mỗi đối tượng nhằm cung cấp các tín hiệu giám sát phong phú, giúp cải thiện độ chính xác trong quá trình học.
- **One-to-One Head:** Trong quá trình suy luận, phần này tạo ra một dự đoán tốt nhất duy nhất cho mỗi đối tượng, loại bỏ sự cần thiết của NMS, từ đó giảm độ trễ và tăng cường hiệu suất.

Trong hình 1.15, các tác giả đề xuất phương pháp loại bỏ NMS bằng cơ chế gán nhãn kép (Dual Label Assignments). Sau khi dữ liệu đi qua lớp Backbone và lớp PAN, dữ liệu sẽ được đưa qua lớp một-đối-nhiều để dự đoán ra các lớp, và cho ra hộp giới hạn. Song song với đó, dữ liệu cũng đi qua lớp một-đối-một, và cho ra hộp giới hạn. Điều này nhằm mục đích sử dụng kết quả của nhánh một-đối-nhiều cải thiện cho nhánh một-đối-một, mà không cần sử dụng NMS. Nhánh một-đối-nhiều cung cấp nhiều tín hiệu giám sát hơn so với nhánh một-đối-một. Trực giác cho thấy nếu có thể hài hòa giám sát của nhánh một-đối-một với nhánh một-đối-nhiều, thì có thể tối ưu hóa nhánh một-đối-một theo hướng của nhánh một-đối-nhiều. Kết quả là, nhánh một-đối-một có thể cung cấp chất lượng mẫu cải thiện trong quá trình suy luận, dẫn đến hiệu suất tốt hơn. Để đạt được điều này, các tác giả trước tiên phân tích khoảng cách giám sát giữa hai nhánh. Do tính ngẫu nhiên trong quá trình huấn luyện, các tác giả bắt đầu bằng cách khởi tạo hai nhánh với các giá trị giống nhau và tạo ra các dự đoán giống nhau. Các tác giả nhận ra rằng các mục tiêu hồi quy của hai nhánh không xung đột, vì những dự đoán được so khớp chia sẻ các mục tiêu giống nhau và những dự đoán không được so khớp bị bỏ qua. Điều này giúp cả hai nhánh có thể được tối ưu hóa một cách nhất quán và hài hòa.

Để kiểm chứng sự cải thiện trong việc căn chỉnh bằng cách đếm số lượng cặp ghép một-đối-một trong top-1/5/10 của kết quả một-đối-nhiều sau khi huấn luyện, cho thấy căn chỉnh được cải thiện.

1.3.2.2. So sánh với các phiên bản trước

Như trình bày trong Bảng 1, YOLOv10 đạt được hiệu suất hàng đầu và độ trễ từ đầu đến cuối trên nhiều quy mô mô hình khác nhau.

Bảng 1. So sánh hiệu suất YOLOv10 so với các phiên bản trước

Model	#Param.(M)	FLOPs(G)	AP ^{val} (%)	Latency(ms)	Latency ^f (ms)
YOLOv6-3.0-N	4.7	11.4	37.0	2.69	1.76
Gold-YOLO-N	5.6	12.1	39.6	2.92	1.82
YOLOv8-N	3.2	8.7	37.3	6.16	1.77
YOLOv10-N	2.3	6.7	38.5 / 39.5[†]	1.84	1.79
YOLOv6-3.0-S	18.5	45.3	44.3	3.42	2.35
Gold-YOLO-S	21.5	46.0	45.4	3.82	2.73
YOLO-MS-XS	4.5	17.4	43.4	8.23	2.80
YOLO-MS-S	8.1	31.2	46.2	10.12	4.83
YOLOv8-S	11.2	28.6	44.9	7.07	2.33
YOLOv9-S	7.1	26.4	46.7	-	-
RT-DETR-R18	20.0	60.0	46.5	4.58	4.49
YOLOv10-S	7.2	21.6	46.3 / 46.8[†]	2.49	2.39
YOLOv6-3.0-M	34.9	85.8	49.1	5.63	4.56
Gold-YOLO-M	41.3	87.5	49.8	6.38	5.45
YOLO-MS	22.2	80.2	51.0	12.41	7.30
YOLOv8-M	25.9	78.9	50.6	9.50	5.09
YOLOv9-M	20.0	76.3	51.1	-	-
RT-DETR-R34	31.0	92.0	48.9	6.32	6.21
RT-DETR-R50m	36.0	100.0	51.3	6.90	6.84
YOLOv10-M	15.4	59.1	51.1 / 51.3[†]	4.74	4.63
YOLOv6-3.0-L	59.6	150.7	51.8	9.02	7.90
Gold-YOLO-L	75.1	151.7	51.8	10.65	9.78
YOLOv9-C	25.3	102.1	52.5	10.57	6.13
YOLOv10-B	19.1	92.0	52.5 / 52.7[†]	5.74	5.67
YOLOv8-L	43.7	165.2	52.9	12.39	8.06
RT-DETR-R50	42.0	136.0	53.1	9.20	9.07
YOLOv10-L	24.4	120.3	53.2 / 53.4[†]	7.28	7.21
YOLOv8-X	68.2	257.8	53.9	16.86	12.83
RT-DETR-R101	76.0	259.0	54.3	13.71	13.58
YOLOv10-X	29.5	160.4	54.4 / 54.4[†]	10.70	10.60

Trước tiên, so sánh YOLOv10 với các mô hình cơ bản YOLO khác. Trên năm biến thể N/S/M/L/X, YOLOv10 đạt được cải thiện 1.2%/1.4%/0.5%/0.3%/0.5% AP, với số lượng tham số giảm 28%/36%/41%/44%/57%, số lượng tính toán giảm 23%/24%/25%/27%/38%, và độ trễ giảm 70%/65%/50%/41%/37%.

So với các phiên bản YOLO khác, YOLOv10 cũng cho thấy sự cân bằng vượt trội giữa độ chính xác và chi phí tính toán. Cụ thể, đối với các mô hình nhẹ và nhỏ, YOLOv10-N/S vượt trội hơn YOLOv6-3.0-N/S lần lượt 1.5% AP với số lượng tham số ít hơn 51%/61% và khối lượng tính toán ít hơn 41%/52%. Đối với các mô hình trung bình, so với YOLOv9-C/YOLO-MS, YOLOv10-B/M giảm độ trễ 46%/62% với cùng hoặc tốt hơn hiệu suất. Đối với các mô hình lớn, so với Gold-YOLO-L, YOLOv10-L có số lượng tham số ít hơn 68% và độ trễ thấp hơn 32%, cùng với cải thiện đáng kể 1.4% AP.

Hơn nữa, so với RT-DETR, YOLOv10 đạt được cải thiện đáng kể về hiệu suất và độ trễ. Đặc biệt, YOLOv10-S/X cho tốc độ suy luận nhanh hơn $1.8\times$ và $1.3\times$ so với RT-DETR-R18/R101 với hiệu suất tương đương. Những kết quả này chứng minh sự vượt trội của YOLOv10 như một bộ phát hiện từ đầu đến cuối thời gian thực.

So sánh YOLOv10 với các phiên bản YOLO khác sử dụng phương pháp huấn luyện gán nhãn một-đối-nhiều ban đầu. Các tác giả xem xét hiệu suất và độ trễ trong quá trình tiến tới của mô hình (Latency) trong tình huống này, theo 56, 20, 54. Như trình bày trong Bảng 1, YOLOv10 cũng thể hiện hiệu suất và hiệu quả hàng đầu trên các quy mô mô hình khác nhau, chỉ ra tính hiệu quả của các thiết kế kiến trúc khác.

Từ những ưu điểm trên, trong nghiên cứu này sẽ lựa chọn của YOLOv10 để ứng dụng vào trong việc phát hiện tấn công thay đổi giao diện web dựa trên học sâu.

1.4. Kết luận chương 1

Chương này cơ bản đã tìm hiểu tổng quan về tấn công thay đổi giao diện web, nguyên nhân và các phương pháp phát hiện tấn công đã được giới thiệu, ưu nhược điểm của từng phương pháp. Đồng thời tìm hiểu về học sâu, các mạng học sâu nổi tiếng và mô hình YOLO cho chương tiếp theo.

CHƯƠNG 2: NGHIÊN CỨU PHƯƠNG PHÁP PHÁT HIỆN TẤN CÔNG THAY ĐỔI GIAO DIỆN WEB

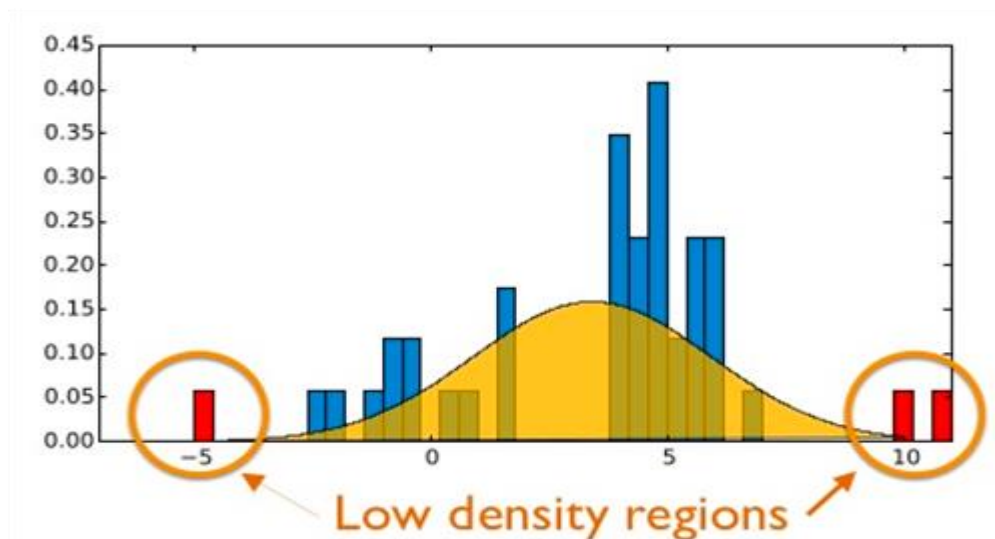
2.1. Phương pháp tiếp cận

Đối với phương pháp sử dụng công nghệ học sâu, nghiên cứu sẽ tiếp cận bằng cách chụp lại các bức ảnh của trang web được theo dõi theo định kì. Từ các bức ảnh chụp về, nghiên cứu sẽ nhờ máy tính tự động phân tích xem nó đã bị tấn công thay đổi giao diện hay chưa. Để làm được công việc này, hiện nay có thể ứng dụng một số kĩ thuật trong học sâu trên ảnh số như phát hiện bất thường/ngoại lệ và kĩ thuật phân loại ảnh. Nghiên cứu sẽ phân tích các điểm mạnh và bất cập của các thuật toán và chọn ra một thuật toán phù hợp nhất để áp dụng cho nghiên cứu này.

a) Phát hiện ngoại lệ

Một ngoại lệ là một quan sát trong tập dữ liệu dường như không phù hợp với phần còn lại của tập dữ liệu đó. Một ngoại lệ là một quan sát sai lệch quá nhiều so với các quan sát khác để làm dấy lên nghi ngờ rằng nó được tạo ra bởi một cơ chế khác. Sự bất thường được định nghĩa là những sự kiện đi chệch khỏi tiêu chuẩn, hiếm khi xảy ra và không tuân theo phần còn lại của “khuôn mẫu”. Các thuật toán phát hiện bất thường có thể được chia thành hai lớp con:

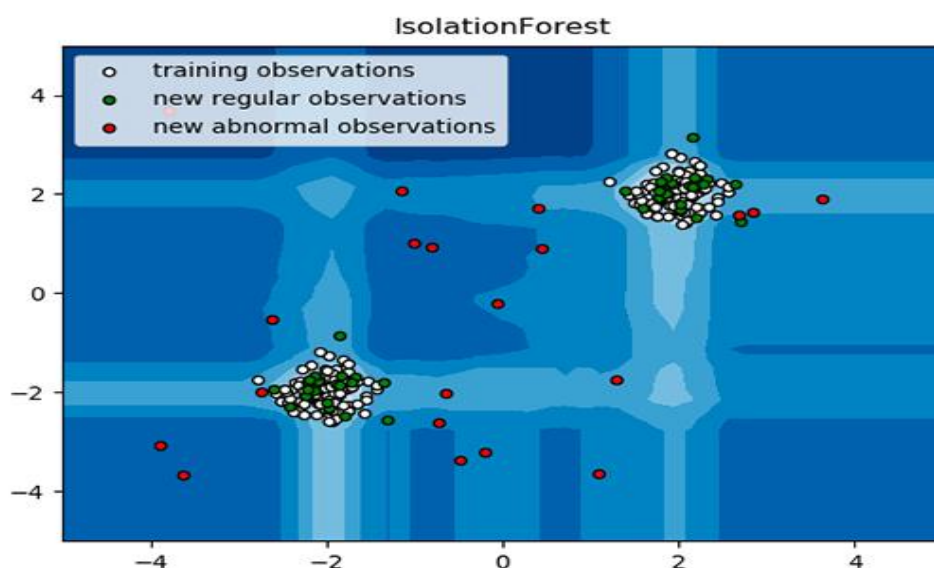
- Phát hiện ngoại lệ: Tập dữ liệu đầu vào sẽ chứa các ví dụ về cả sự kiện tiêu chuẩn và sự kiện bất thường. Các thuật toán này tìm cách khớp các vùng của dữ liệu huấn luyện, nơi mà các sự kiện tiêu chuẩn tập chung nhiều nhất và bỏ qua, do đó cô lập được các sự kiện bất thường. Các thuật toán như vậy thường được huấn luyện theo kiểu không giám sát (tức là không có nhãn).
- Phát hiện mới lạ: Không giống như phát hiện ngoại lệ, bao gồm các ví dụ về cả sự kiện tiêu chuẩn và bất thường, thuật toán phát hiện tính mới chỉ có các điểm dữ liệu sự kiện tiêu chuẩn (tức không có sự kiện bất thường) trong quá trình huấn luyện. Trong quá trình huấn luyện, sẽ cung cấp các thuật toán này với các mẫu được gắn nhãn về các sự kiện tiêu chuẩn (học có giám sát).



Hình 2.1. Dị thường tồn tại ở hai bên của đường cong hình chuông

Hiện nay đã có một số thuật toán học sâu trong việc phát hiện những sự thay đổi, điểm bất thường trong ảnh nổi tiếng như Isolation Forest, Autoencoder hay một số giải thuật trong OpenCV hoạt động rất hiệu quả.

Isolation Forest là loại thuật toán tổng hợp và bao gồm nhiều cây quyết định được sử dụng để phân vùng tập dữ liệu đầu vào thành các nhóm nội số riêng biệt. Như hình 2.2 ở dưới cho thấy, Isolation Forest nhận các điểm đầu vào (các điểm trắng) và sau đó xây dựng một lớp cách ly bao quanh chúng. Tại thời điểm thử nghiệm, Isolation Forest sau đó có thể xác định xem các điểm đầu vào nằm bên trong lớp cách ly (sự kiện tiêu chuẩn – điểm màu xanh lá cây) hay bên ngoài vùng mật độ cao (sự kiện bất thường – điểm màu đỏ).



Hình 2.2. Các điểm dị thường nằm ngoài lớp cách ly được mô hình xây dựng trong Isolation Forest

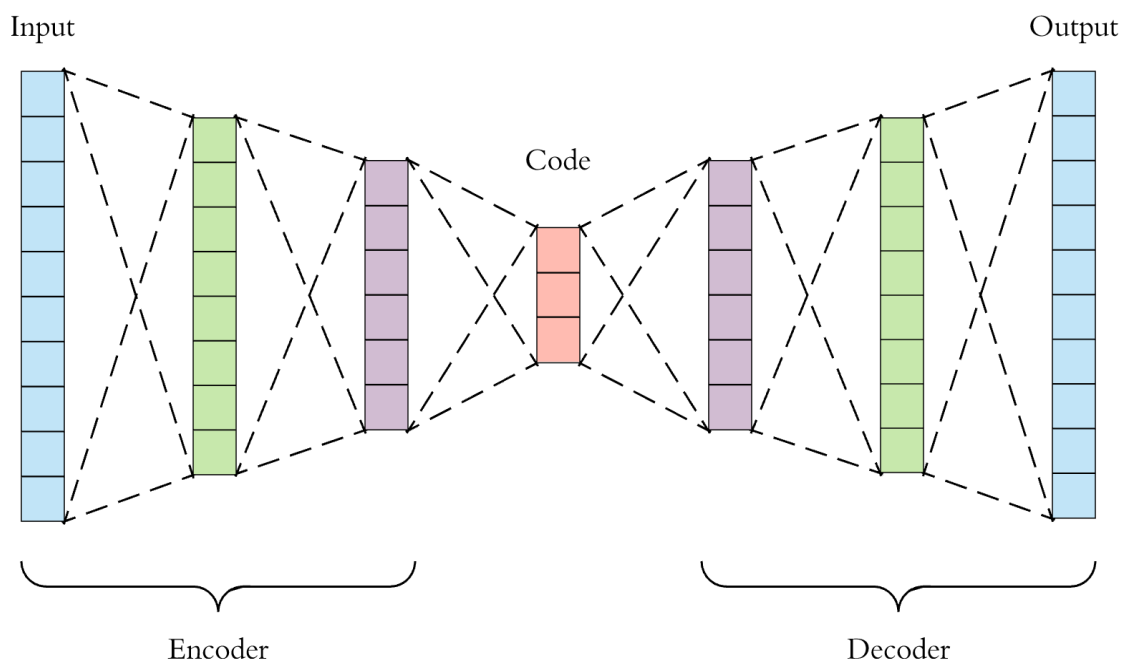
Tuy nhiên, để áp dụng được mô hình này vào nghiên cứu này, trước tiên cần phải thu thập được một tập dữ liệu đủ lớn về trang web cần theo dõi, không thể là các trang web khác. Vì cần phải mô hình lại các nét đặc trưng và xây dựng được các một lớp các ly – các điểm đầu vào màu trắng sát nhau tức gần giống nhau (như hình 2.2). Dữ liệu về một trang web khá nghèo nàn khó có thể thu thập được số lượng đủ nhiều để phục vụ cho huấn luyện học sâu, các chức năng khác nhau của một trang web có thể trả lại các hình ảnh hoàn toàn khác nhau, có thể áp dụng một số phương pháp tăng cường dữ liệu để làm đa dạng đầu vào nhưng vẫn chưa đủ. Ngoài ra mỗi khi cấu hình theo dõi một trang web mới, hệ thống sẽ phải thực hiện lại khâu thu thập dữ liệu và huấn luyện lại mô hình chưa kể khi trang web có cập nhật lớn về giao diện cũng sẽ phải thực hiện lại quá trình này.

b) Autoencoder

Autoencoder là một loại mạng nơ-ron được sử dụng để học các đặc trưng của dữ liệu nhằm mục đích mã hóa và nén dữ liệu một cách tự động. Mô hình này được sử dụng để xác định các đặc trưng nén hiệu quả nhất của bộ dữ liệu bằng phương pháp học không giám sát. Autoencoder thường bao gồm ba thành phần chính: khối mã hóa, mã và khối giải mã (như minh họa trong hình 2.3).

Mục đích của khối mã hóa là chuyển đổi dữ liệu đầu vào thành một dạng mã hóa. Đồng thời, khối giải mã cố gắng khôi phục dữ liệu ban đầu từ mã đã được mã hóa, sao cho dữ liệu được giải mã khớp với dữ liệu đầu vào nhiều nhất có thể. Với việc Autoencoder có đầu vào và đầu ra giống nhau, nó không cần dữ liệu được gắn nhãn như các mô hình khác, giúp tiết kiệm thời gian và công sức. Do đó, mô hình này có thể được huấn luyện bằng phương pháp học không giám sát.

Ý tưởng của thuật toán này là đưa dữ liệu đầu vào (Encoder) đi qua một khối nén mã (Code) sẽ khử nhiễu và chỉ giữ lại các đặc điểm quan trọng của bức ảnh, đầu ra thuật toán sẽ cố gắng tái tạo lại bức ảnh dựa trên những đặc trưng sau khi nén.



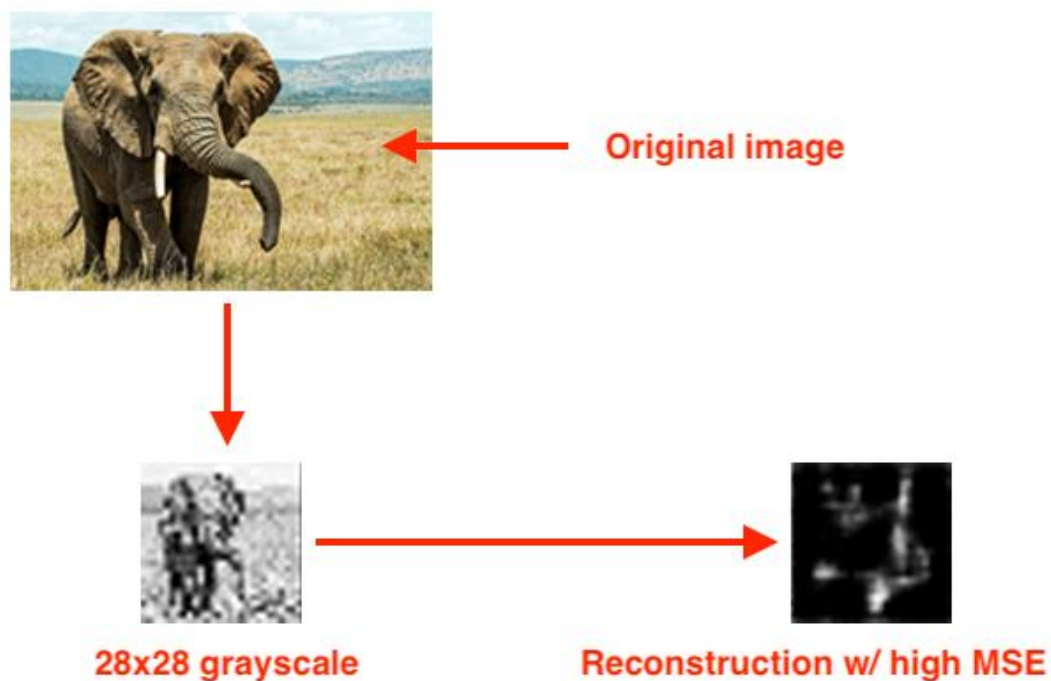
Hình 2.3. Mô hình nén và tái cấu trúc dữ liệu trong Autoencoder

Từ góc độ ứng dụng vào việc phát hiện bất thường, đầu tiên nghiên cứu sẽ phải huấn luyện được một mô hình autoencoder chứa các thông số nén, xử lý bức ảnh mẫu sao cho ảnh được tái tạo là tốt nhất. Trong quá trình kiểm tra bất thường, nghiên cứu sẽ sử dụng mô hình autoencoder được huấn luyện trước đó để tái cấu trúc lại bức ảnh kiểm tra, sau đó đo sai số toàn phương trung bình (Mean squared error - MSE) giữa hình ảnh đầu vào và bản tái dựng lại, nếu MSE thấp thì đây là bức ảnh bình thường và ngược lại.



Hình 2.4. Tái cấu trúc chữ số 4 dựa trên mô hình autoencoder đã huấn luyện

Như hình 2.4, mô hình autoencoder đã làm việc rất tốt khi tái tạo lại chữ số 4 với MSE rất là thấp. Bây giờ, giả sử nghiên cứu đưa mô hình autoencoder trên với một bức ảnh con voi mà nó chưa bao giờ nhìn thấy trước đó và yêu cầu nó tái tạo lại nó.



Hình 2.5. Tái cấu trúc ảnh con voi dựa trên mô hình autoencoder đã huấn luyện

Vì mô hình autoencoder chưa bao giờ nhìn thấy một con voi trước đây, và nó chưa bao giờ được huấn luyện lại để tái cấu trúc lại một con voi cho nên MSE giữa bức ảnh tái tạo và ảnh gốc là rất lớn. Nếu MSE của quá trình tái cấu trúc cao thì nó sẽ là một ngoại lệ.

Ưu điểm của phương pháp này đó là không cần một tập dữ liệu huấn luyện lớn, thuật toán có thể phát hiện được những sự thay đổi rất nhỏ như thay đổi trái phép nội dung. Trong nghiên cứu này, khi áp dụng thuật toán Autoencoder nghiên cứu gặp một số khó khăn như sau:

- Một bức ảnh về giao diện của một trang web có rất nhiều nhiễu gây khó khăn trong việc xây dựng mô hình autoencoder (ví dụ: các chi tiết trang trí, các kí tự văn bản nhỏ, màu sắc của giao diện, ...) để lấy đặc trưng của giao diện trang web.
- Mô hình áp dụng cho mỗi trang web là độc lập và phải tối ưu hóa thủ công, phải huấn luyện lại mô hình khi theo dõi một trang web mới, hoặc khi trang web có sự cập nhật giao diện lớn.
- Các trang web động có thể gây nhiều cảnh báo giả.

Do đó nghiên cứu quyết định không sử dụng thuật toán này áp dụng vào nghiên cứu.

Ngoài mục tiêu phát hiện được trang web bị tấn công một cách tự động, nghiên cứu này cũng phải thể hiện được tính linh hoạt của nó, tức là khi theo dõi một trang web mới hay khi có sự cập nhật giao diện đáng kể cũng không thể gây ảnh hưởng nhiều tới độ chính xác của hệ thống.

c) YOLO

Một thuật toán học máy nữa cũng làm việc hiệu quả trong vấn đề xử lý ảnh đó là mạng nơ-ron tích chập (Convolutional Neural Network - CNN), đây là thuật toán hiệu quả học có giám sát trong việc phân loại ảnh. Trong phần 1.3 đã giới thiệu mô hình YOLO có kiến trúc rất phức tạp, ứng dụng hiệu quả mô hình CNN để trích xuất các đặc trưng của bức ảnh, và các lớp khác để phân tích các kết quả đầu ra, cuối cùng đưa ra dự đoán. Xét về bài toán của nghiên cứu này, nghiên cứu đưa một trang web sẽ đánh giá về có hai trạng thái cơ bản đó là “bị tấn công thay đổi giao diện” và trạng thái “bình thường”. Mặc dù giao diện của các trang web trên thế giới là rất đa dạng, nhưng giữa trang web bị tấn công thay đổi giao diện và trang web bình thường luôn có sự khác biệt rất lớn, mô hình phân loại có hai lớp đầu vào tức là có hai nhãn dữ liệu đó là “bị thay đổi giao diện” và “bình thường”. Sử dụng tập dữ liệu rất lớn để huấn luyện, nghiên cứu cho mô hình học về thể nào là ảnh một trang web bình thường, thể nào là ảnh một trang web bị thay đổi giao diện, không phải so sánh với ảnh gốc hay huấn luyện dựa trên ảnh gốc - ảnh trang web được theo dõi ở trạng thái bình thường. Do đó, mô hình của thuật toán thể hiện sự linh hoạt rất tốt, không cần phải huấn luyện lại khi thực hiện theo dõi một trang web mới, khi trang web là web động hay có sự cập nhật lớn về giao diện. Do vậy nghiên cứu quyết định lựa chọn mô hình YOLOv10 để xây dựng hệ thống.



Hình 2.6. Quy trình phát triển mô hình học sâu

Nghiên cứu sẽ tiếp cận và giải quyết bài toán theo mô hình 2.6. Bước đầu tiên, nghiên cứu sẽ trình bày các cách thức thu thập tập dữ liệu cho mô hình học

sâu của mình. Bước 2, sau khi có tập dữ liệu huấn luyện đủ lớn, nghiên cứu sẽ trình bày cách để phân chia gán nhãn, lưu trữ tập dữ liệu, xử lý kích thước trước khi đưa vào mô hình huấn luyện. Bước 3 và bước 4 là giai đoạn huấn luyện mô hình, nghiên cứu sẽ trình bày về quá trình xây dựng và chọn ra một hình học sâu cho bài toán, nghiên cứu thực hiện thử nghiệm huấn luyện trên nhiều mô hình khác nhau như VGG16, ResNet50, InceptionV3, MobileNetV2; đồng thời áp dụng các cách để tối ưu hóa giúp cho quá trình huấn luyện đạt hiệu quả tốt nhất. Bước cuối cùng, nghiên cứu sẽ đánh giá hiệu quả của mô hình bằng cách phân loại trực tiếp trên tập dữ liệu thực tế. Các bước 3,4 và 5 sẽ được thực hiện và lặp lại liên tục cho đến khi tìm được mô hình đạt hiệu quả tốt nhất, nghiên cứu này sẽ chỉ trình bày chi tiết về mô hình YOLOv10 – đây là mô hình đạt hiệu quả tốt nhất trong quá trình đánh giá.

2.2. Thu thập dữ liệu đầu vào

Việc thu thập được một số lượng lớn về hình ảnh các trang web bình thường và bị tấn công là hết sức khó khăn, vì hiện chưa có kho lưu trữ dataset mẫu nào trên thế giới có sẵn về dữ liệu này.

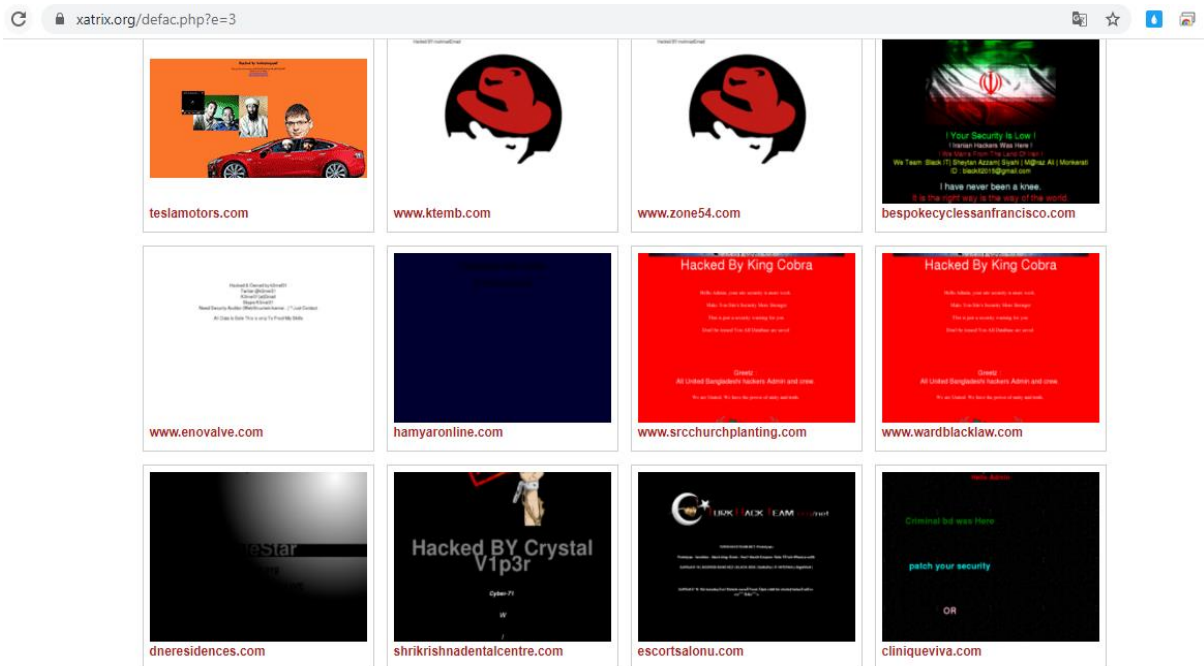
2.2.1. Dữ liệu về các trang web bị tấn công thay đổi giao diện

Các cuộc tấn công thay đổi giao diện web đã được báo cáo thường xuyên, và đã có một số trang web lưu trữ công khai và phản ánh các cuộc tấn công thay đổi giao diện trong thời gian thực. Một trong những trang web nổi tiếng nhất thuộc loại này là Zone-H, Mirror-H, Zone.KURD-H và Xatrix.org, nó đã được sử dụng rộng rãi trong các nghiên cứu về các cuộc tấn công. Các trang web phản ánh và lưu trữ hình ảnh bị tấn công thay đổi giao diện như vậy chủ yếu phụ thuộc vào việc tự báo cáo của chính kẻ tấn công.

Attacker	Country	Web URL	IP's	Date	Preview
/MaXimous48	(US)	https://bostonadagency.com/index.html	198.71.233.197	16/05/2021	
Revonzy	(US)	https://snald.net/	172.67.206.223	16/05/2021	
Revonzy	(US)	https://best-juicers.com/	172.67.173.74	16/05/2021	
Revonzy	(CA)	http://gardenruler.com/	104.152.168.7	16/05/2021	
Revonzy	(CA)	http://gardenintro.com/	104.152.168.7	16/05/2021	

Hình 2.7. Thông tin cập nhật các trang web đã bị tấn công thay đổi giao diện

Khi đã có nguồn dữ liệu lớn nghiên cứu sẽ tiến hành viết ra các công cụ giúp thu thập nhanh chóng. Ý tưởng ban đầu là nghiên cứu sẽ viết một chương trình để lấy các url trong các trang web này, lọc ra thu được các url của các trang web bị tấn công mà các trang web này tham chiếu đến. Tuy nhiên, các trang này đã thiết lập nhiều cơ chế chặn bot như sử dụng mã captcha mỗi lần duyệt, cloudflare. Cho nên các đoạn script của nghiên cứu không thể hoạt động được như mong muốn, do đó phải tiến hành thu thập một cách thủ công. Riêng trang web Xatrix.org lưu trữ dữ liệu về các trang bị tấn công dưới dạng hình ảnh thông thường, mặc dù dữ liệu của trang web này đã dừng cập nhật từ năm 2016, tuy nhiên có vẫn hữu ích cho việc huấn luyện mô hình của em, nghiên cứu sẽ sử dụng trang extract.pics để trích xuất các hình ảnh trong một trang web ra rồi tải nó về.



Hình 2.8. Dữ liệu từ trang Xatrix.org

Sau khi đã thu thập được các url bị tấn công từ các trang là Zone-H, Mirror-H, Zone.KURD-H xong, nghiên cứu sẽ viết một chương trình để tự động chụp lại ảnh các trang web từ url đó. Nghiên cứu sử dụng thư viện Selenium trong python để thực hiện công việc này.

```

12 def screenshot(url):
13     width = 1366 # Set your desired width
14     height = 768 # Set your desired height
15     options = webdriver.ChromeOptions()
16     options.headless = True
17     options.add_argument(f"--window-size={width},{height}")
18     options.add_argument("--start-maximized")
19     options.add_argument('--disable-dev-shm-usage')
20     options.add_argument('--ignore-certificate-errors')
21     options.add_argument('--disable-setuid-sandbox')
22     options.add_argument('--no-sandbox')
23     options.add_argument('--headless')
24     options.add_experimental_option("excludeSwitches", ["enable-automation"])
25     options.add_experimental_option('useAutomationExtension', False)
26     options.add_argument("--disable-blink-features=AutomationControlled")
27     driver = webdriver.Chrome(options=options)
28     try:
29         if not url.startswith("http://") and not url.startswith("https://"):
30             url = "https://" + url
31         driver.get(url)
32         driver.implicitly_wait(2)
33         print("Screenshotsing..." + url)
34         driver.get_screenshot_as_file("gov_images/web_screenshot.png")
35         driver.quit()
36     except:
37         print(url + " was died")
38         pass
39     #resize gov_images
40     print("resizing...")
41     image = Image.open('gov_images/web_screenshot.png')
42     name = hashlib.md5(url.encode())
43     image.save('gov_images/' + name.hexdigest() + '.png')

```

Trong đoạn mã trên, Python Selenium sẽ sử dụng Chromium để duyệt url, các dòng từ 17 đến 25 sẽ giúp chặn đầy các cảnh báo lên và cho phép Chromium thực hiện duyệt web ngầm dưới hệ thống, không hiển thị cửa sổ trình duyệt, dòng 17 sẽ cấu hình cho selenium chụp ảnh màn hình dưới dạng cửa sổ máy tính với window-size bằng 1366x768. Sau khi duyệt web, nghiên cứu cho Chromium chờ 5 giây để tải được đầy đủ hình ảnh rồi gọi hàm `get_screenshot_as_file()` để chụp ảnh và lưu lại trên ổ đĩa.

2.2.2. Dữ liệu về các trang web bình thường

Mặc dù các số lượng trang web hiện nay đang hoạt động là rất lớn, nhưng rất ít các trang web thông kê về nó.

Theo trang Domain Authority, Domain Authority (DA) là điểm số xếp hạng của công cụ tìm kiếm do Moz phát triển nhằm dự đoán khả năng một trang web xếp hạng trên các trang kết quả của công cụ tìm kiếm (search engine result pages - SERPs). Trang này cũng cấp một danh sách thống kê top 500 tên miền phổ biến nhất trên thế giới, dữ liệu này cũng cho phép xuất dưới dạng CSV cho cộng đồng nghiên cứu, đây là một dữ liệu tốt cho mô hình học máy này.

Theo trang web của Cơ quan quản lý dịch vụ tổng hợp Hoa Kỳ (General Services Administration - GSA), cũng cung cấp một danh sách hơn 8000 tên miền của chính phủ Hoa Kỳ, dữ liệu được lưu trữ mã nguồn mở trên Github dưới dạng CSV. Đây là một nguồn dữ liệu lớn có thể tận dụng.

Đối với các tên miền .vn, trang <https://nukeviet.vn/webcatalog/domain/> cung cấp một lượng lớn tên miền, nhưng các tên miền này lại không có chọn lọc, rất nhiều liên kết đã chết và dữ liệu không được liệt kê sẵn trong một tệp tin, nên cần phải viết một chương trình riêng để tự động tách các tên miền ra từ trang thống kê.

Khi đã có các tệp tin dữ liệu dưới dạng CSV, nghiên cứu sẽ tiến hành bóc tách các thông tin ra, chỉ lấy lại tên url.


```

1 import csv
2
3 f = open("top500Domains.txt", "w")
4 with open('top500Domains.csv') as csv_file:
5     csv_reader = csv.DictReader(csv_file, delimiter=',')
6     line_count = 0
7     for row in csv_reader:
8         f.write(row["Root Domain"]+"\n")
9         line_count += 1
10 f.close()
11

```

Kết quả:

```

1 http://patchoguevillage.org/
2 http://pateros.com/
3 http://patientsafetyauthority.org/Pages/Default.aspx
4 http://patriotspoint.org/
5 http://pattersonny.org/
6 http://www.pattersontwp.com/
7 http://pattonboro.com/
8 http://www.paturnpike.com/
9 http://web.paulbunyan.net/tenlake/Ten_Lake_Township.html
10 http://paulsboronj.org/
11 http://pawlingny.virtualtownhall.net/Pages/index
12 http://pawneecountykansas.com/
13 http://pawtucketri.com/
14 http://payettecounty.org/
15 http://www.paynecounty.org/
16 http://www.paynesvillemn.com/
17 http://www.pbcgov.com/
18 http://pbgfl.com/
19 http://pcaobus.org/Pages/default.aspx

```

Từ danh sách này, nghiên cứu sẽ đưa vào đoạn chương trình như đã trình bày ở phần trước để thực hiện lưu lại ảnh màn hình của chúng.

Tổng kết, từ các nguồn thu thập được, có tổng cộng 7,270 bức ảnh về các trang web bình thường và 4,850 bức ảnh về trang web đã bị tấn công thay đổi giao diện. Nghiên cứu sẽ tải nó lên Google Driver và chia nó vào các thư mục có cấu trúc như sau:

```

Data/
├-- train/
│   ├── clean/
│   └-- defaced/
└-- val/
    ├── clean/
    └-- defaced/

```

Thư mục `Training/` chứa một tập dữ liệu hình ảnh cho mục đích huấn luyện mô hình, thư mục `val/` chứa một tập dữ liệu bằng khoảng 20% thư mục `Training/` để đánh giá mô hình sau khi huấn luyện xong. Mỗi thư mục có hai thư mục con, thư mục `clean/` chứa các ảnh về các trang web bình thường, thư mục `defaced/` chứa các ảnh về các trang web đã bị tấn công thay đổi giao diện.

2.3. Huấn luyện mô hình

2.3.1. Chuẩn bị tập dữ liệu

Trước khi đào tạo, điều cần thiết là phải xử lý trước tập dữ liệu. Quá trình này bao gồm việc thay đổi kích thước hình ảnh, chuẩn hóa các giá trị pixel và chuyển đổi chú thích sang định dạng theo yêu cầu của YOLOv10.

Chuyển đổi các chú thích sang định dạng YOLO là bước quan trọng trước khi huấn luyện tập dữ liệu tùy chỉnh với YOLOv10. Định dạng này yêu cầu mỗi đối tượng trong ảnh được đại diện bởi một dòng duy nhất trong tệp văn bản tương ứng với từng ảnh. Cụ thể, mỗi dòng bao gồm:

- **Chỉ số lớp (class index):** Số đại diện cho nhãn của đối tượng.
- **Tọa độ trung tâm của đối tượng (x_center, y_center):** Được chuẩn hóa trong khoảng từ 0 đến 1, với (0,0) là góc trên bên trái và (1,1) là góc dưới bên phải.
- **Chiều rộng và chiều cao (width, height):** Cũng được chuẩn hóa từ 0 đến 1, đại diện cho kích thước của đối tượng theo tỷ lệ ảnh.

Mỗi tệp tin `.txt` tương ứng với một ảnh và chứa các dòng như sau:

<code>class_index x_center y_center width height</code>

Nghiên cứu này mong muốn mô hình có thể học được toàn bộ bức ảnh, không chỉ riêng một đối tượng cụ thể nào, do các bức ảnh của các trang web là không giống nhau và không có đặc trưng nào cố định. Do vậy nghiên cứu đã để giá trị mặc định cho 'x_center y_center width height' là 0.5 0.5 1.0 1.0. Để làm được điều này, cần viết một chương trình tự động tạo tệp tin chú thích như sau:

```
[ ] import os

def create_yolo_labels(base_directory):
    label_map = {
        'clean': 0,
        'defaced': 1
    }

    for dataset in ['train', 'val']:
        for label in ['clean', 'defaced']:
            directory = os.path.join(base_directory, dataset, label)
            for filename in os.listdir(directory):
                if filename.endswith((''.jpg', '.jpeg', '.png')):
                    base_name = os.path.splitext(filename)[0]
                    label_file_path = os.path.join(directory, base_name + '.txt')

                    # Nội dung file nhãn chứa class_id của label và tọa độ bounding box (ở đây là toàn bộ ảnh)
                    content = f"{label_map[label]} 0.5 0.5 1.0 1.0\n"

                    # Ghi file nhãn
                    with open(label_file_path, 'w') as label_file:
                        label_file.write(content)

                    print(f"Created label file {label_file_path} for {filename}")

# Thực thi hàm để tạo file nhãn
base_directory = 'Data'
create_yolo_labels(base_directory)
```

Đoạn mã trên sẽ tự động tạo một tệp tin .txt có cùng tên với bức ảnh, trong đó tên nhãn của tệp chú thích sẽ tương ứng với tên thư mục dùng để phân loại đã tạo trước đó.

Ngoài ra, cần tạo một tệp tin .yaml để cấu hình tập dữ liệu với các thông tin sau:

- **Đường dẫn đến tập dữ liệu (train, val paths):** Đường dẫn đến tập dữ liệu huấn luyện và kiểm thử.
- **Số lớp (nc):** Số lượng nhãn trong tập dữ liệu.
- **Tên các lớp (names):** Danh sách tên các nhãn, mỗi nhãn tương ứng với chỉ số lớp trong các tệp tin chú thích.

Tệp tin dataset.yaml của nghiên cứu trông như sau:

```
train: /content/Data/train
val: /content/Data/val

nc: 2
names: ["clean", "defaced"]
```

2.3.2. Huấn luyện mô hình

Đầu tiên cần cài đặt thư viện ultralytics của python, nơi chứa mô hình huấn luyện cho các phiên bản của YOLO. Cài đặt:

```
pip install ultralytics
```

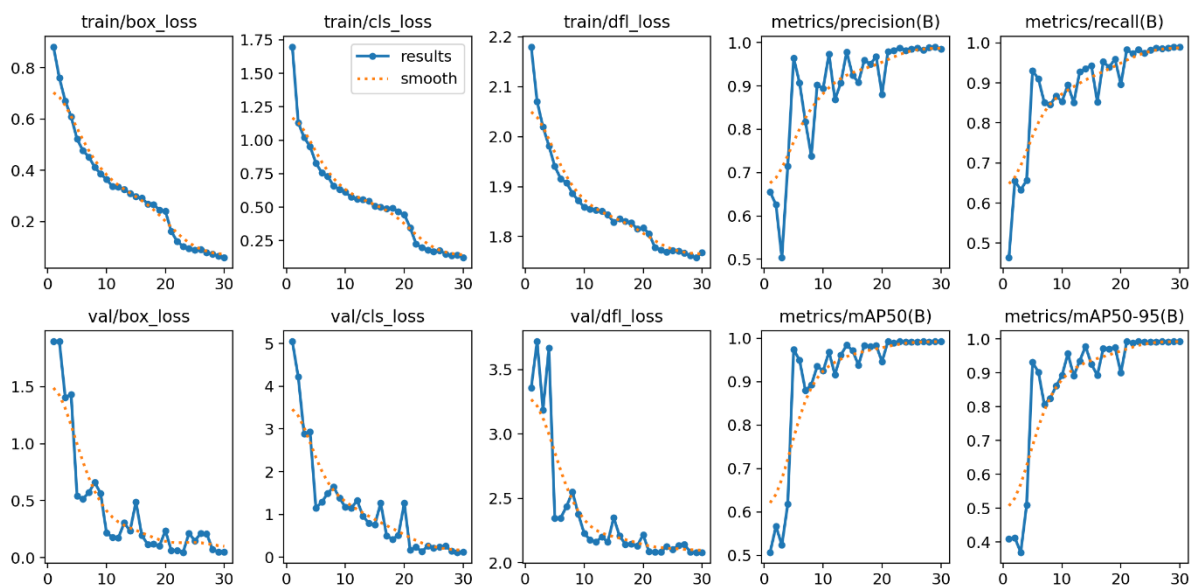
Tiếp theo, cần tải mô hình đã huấn luyện của YOLOv10 để thực hiện học chuyển giao, nghiên cứu này sử dụng phiên bản YOLOv10-X, đây là phiên bản có độ tin cậy cao nhất nhưng sẽ tốn chi phí huấn luyện và sử dụng nhất. Nhưng do yêu cầu hệ thống phát hiện phải đảm bảo có ít cảnh báo giả nhất và tránh bỏ sót, do đó sẽ ưu tiên sử dụng phiên bản này.

```
!wget -P /content  
https://github.com/jameslahm/yolov10/releases/download/v1.0/yolov10x.pt
```

Cuối cùng, thực hiện học chuyển giao trên YOLOv10-X trên mô hình vừa tải với các thông số cài đặt trong tệp tin dataset.yaml.

```
from ultralytics import YOLO  
  
model = YOLO()  
model = YOLO('/content/yolov10x.pt')  
model.train(data='/content/dataset.yaml', epochs=30, batch=32, imgsz=256)
```

Kết quả của quá trình huấn luyện sẽ được lưu trong thư mục /content/runs.



Hình 2.9. Biểu đồ kết quả quá trình huấn luyện

epoch	time	train/box_loss	train/cls_loss	train/dfll_loss	metrics/precision(B)	metrics/recall(B)	metrics/mAP50(B)	metrics/mAP50-95(B)	val/box_loss	val/cls_loss	val/dfll_loss	lr/pg0	lr/pg1	lr/pg2
1	169.942	0.88272	1.69826	2.18001	0.65549	0.46373	0.50695	0.41004	1.89823	5.04733	3.35821	0.000553821	0.000553821	0.000553821
2	327.554	0.76083	1.13161	2.07051	0.62574	0.65515	0.56685	0.41221	1.89666	4.21841	3.71746	0.00107287	0.00107287	0.00107287
3	481.403	0.67076	1.01989	2.01948	0.50369	0.63326	0.52365	0.36968	1.40574	2.883	3.18647	0.00155525	0.00155525	0.00155525
4	634.432	0.60889	0.95408	1.98178	0.71531	0.65649	0.61791	0.5094	1.43299	2.93696	3.66483	0.00150197	0.00150197	0.00150197
5	787.308	0.52259	0.83022	1.94048	0.96483	0.93045	0.97398	0.93088	0.54264	1.1541	2.34833	0.00144696	0.00144696	0.00144696
6	940.707	0.47663	0.7562	1.91605	0.90728	0.90981	0.94933	0.90203	0.51591	1.29049	2.34833	0.00139195	0.00139195	0.00139195
7	1093.33	0.45221	0.73112	1.90761	0.81736	0.85112	0.87961	0.80663	0.57145	1.49629	2.4378	0.00133693	0.00133693	0.00133693
8	1246.26	0.41244	0.65718	1.88667	0.73819	0.84598	0.89258	0.8244	0.65954	1.64542	2.55039	0.00128192	0.00128192	0.00128192
9	1398.97	0.38636	0.6334	1.8718	0.90274	0.86702	0.9356	0.86067	0.56427	1.38558	2.37693	0.00122691	0.00122691	0.00122691
10	1551.69	0.36444	0.61292	1.85865	0.89393	0.85348	0.92499	0.89198	0.21882	1.17086	2.22714	0.0011719	0.0011719	0.0011719
11	1704.2	0.33699	0.57422	1.85531	0.97417	0.89496	0.96804	0.95638	0.17805	1.155	2.1787	0.00111689	0.00111689	0.00111689
12	1857.53	0.33422	0.56264	1.85265	0.86839	0.85041	0.91604	0.89109	0.17185	1.3247	2.16259	0.00106188	0.00106188	0.00106188
13	2010.24	0.32596	0.55589	1.85107	0.90688	0.92728	0.96103	0.93492	0.30426	0.96381	2.20071	0.00100687	0.00100687	0.00100687
14	2162.83	0.30877	0.54442	1.84395	0.97875	0.93593	0.9841	0.9777	0.23571	0.79187	2.1616	0.000951857	0.000951857	0.000951857
15	2315.92	0.29833	0.50737	1.82891	0.92461	0.94289	0.9716	0.9253	0.49018	0.762	2.35019	0.000896846	0.000896846	0.000896846
16	2468.97	0.29269	0.50118	1.83598	0.90849	0.85234	0.93809	0.89236	0.1939	1.27071	2.20809	0.000841835	0.000841835	0.000841835
17	2621.71	0.26755	0.48739	1.83071	0.95971	0.95275	0.98365	0.97121	0.11645	0.50422	2.14292	0.000786824	0.000786824	0.000786824
18	2774.68	0.26634	0.49263	1.82746	0.94996	0.93917	0.98077	0.97056	0.11883	0.42386	2.14543	0.000731813	0.000731813	0.000731813
19	2927.23	0.24467	0.467	1.81519	0.96777	0.95927	0.98382	0.97511	0.1034	0.51896	2.13259	0.000676802	0.000676802	0.000676802
20	3080.14	0.24014	0.445	1.81801	0.8806	0.8963	0.94575	0.89988	0.23706	1.26531	2.21831	0.000621791	0.000621791	0.000621791
21	3233.85	0.16054	0.347	1.80562	0.97934	0.98285	0.99255	0.99255	0.06288	0.16862	2.08616	0.00056678	0.00056678	0.00056678
22	3387.01	0.12193	0.22583	1.7782	0.98184	0.97336	0.98921	0.98874	0.06294	0.23902	2.08339	0.000511769	0.000511769	0.000511769
23	3539.21	0.10298	0.19744	1.77302	0.98796	0.98299	0.99205	0.99205	0.04618	0.13236	2.08208	0.000456758	0.000456758	0.000456758
24	3691.6	0.09466	0.17994	1.76895	0.98234	0.97476	0.99144	0.99144	0.21463	0.27022	2.12831	0.000401747	0.000401747	0.000401747
25	3844.02	0.08927	0.16766	1.77176	0.98571	0.98142	0.99138	0.99138	0.14698	0.21376	2.10391	0.000346736	0.000346736	0.000346736
26	3996.25	0.08997	0.17529	1.77159	0.98814	0.98661	0.99181	0.99179	0.21347	0.24068	2.1336	0.000291725	0.000291725	0.000291725
27	4148.49	0.07962	0.15132	1.7669	0.98347	0.98611	0.99237	0.99235	0.21094	0.26197	2.14328	0.000236714	0.000236714	0.000236714
28	4300.51	0.07201	0.13825	1.76188	0.98877	0.98685	0.99137	0.99137	0.07168	0.14865	2.08371	0.000181703	0.000181703	0.000181703
29	4452.53	0.06528	0.14085	1.75778	0.99046	0.98933	0.99216	0.99216	0.04966	0.11588	2.08137	0.000126692	0.000126692	0.000126692
30	4605	0.05856	0.12258	1.76856	0.9852	0.99011	0.9922	0.9922	0.04849	0.12401	2.08144	7.17E-05	7.17E-05	7.17E-05

Hình 2.10. Chi tiết kết quả quá trình huấn luyện

1) Train/Validation Losses

- Train/Box Loss (tổn thất hộp huấn luyện): Đồ thị cho thấy giá trị loss giảm đều đặn từ khoảng 0.88 xuống gần 0.05 sau 30 epochs. Điều này chỉ ra rằng mô hình đã học cách dự đoán hộp bao quanh (bounding box) ngày càng chính xác hơn trên tập huấn luyện.
- Train/Cls Loss: Loss giảm từ khoảng 1.7 xuống dưới 0.12, cho thấy sự cải thiện lớn trong việc phân loại các đối tượng trên tập huấn luyện.
- Train/DFL Loss: Loss giảm từ khoảng 2.2 xuống 1.7, tiếp tục cho thấy sự cải thiện và mô hình học cách đánh giá độ chính xác của bounding box tốt hơn.
- Val/Box Loss: Trên tập kiểm thử, box loss giảm từ khoảng 1.8 xuống dưới 0.04, thể hiện sự cải thiện đáng kể và nhất quán của mô hình trên dữ liệu kiểm thử.
- Val/Cls Loss: Giảm từ khoảng 5.04 xuống dưới 0.12, cho thấy mô hình có khả năng phân loại tốt hơn với dữ liệu kiểm thử.
- Val/DFL Loss: Loss giảm từ khoảng 3.35 xuống khoảng 2, phản ánh khả năng học của mô hình đối với dữ liệu chưa được thấy trước đó.

2) Metrics

- Metrics/Precision(B): Precision tăng từ khoảng 0.5 lên gần 1.0, cho thấy mô hình đã đạt được độ chính xác cao trong dự đoán các đối tượng.
- Metrics/Recall(B): Recall tăng từ khoảng 0.6 lên gần 1.0, điều này chỉ ra rằng mô hình đã phát hiện được hầu hết các đối tượng cần dò tìm.
- Metrics/mAP50(B): Mean Average Precision (mAP) tại IOU 0.5 tăng từ 0.6 lên gần 1.0, phản ánh rằng mô hình có hiệu năng tốt trong việc phát hiện các đối tượng với mức độ chính xác cao.

- Metrics/mAP50-95(B): mAP tại các ngưỡng IOU từ 0.5 đến 0.99 tăng từ 0.4 lên gần 0.9, chỉ ra rằng mô hình có hiệu năng cao trên các mức độ chính xác khác nhau.

Kết quả huấn luyện và kiểm thử chỉ ra rằng mô hình đã học rất tốt và có khả năng tổng quát hóa cao. Các giá trị loss giảm đều và các chỉ số precision, recall, mAP đều tăng lên đáng kể, mô hình đạt được hiệu năng cao cả trên tập dữ liệu huấn luyện và kiểm thử.

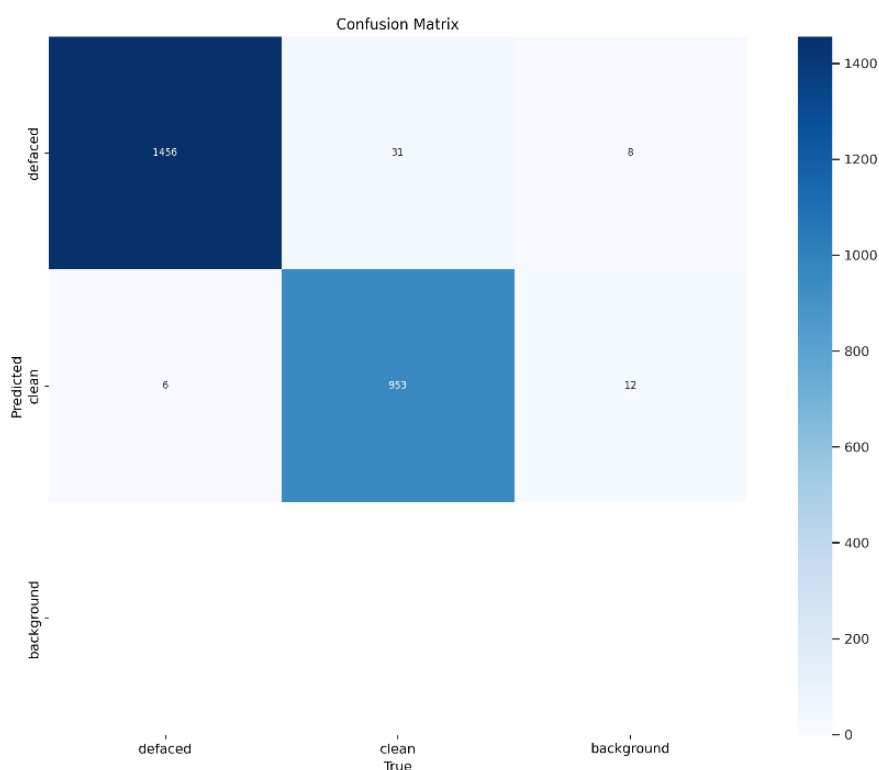
Những kết quả này có thể được xem là minh chứng rõ ràng cho một mô hình thành công và phù hợp với nhiệm vụ phát hiện đối tượng.

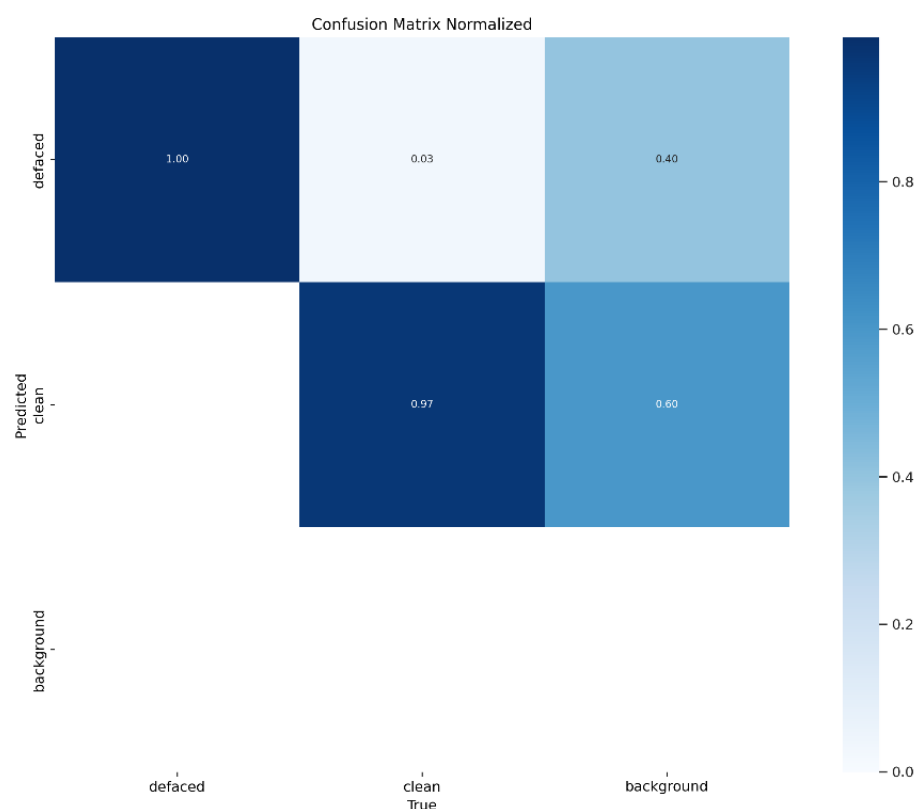
2.4. Đánh giá mô hình

2.4.1. Ma trận nhầm lẫn

Độ chính xác của lớp "clean", tỉ lệ dương tính thật đạt 1.00. Đây là tỷ lệ các mẫu "clean" thực sự được dự đoán chính xác là "clean". Mô hình đạt độ chính xác rất cao cho lớp này, thể hiện mô hình có tỉ lệ cảnh báo giả cực kì thấp.

Độ chính xác của lớp "defaced", tỉ lệ dương tính thật đạt 0.97, tỉ lệ âm tính giả chỉ 0.03. Tỷ lệ các mẫu "defaced" được mô hình dự đoán đúng rất cao, cho thấy mô hình đạt được khả năng xác định tốt các trường hợp tấn công.



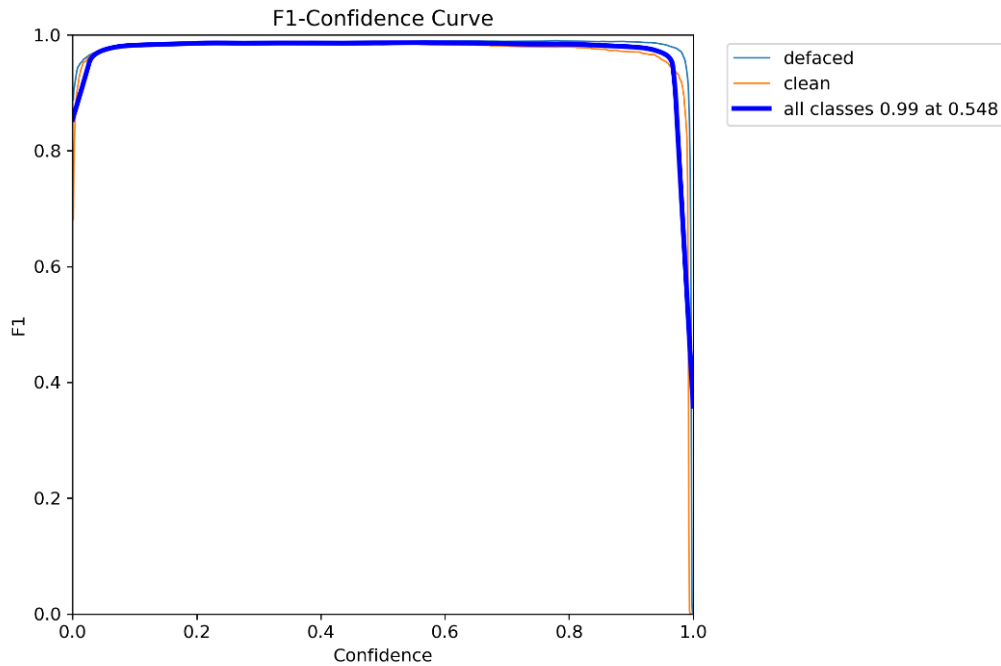


Hình 2.11. Ma trận nhầm lẫn

Còn lại lớp backgroup như trong hình biểu thị một số bức ảnh chưa được chú thích trong tập huấn luyện, do đó mô hình chưa xác định được nhãn gốc. Do đó kết quả của nó không ảnh hưởng tới bảng đánh giá. Mặt khác, trong quá trình thu thập dữ liệu, nghiên cứu đã phân loại thủ công các bức ảnh, trong kết quả đánh giá, mô hình vẫn phân loại được tốt các bức ảnh này, không tồn tại trường hợp không xác định.

2.4.2. Đường cong điểm F1

Đường cong này biểu thị điểm F1 ở các ngưỡng khác nhau. Phân tích và diễn giải đường cong này có thể cung cấp thông tin chi tiết về mức độ cân bằng của mô hình giữa kết quả dương tính giả và âm tính giả trên từng ngưỡng cụ thể.

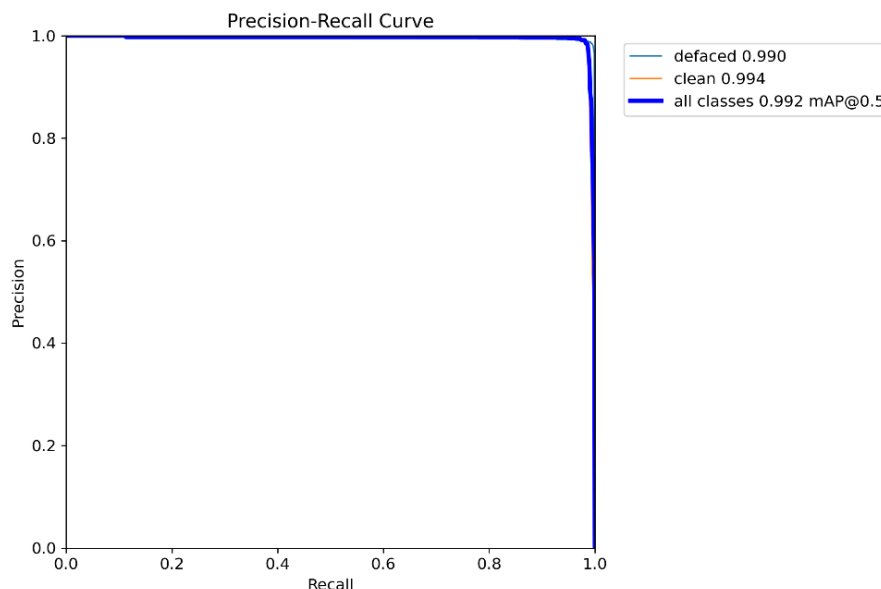


Hình 2.12. *F1-Confidence Curve*

Đường cong nằm rất gần với giá trị $F1 = 1$ trên phần lớn dải confidence, cho thấy mô hình có khả năng dự đoán chính xác rất cao với hầu hết các mức độ tự tin. Các đường cong cho "clean" và "defaced" khá đồng đều, gợi ý rằng mô hình có hiệu suất tương đối ổn định khi phân loại cả hai lớp. Đường cong tổng hợp (all classes) gần sát với giá trị $F1$ cao nhất là 0.98, chứng tỏ mô hình tổng thể hoạt động rất tốt tại mức tự tin 0.307.

Về độ tin cậy của mô hình: Tại mức độ tự tin thấp, giá trị $F1$ giảm mạnh. Điều này cũng hợp lý vì tại confidence thấp, mô hình ít chắc chắn hơn về dự đoán của nó, dẫn đến hiệu suất thấp hơn. Tuy nhiên, khi mức độ tự tin tăng lên, $F1$ score ngày càng tiệm cận đến giá trị tối đa. Điểm chú ý là giá trị $F1 = 0.98$ khi mức độ tự tin là 0.307. Điều này gợi ý rằng điểm ngưỡng tự tin rất thấp, nhưng kết quả vẫn rất tốt. Mô hình này có khả năng duy trì hiệu suất cao ngay cả khi mô hình không quá tự tin về các dự đoán của mình.

2.4.3. Đường cong thu hồi chính xác



Hình 2.13. Biểu đồ Precision-Recall

Đường cong này thể hiện sự đánh đổi giữa độ chính xác và khả năng phát hiện ở các ngưỡng khác nhau. Nó trở nên đặc biệt quan trọng khi xử lý các lớp mất cân bằng. Cả hai lớp "clean" (đường màu xanh nhạt) và "defaced" (đường màu cam) đều thể hiện độ chính xác và độ hồi phục cao, được minh chứng bằng việc các đường cong nằm rất gần góc trên bên phải của biểu đồ. Điều này cho thấy mô hình rất hiệu quả trong việc xác định chính xác các mẫu dương tính với rất ít mẫu dương tính giả và hồi phục cao, nghĩa là nó cũng nắm bắt được hầu hết các mẫu dương tính thật.

Đường cong hiệu suất kết hợp (đường màu xanh đậm) cho tất cả các lớp thể hiện giá trị mAP (mean Average Precision) là 0.992 khi ngưỡng IoU là 0.5. Giá trị mAP gần như hoàn hảo này cho thấy mô hình có hiệu suất cao và ổn định trên cả hai lớp, với sự khác biệt rất nhỏ giữa các đường cong của các lớp riêng lẻ và đường cong tổng hợp. Lớp clean (0.994 Precision-Recall): Số liệu cao này cho thấy mô hình hầu như phân biệt chính xác các mẫu sạch với rất ít sai lệch. Lớp defaced (0.990 Precision-Recall): Tương tự, mô hình rất đáng tin cậy trong việc nhận diện các mẫu bị tấn công. Tất cả các lớp (0.992 mAP@0.5): Giá trị mAP kết hợp là 0.993 cho thấy rằng tổng thể mô hình duy trì hiệu suất cao nhất quán trên các lớp khác nhau.

Biểu đồ Precision-Recall cho thấy mô hình có hiệu suất tuyệt vời đối với cả hai lớp "clean" và "defaced," duy trì độ chính xác và độ hồi phục cao trên toàn bộ các lớp. Tỷ lệ bỏ sót thấp, và tỷ lệ cảnh báo giả gần như bằng 0 (6/1456 mẫu), làm cho mô hình trở thành một giải pháp tin cậy cho nhiệm vụ phân loại được đề ra.

2.4.4. So sánh với các mô hình học sâu khác

Với cùng bộ dữ liệu huấn luyện đầu vào, nghiên cứu đã thực hiện huấn luyện trên 3 mô hình học sâu khác nổi tiếng trong việc phân loại ảnh đó là VGG16, ResNet50, InceptionV3, MobileNetV2. Từ đó có được bảng đánh giá như sau:

Bảng 2. So sánh YOLOv10 với các mô hình học sâu khác

Model	Accuracy	FPR	TPR	PPV	FNR	F1 Score
VGG16	0.94	0.02	0.89	0.96	0.10	0.92
ResNet50	0.87	0.04	0.74	0.92	0.25	0.82
InceptionV3	0.93	0.009	0.85	0.98	0.14	0.91
MobileNetV2	0.96	0.12	0.92	0.97	0.07	0.95
YOLOv10	0.98	0.02	0.99	0.97	0.04	0.98

Nhận thấy mô hình YOLOv10 có hiệu suất vượt trội với độ chính xác cao nhất và các chỉ số TPR, PPV cũng như F1 cao nhất. FPR và FNR đều cực kỳ thấp, cho thấy mô hình này có khả năng phân loại rất tốt với ít lỗi nhất.

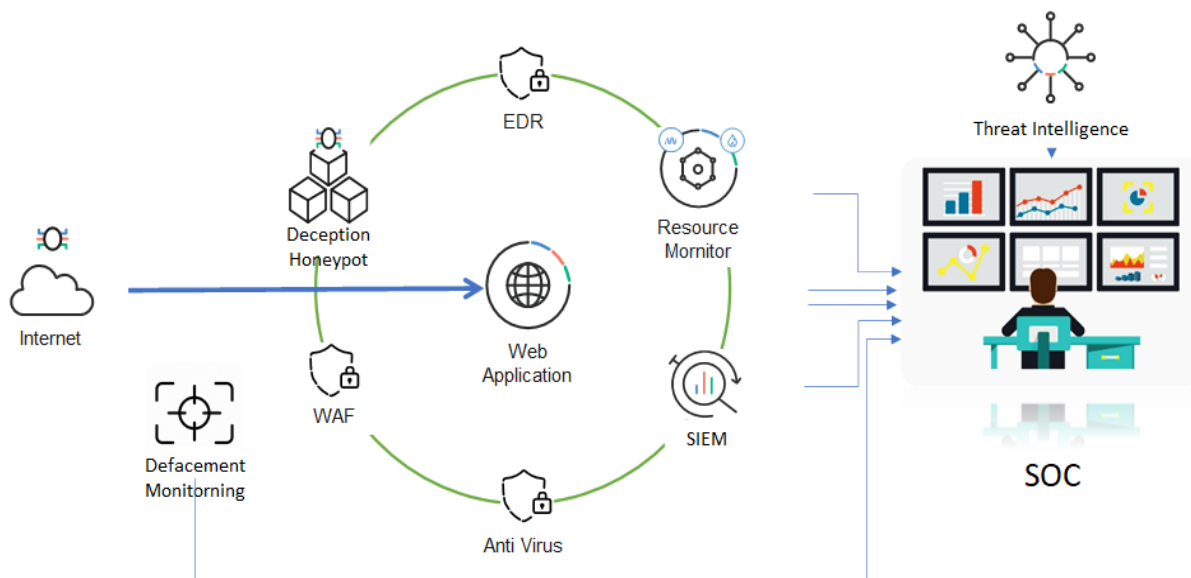
2.5. Kết luận chương 2

Chương này đã tìm hiểu về các phát hiện bất thường trên ảnh số như Autoencoder, Isolation Forest, và thuật toán phân loại ảnh. Ưu nhược điểm của từng thuật toán khi ứng dụng vào đề tài. Cuối cùng xây dựng một mô hình học sâu YOLOv10 đạt độ chính xác cao trong phát hiện tấn công thay đổi giao diện web trên ảnh số.

CHƯƠNG 3: XÂY DỰNG HỆ THỐNG PHÁT HIỆN VÀ CẢNH BÁO TẤN CÔNG THAY ĐỔI GIAO DIỆN WEB

3.1. Đặt vấn đề

Ngày nay, sự bùng nổ thời đại chuyển đổi số ngày càng mạnh mẽ, đi song song với đó là vấn đề an toàn thông tin cũng ngày càng được coi trọng, có rất nhiều công nghệ đảm bảo an toàn ngày nay được phát triển và có độ chính xác ngày càng cao. Tuy nhiên cuộc đua giữa các công nghệ và thuật toán đảm bảo an toàn và kĩ thuật tấn công của kẻ tấn công vẫn dai dẳng, các công nghệ ngày nay vẫn không thể nào ngăn chặn được tất cả các kĩ thuật, chiến thuật tấn công ngày càng tinh vi nên trong những thập kỉ trở lại đây các nhà nghiên cứu tập chung phát triển các công cụ an toàn nhiều vào phần phát hiện hơn là ngăn chặn, hay là phát hiện với Threat Intelligence dựa vào những mối hiểm họa đã biết hoặc dựa vào hành vi của kẻ tấn công để phát hiện mối hiểm họa chưa từng được biết đến. Mối hiểm họa từ khắp nơi cả trong và ngoài tổ chức, việc bị xâm nhập vào hệ thống là điều không thể tránh khỏi, do đó vấn đề đặt ra là làm thế nào để phát hiện và ứng phó với các mối nguy hại này nhanh nhất và chính xác nhất.



Hình 3.1. Một số giải pháp bảo vệ cho hoạt động ứng dụng web

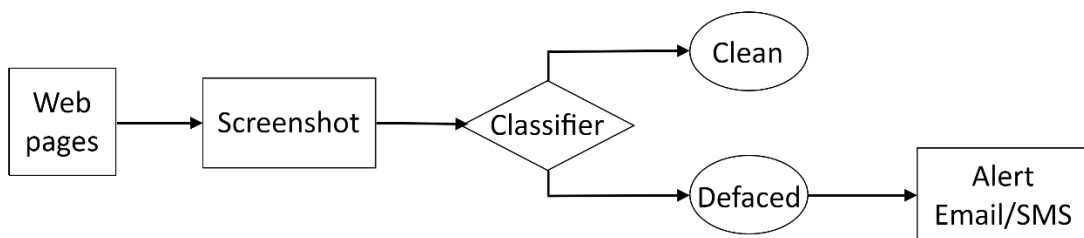
Các máy chủ web ngày nay cũng được đứng dưới các lớp bảo vệ ngày càng chắc chắn. Tuy nhiên không phải tổ chức nào cũng có đủ kinh phí để trang bị

đầy đủ tất cả các giải pháp đảm bảo an toàn, hay các giải pháp cũng còn chưa hoàn toàn toàn diện, các kỹ thuật tinh vi vẫn có thể vượt qua. Việc bị tấn công thay đổi giao diện web là điều chứng tỏ sự thiếu sót đó trong các lớp bảo mật, nhưng quản trị viên cũng cần phải nhanh chóng nắm bắt được thông tin này để đưa ra kịch bản điều tra, ứng phó giảm thiệt hại cho tổ chức nhanh nhất. Hệ thống cảnh báo phát hiện cảnh báo tấn công thay đổi giao diện web sẽ giúp đỡ người quản trị pha này. Hoạt động theo dõi trang web như góc nhìn phía người dùng thông thường, hệ thống có thể nhanh chóng phát hiện ra giao diện trang web đã có dấu hiệu bị thay đổi bởi kẻ tấn công, lập tức cảnh báo về tin nhắn và email cho quản trị viên, đặc biệt có thể tương tác sự kiện này với hoạt động của hệ thống SOC.

3.2. Xây dựng hệ thống cảnh báo

3.2.1. Các thành phần hệ thống cảnh báo

Hệ thống cảnh báo của nghiên cứu có những thành phần như sau:



Hình 3.2. Sơ đồ phát hiện và cảnh báo

Đầu tiên, từ một trang web được theo dõi, sau khoảng một thời gian nhất định (thời gian tùy thuộc vào người dùng cấu hình) sẽ chụp lại ảnh giao diện của trang web đó. Cốt lõi của bộ phân loại là mô hình học máy đã được huấn luyện như nghiên cứu đã trình bày ở chương trước để xác định xem nó đã bị tấn công thay đổi giao diện hay chưa, nếu đã bị tấn công sẽ gửi cảnh báo về email hoặc tin nhắn cho người dùng.

3.2.2. Phân tích thiết kế hệ thống

3.2.2.1. Kịch bản hệ thống

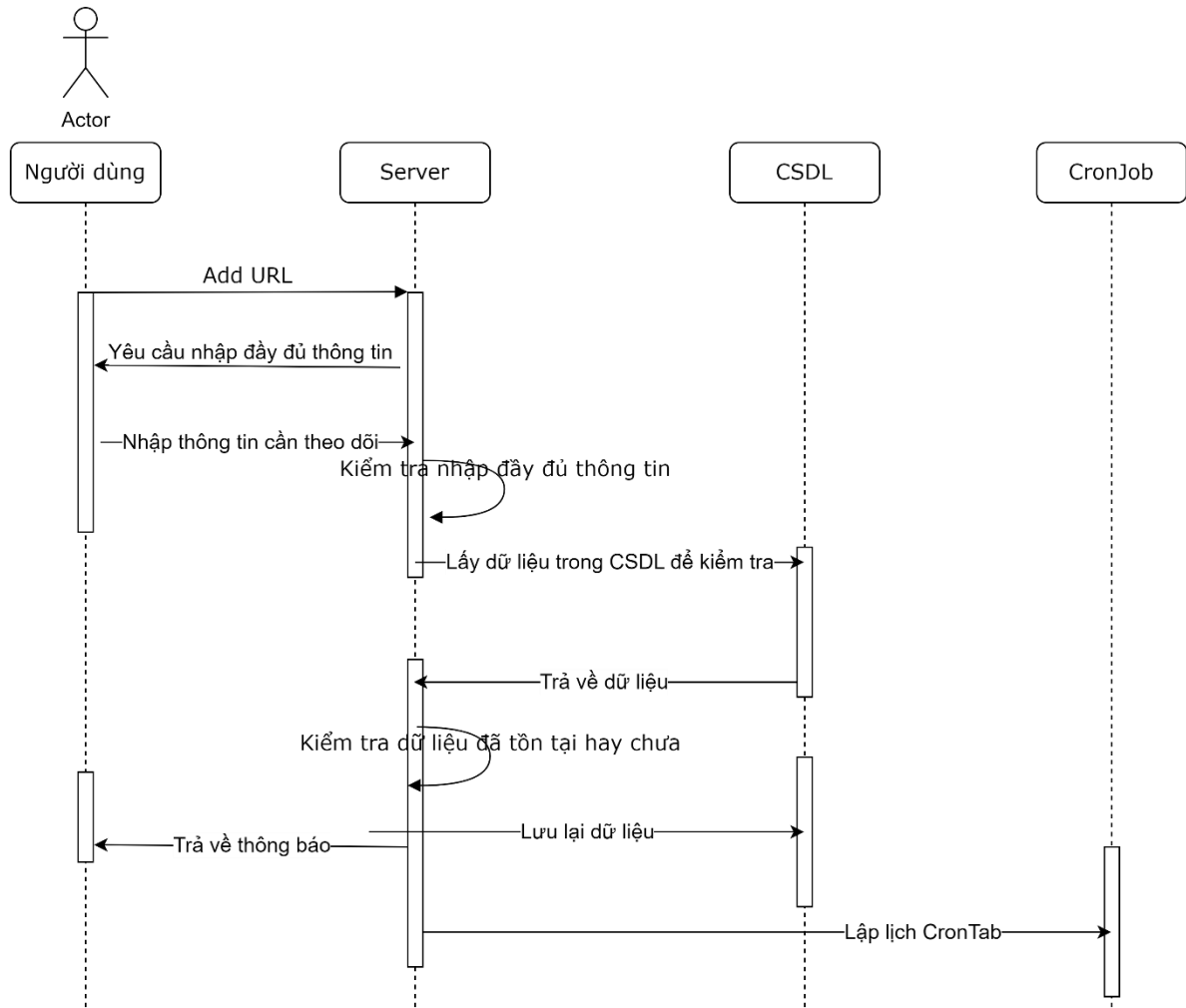
Nghiên cứu sử dụng thư viện Flask trong Python3 để viết giao diện tương tác người dùng. Từ giao diện, chương trình sẽ yêu cầu người dùng nhập thông

tin URL trang web cần theo dõi, email nhận cảnh báo, và cuối cùng là khoảng thời gian mỗi lần kiểm tra.

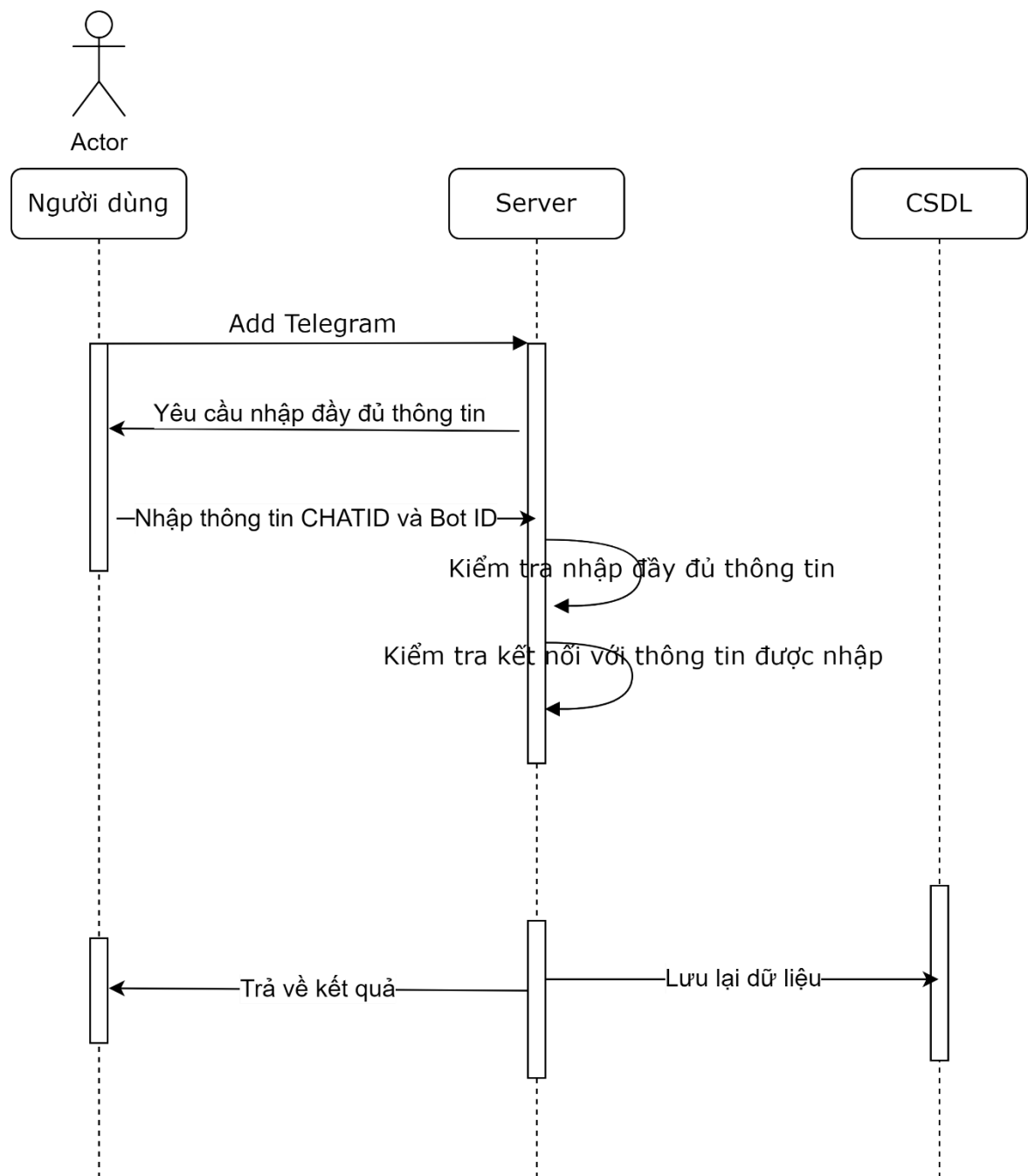
Kịch bản người dùng đăng kí theo dõi một trang web:

Tên usecase	Đăng kí URL
Tác nhân chính	Khách hàng
Điều kiện	Tên email là duy nhất trong hệ thống Tên URL là duy nhất trong hệ thống
Chuỗi sự kiện chính: 1. Khách hàng truy cập hệ thống ở phần đăng kí 2. Hệ thống hiển thị form nhập 3. Khách hàng nhập thông tin bao gồm địa chỉ email, URL, khoảng thời gian kiểm tra mỗi X phút/lần và bấm nút “<i>Add URL</i>” 4. Hệ thống kiểm tra thông tin và báo đăng kí thành công và chuyển hướng về trang chủ.	
Ngoại lệ: 1. Khách hàng nhập thiếu thông tin 1.1. Hệ thống báo lỗi phải nhập đầy đủ thông tin 2. Người dùng nhập email đã tồn tại trong hệ thống 2.1. Hệ thống báo lỗi email đã tồn tại 3. Người dùng nhập email đã tồn tại trong hệ thống 3.1. Hệ thống báo lỗi URL đã tồn tại	

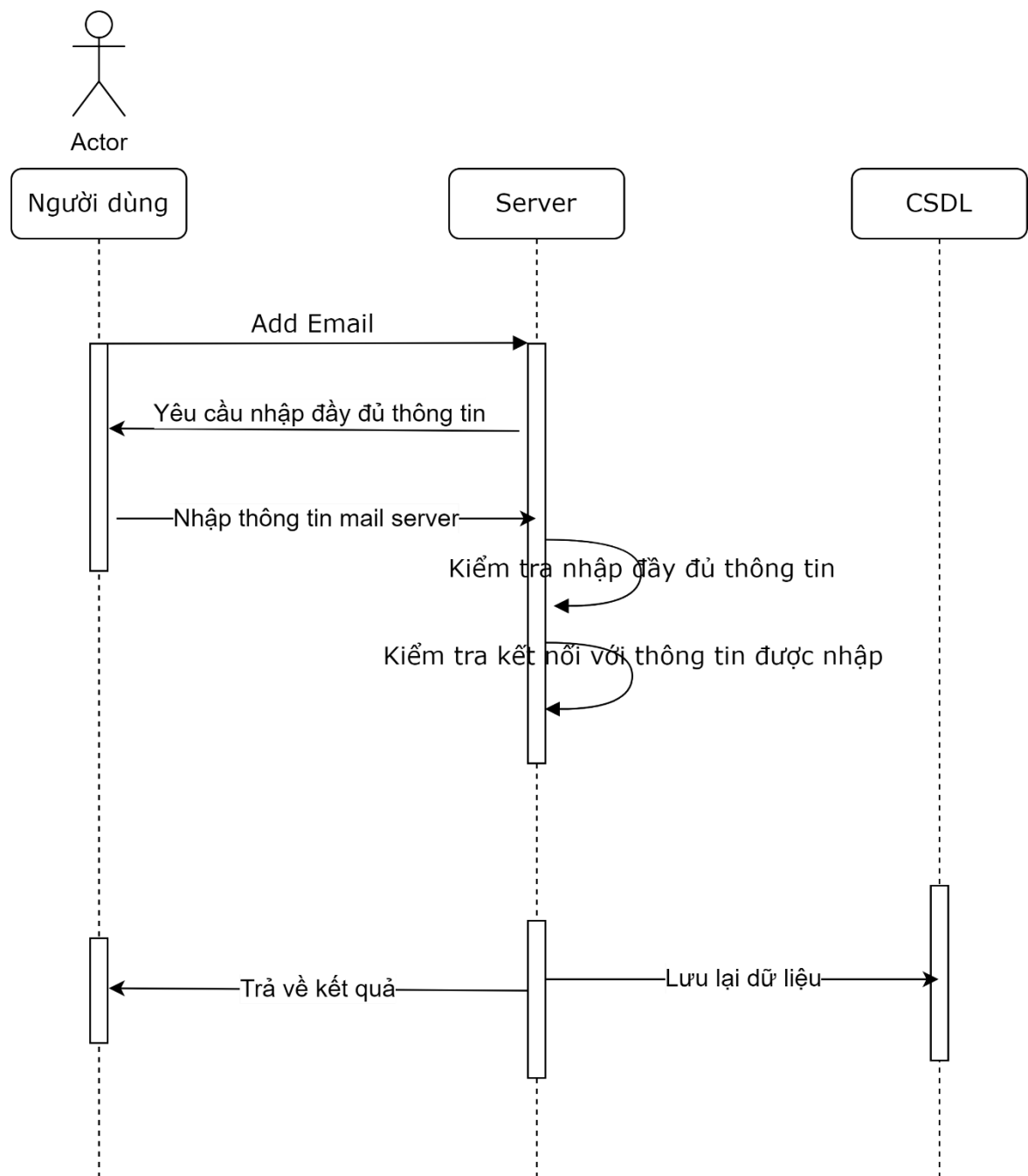
3.2.2.2. Biểu đồ tuần tự



Hình 3.3. Biểu đồ tuần tự đăng kí theo dõi URL

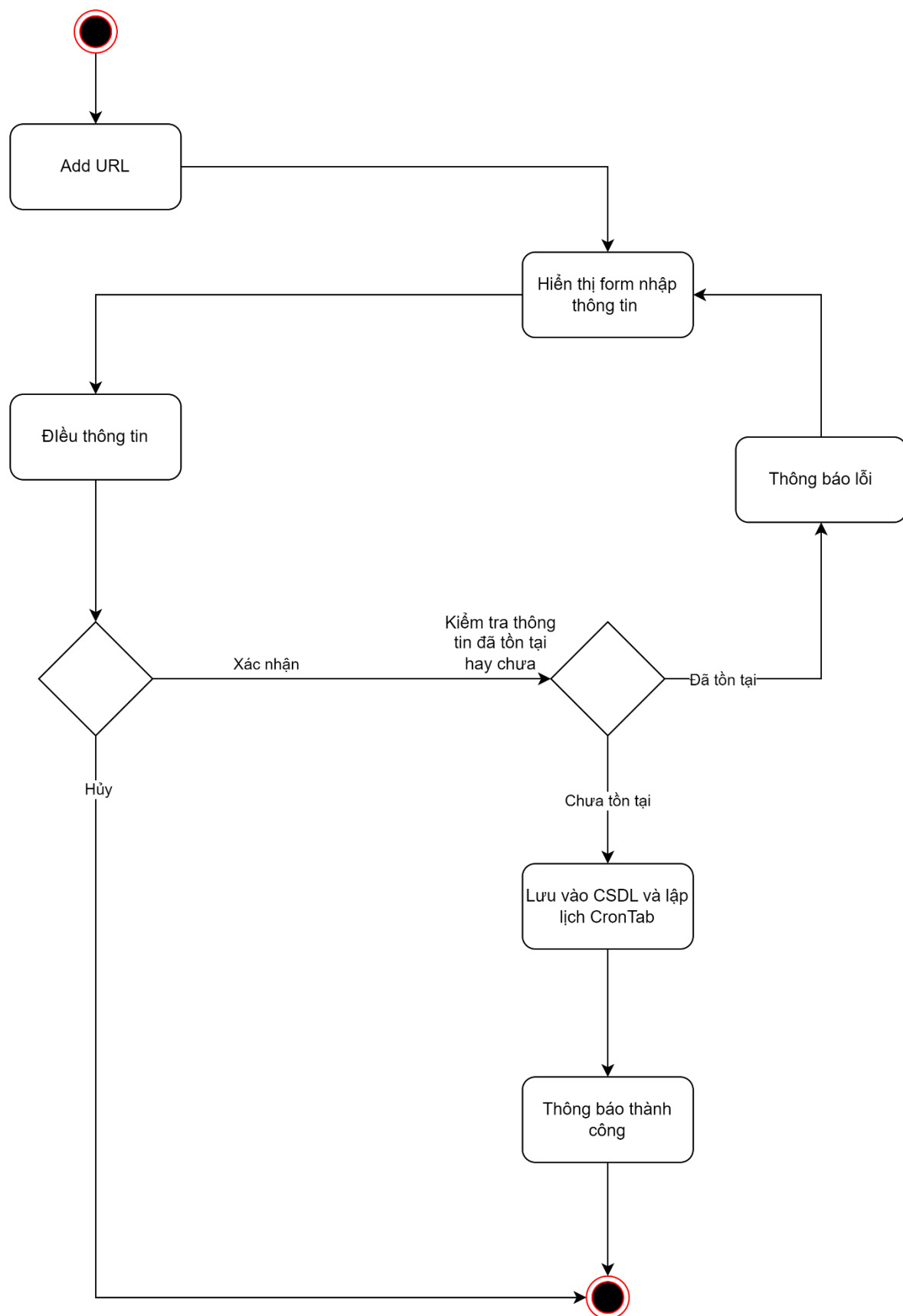


Hình 3.4. Biểu đồ tuần tự đăng kí máy chủ Telegram

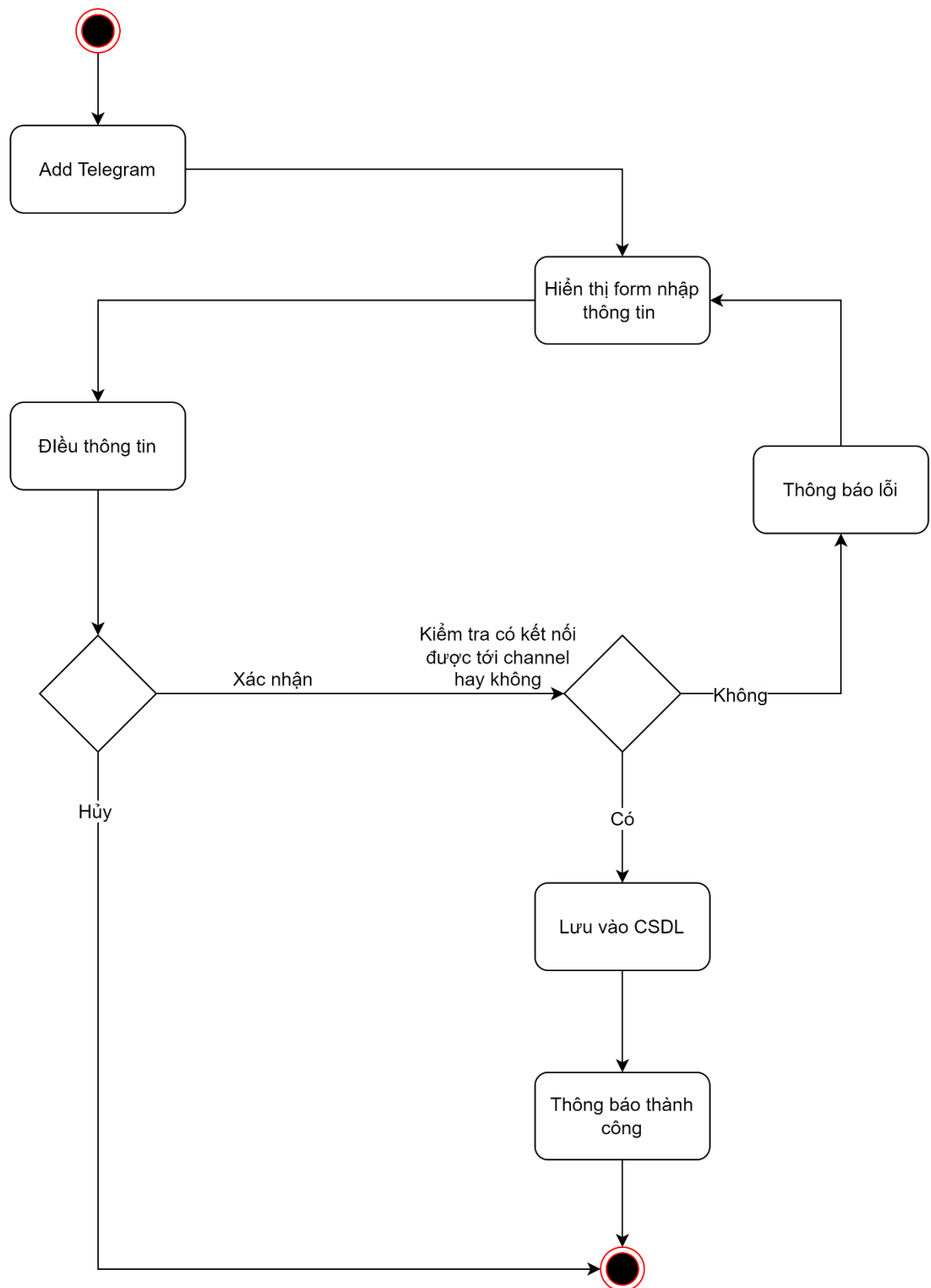


Hình 3.5. Biểu đồ tuần tự đăng kí máy chủ Email

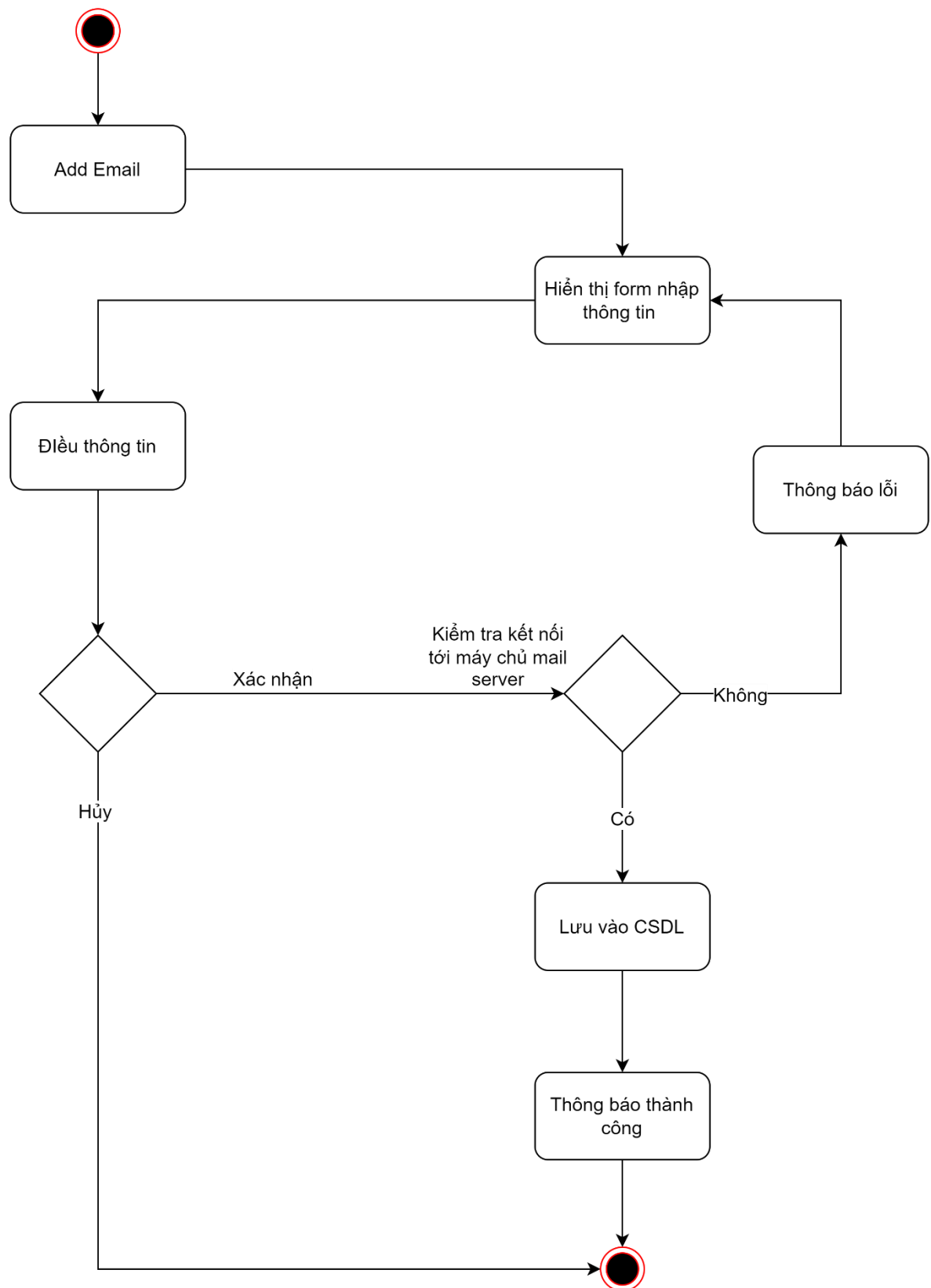
3.2.2.3. Biểu đồ hoạt động



Hình 3.6. Biểu đồ hoạt động nhập URL theo dõi



Hình 3.7. Biểu đồ hoạt động đăng kí máy chủ Telegram



Hình 3.8. Biểu đồ hoạt động đăng kí máy chủ mail server

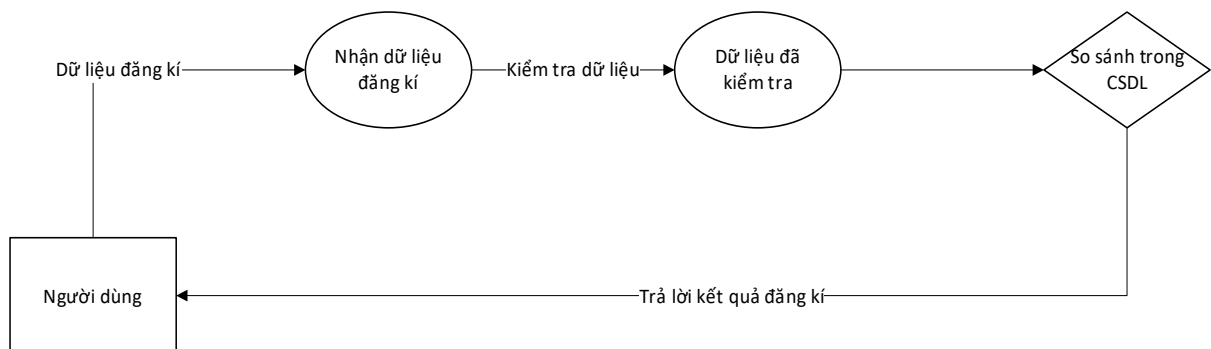
3.2.3. Triển khai hệ thống

a) Thiết kế cơ sở dữ liệu

Hệ thống sử dụng cơ sở dữ liệu NoSQL là MongoDB, nên không yêu cầu về ràng buộc khóa. Cấu trúc của một tài liệu (Documents) lưu trữ thông tin:

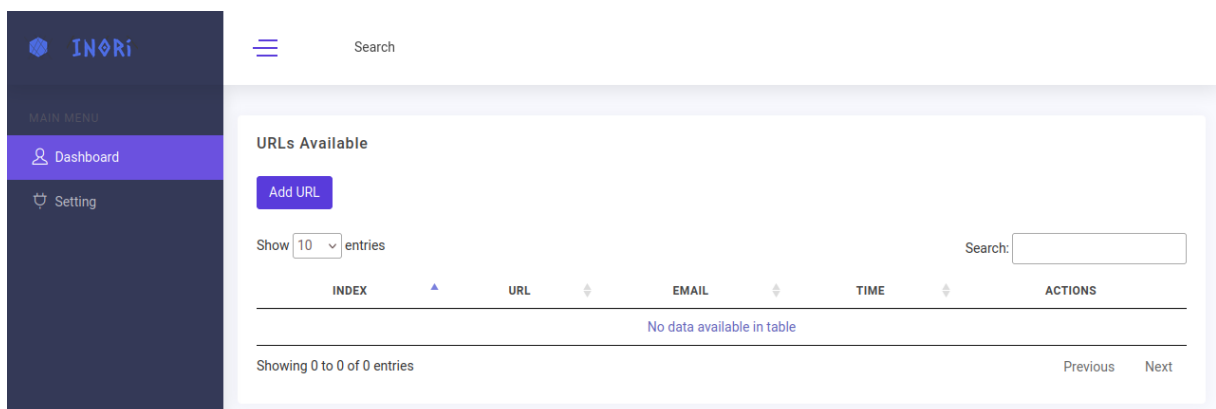
```
{
  _id: '',
  email: '',
  time: '1',
  active_key: '',
  domain: ''
}
```

- Sơ đồ luồng dữ liệu:

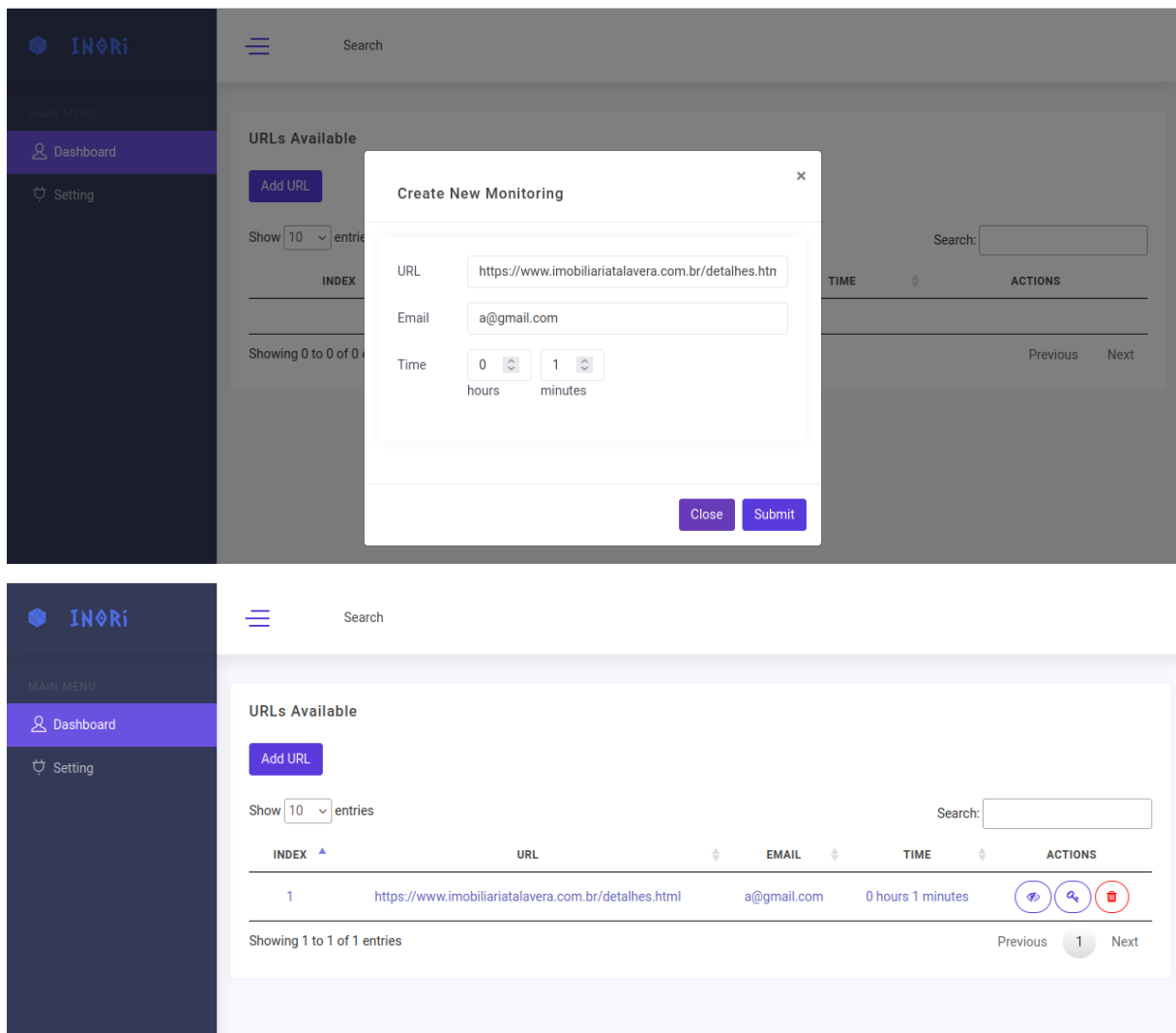


Hình 3.9. Sơ đồ đăng kí theo dõi một URL

a) Thiết kế giao diện đăng kí



Hình 3.10. Giao diện nhận thông tin cấu hình từ người dùng



Hình 3.11. Giao diện đăng kí theo dõi một trang web

Khi người dùng chọn “Add URL”, tệp tin `app.py` sử dụng Python Flask xử lý yêu cầu từ phía người dùng, dòng 80 gọi hàm `insert_data()` từ tệp tin `database.py` để lưu dữ liệu vào MongoDB và dòng 83-86 gọi hàm `create()` từ tệp tin `schedule.py` để tạo lệnh CronTab.

```

55 @app.route("/register", methods=["POST"])
56 def register():
57     url = slug(request.form["url"])
58     email = slug(request.form["email"])
59     hours = slug(request.form["hours"])
60     minutes = slug(request.form["minutes"])
61     int_format = re.compile("[0-9]+$")
62
63     if len(url) == 0 or len(email) == 0 or len(hours) == 0 or len(minutes) == 0:
64         return "Null"
65
66     if re.match(int_format, hours) is False or re.match(int_format, minutes) is False:
67         return "Null"
68
69     if hours == minutes == "0":
70         return "Null"
71
72     db = database.Database("site")
73     time = f"{int(hours)} hours {int(minutes)} minutes"
74     data = {"email": email, "time": time, "active_key": "", "url": url}
75     check = 0
76     for check in db.get_multiple_data():
77         if check["url"] == url:
78             check = 1
79     if check == 1:
80         response = "EXIST"
81     else:
82         db.insert_data(data)
83         if int(hours) == 0:
84             sch.create(url, email, None, minutes)
85         if int(minutes) == 0:
86             sch.create(url, email, hours, None)
87         response = "OKE"
88     return response

```

Thông qua tệp tin `main.py` đã tạo trước đó, Cron sẽ có thể tự động kích hoạt chương trình. Dòng 11 sẽ lấy mã băm của URL để tạo comment cho lệnh CronTab, thông qua comment chương trình mới có thể trở tới chính xác câu lệnh để xóa hay cập nhật câu lệnh.

```

1  from hashlib import md5
2  from getpass import getuser
3  from crontab import CronTab
4
5  user = getuser()
6  my_cron = CronTab(user=user)
7
8
9  def create(domain, email, hours=None, minutes=None):
10     command = f"python3 /opt/In0ri/main.py {domain} {email}"
11     comment = md5(domain.encode()).hexdigest()
12     check = 0
13     for job in my_cron:
14         if job.comment == comment:
15             check = 1
16     if check == 1:
17         print("This domain is available!")
18     else:
19         job = my_cron.new(command=command, comment=comment)
20         if hours is not None:
21             job.hour.every(hours)
22         if minutes is not None:
23             job.minute.every(minutes)
24         my_cron.write(user=user)
25         print(job.is_valid())
26
27
28 def edit(domain, email, hours=None, minutes=None):
29     command = f"python3 /opt/In0ri/main.py {domain} {email}"
30     comment = md5(domain.encode()).hexdigest()
31     check = 0
32     for job in my_cron:
33         if job.comment == comment:
34             check = 1

```

b) Chụp ảnh giao diện

Bước đầu tiên của một hệ thống cảnh báo là phải chụp lại được bức ảnh của trang web tại thời điểm đó. Trong tệp tin `screenshot.py` ta có:

```

15 def screenshot(url):
16     width = 1366 # Set your desired width
17     height = 768 # Set your desired height
18     options = webdriver.ChromeOptions()
19     options.headless = True
20     options.add_argument(f"--window-size={width},{height}")
21     options.add_argument("--start-maximized")
22     options.add_argument('--disable-dev-shm-usage')
23     options.add_argument('--ignore-certificate-errors')
24     options.add_argument('--disable-setuid-sandbox')
25     options.add_argument('--no-sandbox')
26     options.add_argument('--headless')
27     options.add_experimental_option("excludeSwitches", ["enable-automation"])
28     options.add_experimental_option('useAutomationExtension', False)
29     options.add_argument("--disable-blink-features=AutomationControlled")
30     driver = webdriver.Chrome(options=options)
31
32
33     name = hashlib.md5(url.encode())
34     try:
35         if not url.startswith("http://") and not url.startswith("https://"):
36             url = "https://" + url
37         driver.get(url)
38         # driver.implicitly_wait(2)
39         print("Screenshots..." + url)
40         driver.get_screenshot_as_file(
41             "/opt/In0ri/FlaskApp/static/images/" + name.hexdigest() + ".png"
42         )
43         driver.quit()
44     except:
45         print(url + " was died")
46         pass
47     return "/opt/In0ri/FlaskApp/static/images/" + name.hexdigest() + ".png"

```

Trong đoạn mã trên, nghiên cứu sử dụng Python Selenium sẽ gọi Chromium để duyệt url, các dòng từ 20 đến 29 sẽ giúp ẩn đi các cảnh báo (nếu có lỗi), cho phép Chromium thực hiện duyệt web ngầm dưới hệ thống, không hiển thị giao diện trình duyệt. Tại dòng 20 sử dụng cấu hình `--window-size` để chụp với chế độ duyệt web của máy tính, nếu để mặc định sẽ chụp với chế độ góc nhìn điện thoại, vì nhiều trang web chèn quảng cáo nó có thể kín màn hình điện thoại luôn, sẽ gây cảnh báo nhầm.

c) Kiểm tra trạng thái trang web

Sau khi đã có ảnh chụp trang web, tệp tin `checkdefaced.py` sẽ đưa bức ảnh vào mô hình đã huấn luyện trước đó để phân loại ảnh xem có bị thay đổi giao diện hay không.


```

1 import hashlib
2 from ultralytics import YOLO
3 model = YOLO('/opt/In0ri/model.pt')
4
5 def check(images_path):
6     results = model.predict(images_path)
7     for result in results:
8         result.save(filename=f"/opt/In0ri/FlaskApp/static/images/{hashlib.md5(images_path.encode()).hexdigest()}.jpg")
9         for i in range(0,1):
10             id=result.probs.top5[i]
11             return result.names[id]

```

Đầu vào hàm `check()` là đường dẫn bức ảnh đã chụp. Dòng 3 sử dụng tải mô hình của YOLO đã huấn luyện. Tại dòng 11, sau khi phân loại xong, lấy nhãn ở trong `probs.top5[i]` – đây là nhãn kết quả của mô hình dự đoán ra.

Cuối cùng là tệp tin `main.py`, đây là tệp tin liên kết các thành phần của các tệp tin trước đó lại thành một hệ thống, sử dụng thư viện `argv` để lấy đầu vào từ cửa sổ dòng lệnh, làm tăng khả năng tích hợp với các thành phần mở rộng khác.

```

1 from sys import argv
2
3 import alert
4 from checkdefaced import check
5 from screenshot import screenshot
6
7 script, url, receiver = argv
8
9
10 def main(url):
11     al = alert.Alert()
12     print(url)
13     img_path = screenshot(url)
14     defaced = check(img_path)
15     if defaced == 'defaced':
16         al.sendBot(url, img_path)
17         subject = "Website Defacement"
18         message = f"You website was defaced!\nURL: {url}"
19         al.sendMessage(receiver, subject, message, img_path)
20         print("Website was defaced!")
21     else:
22         print("Everything oke!")
23
24
25 main(url)

```

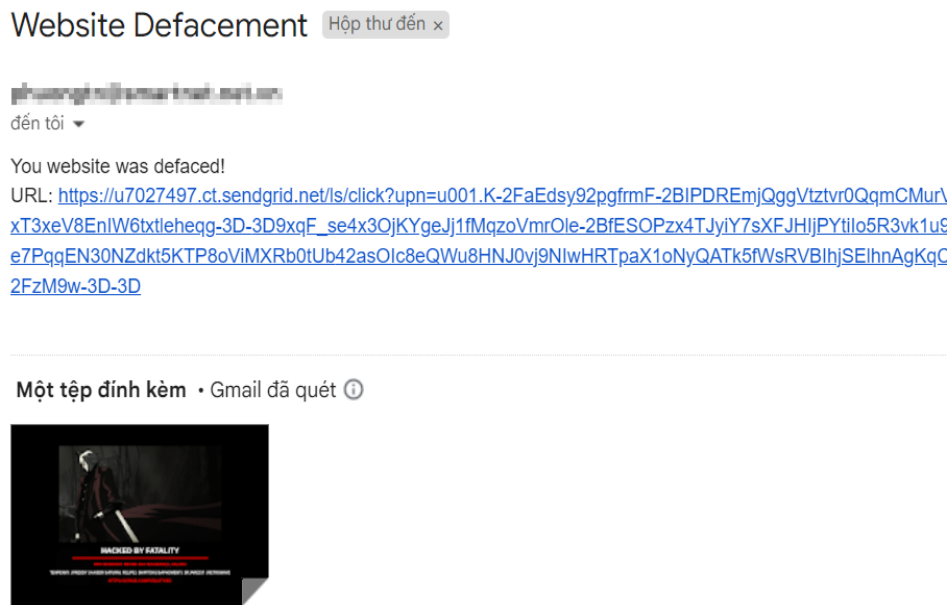
Khi có được kết quả dự đoán, nghiên cứu sẽ gửi cảnh báo về email của người dùng, ngoài ra còn có một kênh chatbot cảnh báo về một chanel trên Telegram cho người quản trị. Ví dụ, để thực thi chương trình `main` chỉ cần thực hiện câu lệnh:

→ `python3 main.py https://dantri.com.vn htnguyenbg@gmail.com`

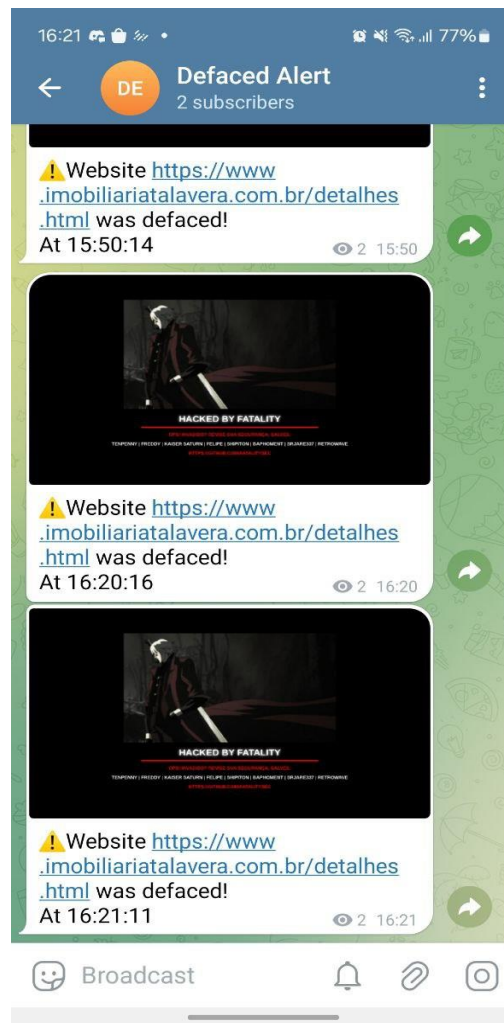
Kết quả: Sau khi nhấn nút “Add URL” hệ thống đã lưu lại dữ liệu người dùng nhập lên MongoDB và tạo được một lệnh lập lịch tự động CronTab.

```
GNU nano 6.2 /tmp/crontab.GehiTV/crontab
* * * * * python3 /opt/In0ri/main.py https://www.imobiliariatalavera.com.br/detalhes.html htnguyenbg@gmail.com # 9647495465fae21d11443e
```

Hình 3.12. Câu lệnh CronTab sau khi đăng kí



Hình 3.13. Cảnh báo được gửi về email



Hình 3.14. Cảnh báo được gửi về tin nhắn Telegram

3.3. Triển khai Agent

Mặc dù hệ thống cảnh báo mà nghiên cứu đã trình bày ở phần trên đã hoạt động tốt với độ chính xác tổng quát lên tới 98%. Tuy nhiên khi nghiên cứu sâu hơn về hành vi của các nhóm tin tặc trên các trang như Zone-Kurd-h, Mirror-h, Zone-h có thể thấy vị bị thay đổi giao diện rất nhiều là không nằm ở trang chủ mà nằm ở đường dẫn thứ cấp, tệp tin index không phải tên là index.html; index.php; index.jsp; ... mà bị đặt tên khác theo chữ kí của nhóm tin tặc (như hình 3.11).

1337/O	 (US)	https://www.dawnsdigitaldesigns.co.uk/kolihani/upload/files/q2p.html	143.95.38.195	24/05/2021	🔍
1337/O	 (SY)	http://sinbad.sy/admin/kctindex/uj/index/kes/KPNe	185.158.92.42	24/05/2021	🔍
1337/O	 (SY)	https://bankofjordansyria.sy/city/default/index/soalform-1.png	185.194.124.105	24/05/2021	🔍
attacker	 (US)	https://www.winmeisters.co	184.170.148.132	24/05/2021	🔍
Emilyano Zapota	 (TH)	http://www.sanid.go.th/EmilyanoZapota.html	1179.156.98	24/05/2021	🔍
Ayyildiz Tim	 (TR)	https://www.eraykirma.com/	62.244.220.36	24/05/2021	🔍
Ayyildiz Tim	 (BR)	http://www.limeiraalmeida.com.br/site/ajv/	186.202.95.3	24/05/2021	🔍
Senyum Gan :)	 (DE)	http://abst-brandschutztechnik.at/images/senyum.jpg	189.237.137.122	24/05/2021	🔍
Senyum Gan :)	 (DE)	http://www.dieter-sauter.com/images/senyum.jpg	134.119.234.57	24/05/2021	🔍

2021/05/21	1877	 US	198.187.29.115	http://www.maxcartdeals.com/1877.html
2021/05/21	1877	 US	198.187.29.115	http://www.kainyn.com/1877.html
2021/05/21	1877	 US	172.67.178.64	http://www.idlish.com/1877.html
2021/05/21	1877	 US	198.187.29.115	http://www.glocalwave.info/1877.html
2021/05/21	1877	 US	198.187.29.115	http://www.gawcie.com/1877.html
2021/05/21	1877	 US	172.67.132.238	http://www.dvd-easy-workshop.com/1877.html
2021/05/21	1877	 US	198.187.29.115	http://www.disman3.com/1877.html
2021/05/21	1877	 US	104.21.41.42	http://www.discount-valtrex.com/1877.html
2021/05/21	1877	 US	198.187.29.115	http://www.descrevos.com/1877.html
2021/05/21	1877	 US	104.21.71.210	http://www.criminalrecordsostudy.stream/
2021/05/21	1877	 US	172.67.138.16	http://www.chat901.info/1877.html
2021/05/21	1877	 US	198.187.29.115	http://www.bocaiboss.com/1877.html
2021/05/21	1877	 US	172.67.218.41	http://www.bestrefrigeratordeals.com/187
2021/05/21	1877	 US	198.187.29.115	http://www.alanseocompany.com/1877.html
2021/05/21	1877	 IN	103.21.59.7	http://www.royalhomespaces.com/1877.html
2021/05/21	1877	 IN	103.21.59.7	http://www.fbilindia.in/1877.html
2021/05/21	1877	 US	162.241.2.45	http://www.mfcp.com.br/1877.html
2021/05/21	1877	 GB	35.214.48.223	http://www.xn--ffnenfinder-qfb.de/1877.h

Hình 3.15. Các vị trí bị thay đổi giao diện (theo Mirror-h, Zone-Kurd-h)

Rõ ràng việc theo dõi duy nhất độc trang chủ (ví dụ: <https://example.com/>) không thể phát hiện được toàn bộ các chiến dịch tấn công, để giải quyết vấn đề này của hệ thống nghiên cứu có thể đưa ra một số giải pháp như: thêm các đường dẫn quan trọng cần theo dõi, crawl các liên kết con của trang web sau đó theo dõi toàn bộ.

Tuy nhiên, các giải pháp này có một số nhược điểm trước các nhóm tin tặc như sau. Đối với giải pháp thêm các đường dẫn quan trọng để theo dõi, thứ nhất sẽ cần phải làm thủ công rất là nhiều, do đó có thể bỏ sót những vị trí không ngờ tới. Đối với giải pháp crawl các liên kết con của trang web, bản thân các công cụ hay script để crawl được thực hiện theo các cách như: thứ nhất là dạng vét cạn theo từ điển, tức là có một tập từ điển có sẵn về các đường dẫn con, các

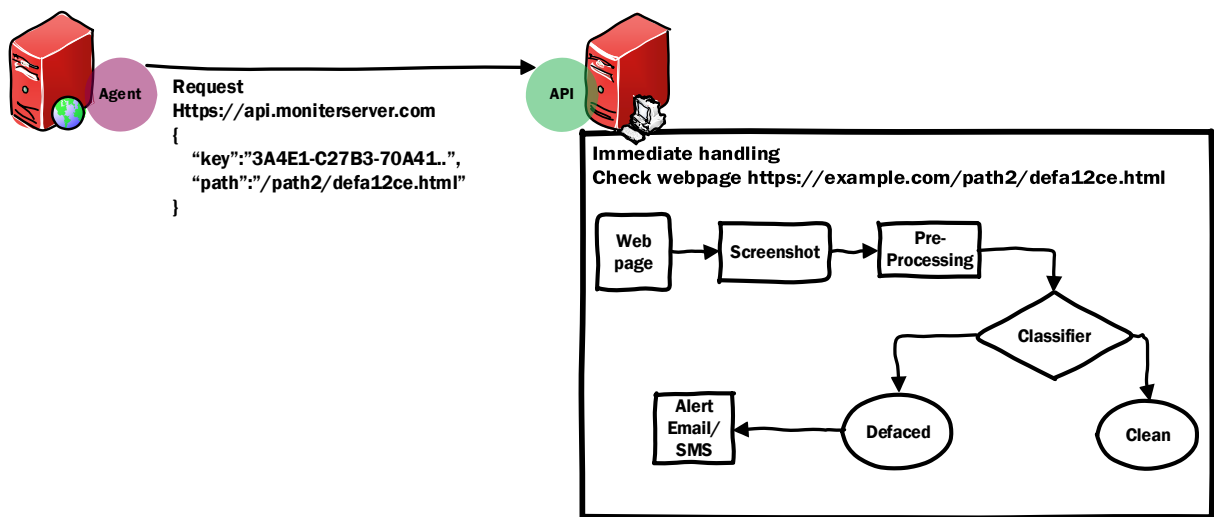
công cụ sẽ thêm vào URL gốc và theo dõi mã trạng thái trả về để biết được có đường dẫn đó tồn tại hay không, tiếp tục theo độ sâu mà người dùng cấu hình, kết quả linh hoạt động của bước trước sẽ được làm đầu vào để vết cận tiếp; thứ hai là tìm kiếm theo dấu vết, tức là các công cụ sẽ duyệt các liên kết cho trước, tìm trong trang phản hồi (dạng html) của trang web đó có đính kèm các liên kết nào, chương trình sẽ bóc tách các liên kết đó ra và duyệt tiếp và tiếp tục bóc tách liên kết khi nhận được phản hồi, công việc cứ tiếp tục đến khi nào không còn liên kết mới xuất hiện thì thôi. Hai phương pháp này có thể kết hợp với nhau để crawl một các sâu rộng, tuy nhiên khi nhìn vào các trang báo cáo tấn công ta thấy các tệp tin thay đổi giao diện có tên đặc trưng nên có thể phần lớn nó không có trong tập từ điển, và nó cũng không được đính kèm vào bất kì trang web vào hiện có nên hệ thống cảnh báo không thể phát hiện được.

Để giải quyết khó khăn này, nghiên cứu đề xuất giải pháp cài một agent lên máy chủ. Agent này sẽ làm nhiệm vụ theo dõi sự thay đổi của các tệp tin trong hệ thống, những tệp tin mới được đưa vào hệ thống một cách liên tục, sau đó nó sẽ gửi các thông tin này lên phía máy chủ. Khi nhận được thông tin từ các agent, máy chủ sẽ duyệt đường dẫn theo các tệp tin bị thay đổi và chụp ảnh màn hình để kiểm tra có phải bị thay đổi giao diện ở đó hay không tương tự như hệ thống cảnh báo đã trình bày ở phần trước.

***Lưu ý:** Agent sẽ chỉ là một tùy chọn, một số nơi có thể sẽ không cho phép cài agent lên máy chủ web do chính sách riêng của tổ chức hoặc vấn đề lo ngại ảnh hưởng tới hiệu năng. Không có agent hệ thống cảnh báo vẫn có thể hoạt động tốt.*

3.3.1. Các thành phần hệ thống kết nối Agent

Hệ thống cảnh báo có sử dụng agent cài trên máy chủ có mô hình như sau:



Hình 3.16. Sơ đồ xử lý sự kiện từ Agent

Tại máy chủ web, một agent được cài chạy ngầm dưới hệ thống kiểm tra các sự kiện thay đổi mã nguồn web. Khi có sự kiện xảy ra sẽ gửi đường dẫn tệp tin bị thay đổi lên máy chủ theo dõi tấn công thông qua một API. Nghiên cứu sử dụng phương thức xác thực cho API là xác thực sử dụng khóa bí mật, khóa bí mật này chỉ dùng cho một URL duy nhất và email cảnh báo tương ứng đã đăng kí. Nếu không đúng khóa bí mật, hệ thống sẽ từ chối yêu cầu, khóa bí mật này được đăng kí ở trên giao diện web. Ở phía máy chủ phát hiện tấn công, các thành phần phát hiện cảnh báo tương tự như hệ thống đã trình bày ở **phần 3.1**. Thông qua key trong API, máy chủ sẽ lấy ra URL tương ứng trong cơ sở dữ liệu, nối với tên tệp tin rồi được một đường dẫn hoàn chỉnh và kiểm tra đường dẫn này có phải bị tấn công hay không rồi gửi cảnh báo về email tương ứng với URL.

3.3.2. Phiên bản cho hệ điều hành Linux

```
hoang@Inori:~/Project$ tree -L 2 Code/
Code/
├── Agent
│   ├── checkfloder.py
│   ├── config.json
│   ├── LICENSE
│   ├── README.md
│   └── setup.py
├── api.py
├── chatbot.py
├── checkdefaced.py
├── final_model.h5
├── FlaskApp
│   ├── app.py
│   ├── createLicense.py
│   ├── database.py
│   ├── schedule.py
│   ├── sendEmail.py
│   ├── static
│   └── templates
├── images
├── keys
│   ├── dsaparam.pem
│   ├── privkey.pem
│   └── pubkey.pem
├── main.py
└── screenshot.py

6 directories, 19 files
```

Hình 3.17. Cây thư mục các thành phần của Agent

Đầu tiên là thư mục Agent, có các tệp tin quan trọng như:

- `checkfloder.py`, tệp tin có chức năng kiểm tra sự thay đổi, di dời của các tệp tin.
- `config.json`, tệp tin lưu các tham số cấu hình của Agent.
- `setup.py`, tệp tin giúp cài đặt các gói phụ trợ khi cài Agent trên server.

Trên phía Server có các thành phần phục vụ cho chức năng này như:

- `api.py`, tệp tin xử lý các yêu cầu từ phía client.
- `createLicense.py`, tệp tin giúp tạo khóa xác thực cho Agent.
- Thư mục `keys/`, chứa bộ khóa bất đối xứng giúp tạo khóa xác thực cho Agent

a) Theo dõi sự thay đổi trong hệ thống

Tệp tin `checkfloder.py` sử dụng thư viện `watchdog` trong python để theo dõi các sự kiện thay đổi tệp tin.

```
11 f = open(f"{path}/config.json", "r")
12 config = json.load(f)
13 key = config["key"]
14 excludePath = config["excludePath"]
15 server = config["apiServer"]
16 f.close()
17
18
19 def on_modified(event):
20     if len(excludePath) != 0 and re.search(excludePath, event.src_path) is not None:
21         return 0
22     else:
23         print(f"Notification, {event.src_path} has been modified")
24         path = event.src_path
25         path = path.replace(config["rootPath"], "")
26         try:
27             response = requests.post(
28                 server, json={"key": key, "path": path}
29             )
30             print(response.json())
31         except requests.ConnectionError as error:
32             print("Server not found!")
33
34
35 def on_moved(event):
36     if len(excludePath) != 0 and re.search(excludePath, event.src_path) is not None:
37         return 0
38     else:
39         print(
40             f"Notification, File {event.src_path} has been moved to {event.dest_path}"
41         )
42         path = event.dest_path
43         path = path.replace(config["rootPath"], "")
44         try:
45             response = requests.post(
46                 server, json={"key": key, "path": path}
47             )
48             print(response.json())
49         except requests.ConnectionError as error:
```

Chương trình tải cầu hình từ tệp tin `config.json`, lý do nghiên cứu tách ra là do sau này agent sẽ được đóng gói lại thành một tệp tin (trừ tệp tin `config.json`) giúp thuận lợi cho việc đem đi cài đặt trên các máy chủ web, người dùng không thể can thiệp vào mã nguồn của agent. Chương trình kiểm tra hai sự kiện là thay đổi và di chuyển tệp tin. Khi có sự kiện thay đổi sẽ lập tức tạo một yêu cầu lên máy chủ thông qua một API để kiểm tra xem tệp tin bị thay đổi đó có phải do kẻ tấn công thực hiện thay đổi giao diện hay không.


```

53 if __name__ == "__main__":
54     patterns = [
55         "*.html",
56         "*.htm",
57         "*.php",
58         "*.txt",
59         "*.jsp",
60         "*.aspx",
61         "*.shtml",
62         "*.hta",
63     ]
64     ignore_patterns = None
65     ignore_directories = False
66     case_sensitive = True
67     my_event_handler = PatternMatchingEventHandler(
68         patterns, ignore_patterns, ignore_directories, case_sensitive
69     )
70     my_event_handler.on_modified = on_modified
71     my_event_handler.on_moved = on_moved
72
73     path = config["rootPath"]
74     go_recursively = True
75     my_observer = Observer()
76     my_observer.schedule(my_event_handler, path, recursive=go_recursively)
77
78     my_observer.start()
79     try:
80         while True:
81             time.sleep(1)
82     except KeyboardInterrupt:
83         my_observer.stop()
84         my_observer.join()

```

Như tại dòng 54, chương trình sẽ chỉ theo dõi sự kiện tại một số tệp tin nhất định để giảm các sự kiện không cần thiết và có thể gây cảnh báo sai.

3.3.3. Phiên bản cho hệ điều hành Windows

Để phát triển dễ dàng phát triển agent và hoạt động ổn định trên môi trường Windows, nghiên cứu sử dụng ngôn ngữ C# để phát triển. FileSystemWatcher, một lớp trong bộ thư viện .NET của Microsoft, cơ chế hoạt động của FileSystemWatcher là lắng nghe các sự kiện thay đổi trên hệ thống tập tin như: tạo mới, chỉnh sửa, xóa bỏ, hoặc đổi tên tệp tin/thư mục trong một thư mục được chỉ định. Thông qua đó, ứng dụng có thể ghi nhận các sự kiện này và thực hiện các hành động tiếp theo một cách tự động.

Agent phát triển dựa trên FileSystemWatcher sẽ giúp triển khai hiệu quả các giải pháp quản lý truy cập an ninh, theo dõi thực thi chính sách lưu trữ, và cung cấp cái nhìn toàn diện về tình trạng hoạt động của hệ thống tập tin trong môi trường doanh nghiệp hoặc cá nhân. FileSystemWatcher trực tiếp sử dụng

Windows API giúp tối ưu và tùy chỉnh sâu hơn trong việc quản lý theo dõi tệp tin và thư mục.

Sau đây nghiên cứu tạo một chương trình sử dụng FileSystemWatcher để theo dõi thư mục được chỉ định tại thời điểm chạy. Thành phần được thiết lập để theo dõi các thay đổi trong LastWrite và LastAccess, việc tạo, xóa hoặc đổi tên các tệp văn bản trong thư mục. Nếu một tệp bị thay đổi, tạo mới, đường dẫn đến tệp sẽ được ghi lại sự kiện và gửi tới server.

Dưới đây là mã nguồn theo dõi sự kiện:

```
23 0 references
24 static async Task Main()
25 {
26     EnsureConfigFileExists();
27
28     Config config = JsonSerializer.Deserialize<Config>(File.ReadAllText(ConfigFilePath));
29
30     if (!Directory.Exists(config.RootPath))
31     {
32         Console.WriteLine("The specified root path does not exist.");
33         return;
34     }
35
36     using (FileSystemWatcher watcher = new FileSystemWatcher())
37     {
38         watcher.Path = config.RootPath;
39         watcher.Filter = "*.*";
40         watcher.IncludeSubdirectories = true;
41         watcher.NotifyFilter = NotifyFilters.FileName | NotifyFilters.LastWrite;
42
43         // Sử dụng hàm callback với các sự kiện của FileSystemWatcher
44         watcher.Created += async (sender, e) => await OnChanged(e, config);
45         watcher.Changed += async (sender, e) => await OnChanged(e, config);
46
47         watcher.EnableRaisingEvents = true;
48
49         Console.WriteLine($"Monitoring file changes in: {config.RootPath}. Press Ctrl+C to exit.");
50         await Task.Delay(-1);
51     }
```

Agent sẽ yêu cầu nhập các thông tin cấu hình vào qua màn hình dòng lệnh, và lưu lại trong một tệp tin config.json.

```

53 1 reference
54 private static void EnsureConfigFileExists()
55 {
56     if (!File.Exists(ConfigFilePath))
57     {
58         Directory.CreateDirectory(Path.GetDirectoryName(ConfigFilePath));
59
60         // Yêu cầu người dùng nhập thông tin và lưu vào file cấu hình
61         Console.WriteLine("Please enter the server URL: ");
62         string apiServer = Console.ReadLine();
63
64         Console.WriteLine("Please enter the API key: ");
65         string apiKey = Console.ReadLine();
66
67         Console.WriteLine("Please enter the path of the folder to monitor: ");
68         string rootPath = Console.ReadLine();
69
70         Console.WriteLine("Please enter the exclude path, if any (leave empty if none): ");
71         string excludePath = Console.ReadLine();
72
73         var config = new Config
74         {
75             Id = Guid.NewGuid().ToString(), // Tự động tạo ID
76             Key = apiKey,
77             RootPath = rootPath,
78             ExcludePath = excludePath,
79             ApiServer = apiServer
80         };
81
82         string json = JsonSerializer.Serialize(config, new JsonSerializerOptions { WriteIndented = true });
83         File.WriteAllText(ConfigFilePath, json);
84     }

```

Chỉ các sự kiện khớp với các extensions như trên thì mới gửi lên máy chủ kiểm tra, đồng thời sử dụng sử dụng async và await để lập trình bất đồng bộ (asynchronous programming), cho phép chương trình thực hiện các tác vụ mất nhiều thời gian (như truy cập mạng, đọc/ghi tệp tin, gọi API) mà không làm gián đoạn luồng chính.

```

40 3 references
41 private static async Task OnChanged(FileSystemEventArgs e)
42 {
43     if (IsValidFileExtension(e.FullPath))
44     {
45         await SendToApi(e.ChangeType.ToString(), e.FullPath);
46     }
47
48 1 reference
49 private static bool IsValidFileExtension(string filePath)
50 {
51     string[] validExtensions = { ".html", ".htm", ".php", ".txt", ".jsp", ".aspx", ".shtml", ".hta" };
52     return Array.Exists(validExtensions, ext => filePath.EndsWith(ext, StringComparison.OrdinalIgnoreCase));
53 }

```

Cuối cùng, gửi sự kiện lên máy chủ qua API để kiểm tra.

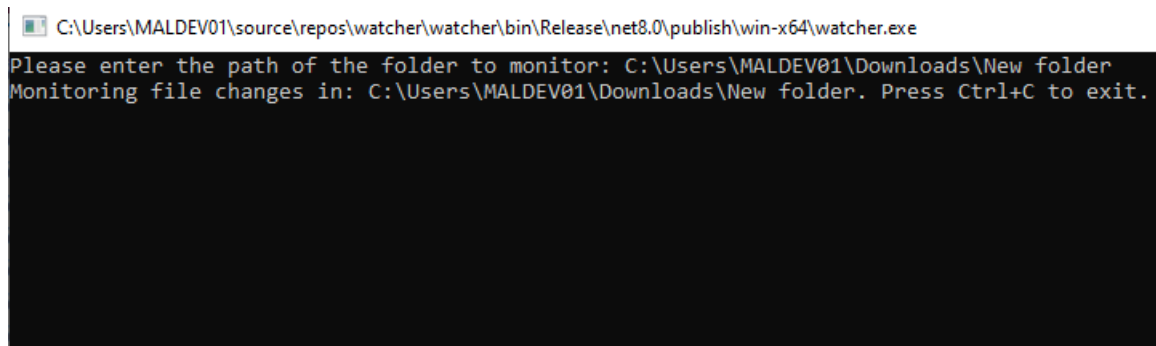
```

101 1 reference
102 private static async Task SendToApi(string apiServer, string apiKey, string filePath)
103 {
104     using (HttpClient client = new HttpClient())
105     {
106         var payload = new
107         {
108             key = apiKey,
109             path = filePath
110         };
111         string json = JsonSerializer.Serialize(payload);
112         var content = new StringContent(json, Encoding.UTF8, "application/json");
113
114         try
115         {
116             HttpResponseMessage response = await client.PostAsync(apiServer, content);
117             if (response.IsSuccessStatusCode)
118             {
119                 Console.WriteLine($"Successfully sent event for file: {filePath}");
120             }
121             else
122             {
123                 Console.WriteLine($"Failed to send event for file: {filePath}. Status: {response.StatusCode}");
124             }
125         }
126         catch (Exception ex)
127         {
128             Console.WriteLine($"Error sending event: {ex.Message}");
129         }
130     }
131 }
132

```

Sau khi lập trình xong, cần phải sử dụng tính năng Publish trong Visual Code để tạo ra một tệp tin .exe với đầy đủ các thư viện đã được nạp, khi đó tệp tin .exe này mới có thể chạy độc lập mà không gặp lỗi thiếu thư viện.

Chạy thử nghiệm chương trình



Nhật kí máy chủ phản hồi khi có sự kiện tới

```

image 1/1 /opt/In0ri/FlaskApp/static/images/86a9106ae65537651a8e456835b316ab.png: 256x256 cl
ean 1.00, defaced 0.00, 92.2ms
Speed: 93.8ms preprocess, 92.2ms inference, 1.2ms postprocess per image at shape (1, 3, 256,
256)
Everything oke!
192.168.244.135 - - [29/Oct/2024 15:24:24] "POST /checkdeface HTTP/1.1" 200 -

```

3.3.4. Giao diện tạo khóa kích hoạt cho Agent

Nghiên cứu sử dụng thư viện Flask trong Python3 để viết giao diện tương tác người dùng. Từ giao diện, nghiên cứu sẽ yêu cầu người dùng nhập thông tin URL trang web cần theo dõi, email đăng kí.

a) Kịch bản chức năng chính của hệ thống

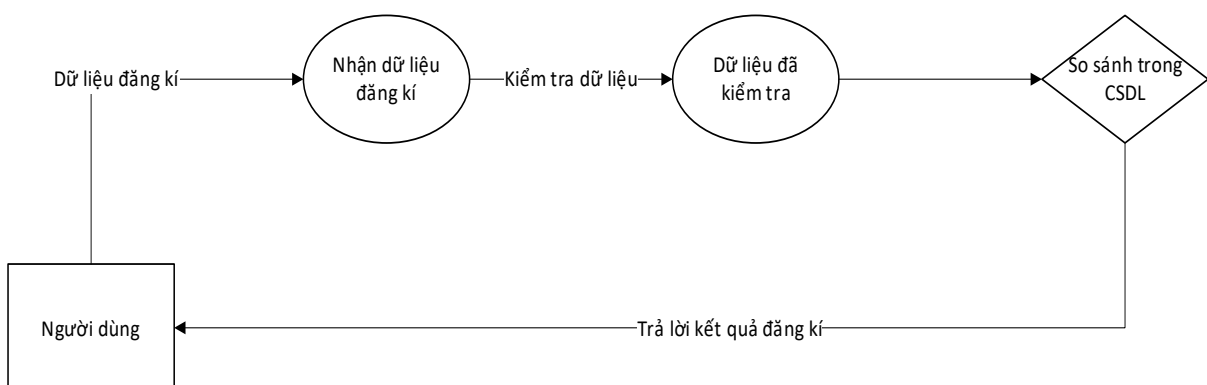
- Kịch bản người dùng đăng kí khóa sử dụng Agent

Tên usecase	Tạo khóa kích hoạt
Tác nhân chính	Khách hàng
Điều kiện	Tên URL đã phải đăng kí theo dõi trước đó hệ thống
Chuỗi sự kiện chính: 1. Khách hàng truy cập hệ thống ở phần tạo agent 2. Hệ thống hiển thị form nhập 3. Khách hàng nhập thông tin bao gồm địa chỉ email, URL và bấm nút “<i>Create Key</i>” 4. Hệ thống kiểm tra thông tin, thông báo đăng kí thành công, gửi key về email người dùng và chuyển hướng về trang chủ.	
Ngoại lệ: 1. Khách hàng nhập thiếu thông tin 1.1. Hệ thống báo lỗi phải nhập đầy đủ thông tin 2. Người dùng nhập URL chưa được đăng kí 2.1. Hệ thống báo lỗi URL chưa tồn tại	

b) Thiết kế cơ sở dữ liệu

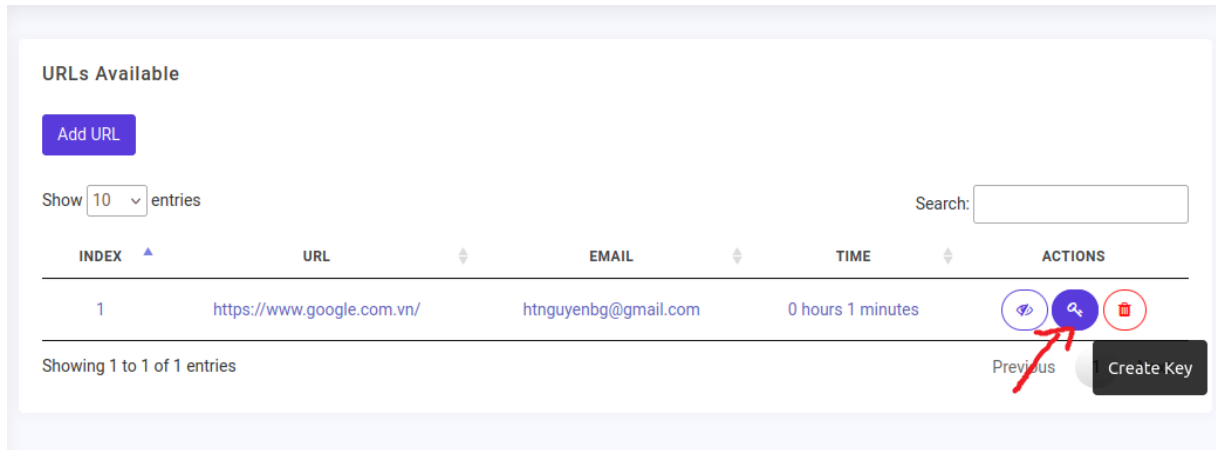
Chương trình sẽ cập nhật khóa vào trường `active_key` đã có sẵn trong CSDL.

• Sơ đồ luồng dữ liệu:



Hình 3.18. Sơ đồ đăng kí theo dõi một URL

c) Giao diện đăng kí



Hình 3.19. Giao diện tạo khóa xác thực Agent

Khi người dùng nhấn “Create Key”, tệp tin `app.py` sử dụng Python Flask xử lý yêu cầu từ phía người dùng. Sau khi kiểm tra sự tồn tại của URL đã đăng kí hay chưa (dòng 96), chương trình sẽ gọi hàm `make_license()` từ tệp tin `createLicense.py` để tạo khóa (dòng 103). Sau khi tạo khóa xong chương trình gọi hàm `update_existing()` từ tệp tin `database.py` để cập nhật vào CSDL (dòng 106), khóa trả về cho người dùng thông qua email (dòng 109), không hiển thị lên giao diện web.

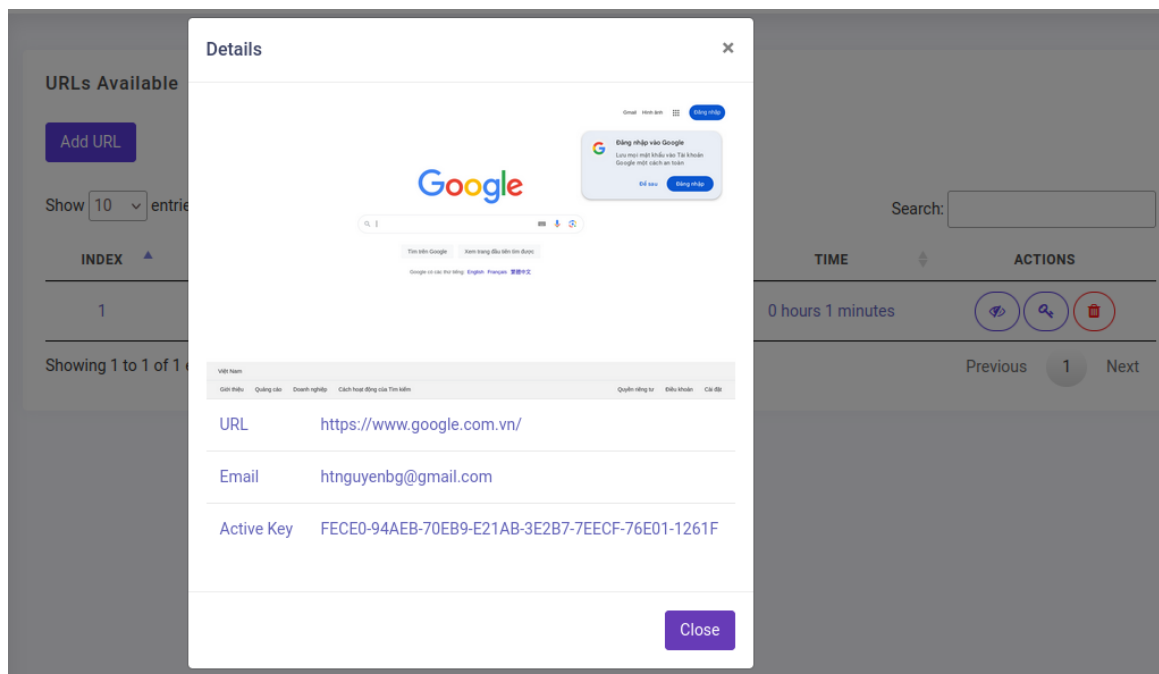
```
91 @app.route("/createAgent", methods=["POST"])
92 def createAgent():
93     url = slug(request.form["url"])
94     db = database.Database("site")
95     check = 1
96     for data in db.get_multiple_data():
97         if data["url"] == url:
98             receiver = data["email"]
99             check = 0
100     if check == 1:
101         response = "ERROR"
102     else:
103         active_key = license.make_license(url)
104         unique = {"url": url}
105         key_update = {"active_key": active_key}
106         db.update_existing(unique, key_update)
107         subject = "Get active key!"
108         message = f"You Agent's active key is: {active_key}"
109         al.sendMessage(receiver, subject, message)
110         response = "OKE"
111     return response
```

Để tạo khóa xác thực cho Agent, nghiên cứu sử dụng OpenSSL để tạo cặp khóa bất đối xứng DSA lưu tại thư mục `keys/`, việc sử dụng khóa riêng đăng kí làm giảm đáng kể khả năng những kẻ tấn công tạo ra trình tạo

khóa có thể tạo ra mã tương tự. Một trong những lợi thế của giải thuật DSA là đối với một URL nhất định, mỗi mã được tạo là khác nhau tại mỗi lần thực thi khác nhau, vì có một yếu tố ngẫu nhiên được đưa vào trong quá trình này. Nghiên cứu kí URL cùng với một chuỗi bí mật `product_code` và để con người có thể đọc được mã đăng ký, nghiên cứu mã hóa chúng bằng cách sử dụng base32 sửa đổi một chút để tránh các ký tự không rõ ràng. Nó cũng nhóm các mã theo bộ gồm năm ký tự được phân tách bằng dấu gạch ngang.

```
24 def make_license(domain):
25     private_key = crypto.load_privatekey(crypto.FILETYPE_PEM, private_key_string)
26     signature = crypto.sign(private_key, make_license_source(domain), "sha1")
27     # Use sha1 instead of dss1 to avoid 'ValueError("No such digest method")'
28     encoded_signature = base64.b32encode(signature)
29     encoded_signature = hashlib.sha1(encoded_signature).hexdigest()
30     encoded_signature = "-".join(
31         [encoded_signature[i : i + 5] for i in range(0, len(encoded_signature), 5)]
32     )
33     return encoded_signature.upper()
```

Kết quả: Sau khi đăng kí, một mã active key được gửi về email, người dùng có thể sử dụng nó để kích hoạt cho agent trên máy chủ.

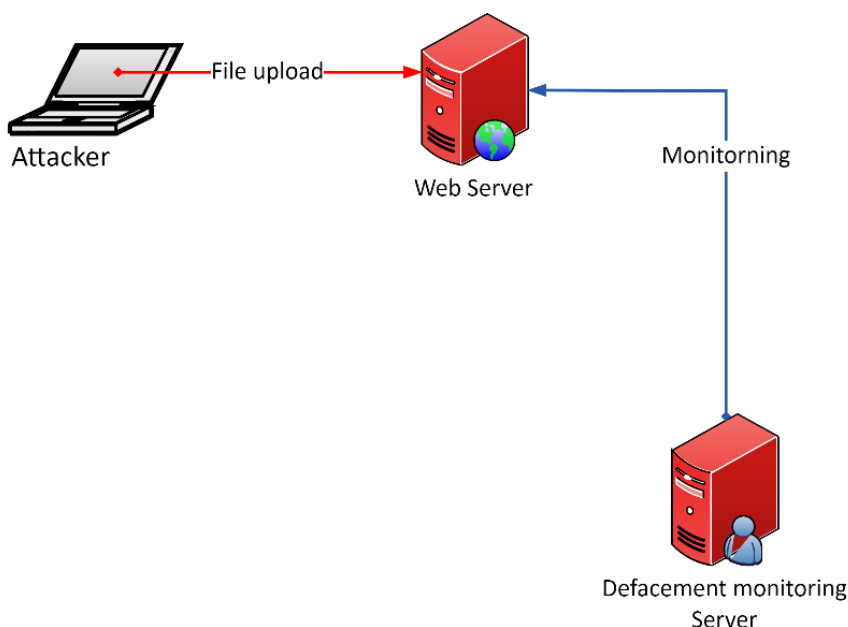


Hình 3.20. Trạng thái giao diện web tại lần cuối cùng được chụp và thông tin khóa

3.4. Thực nghiệm đánh giá kết quả

3.4.1. Kịch bản 1

Mục tiêu: Trong kịch bản này, nghiên cứu sẽ thực hiện tấn công thay đổi giao diện web một trang web (chưa được cài agent) để kiểm tra khả năng phát hiện và cảnh báo của hệ thống.



Hình 3.21. Sơ đồ kịch bản 1

Trong sơ đồ này, có một máy chủ web chạy hệ điều hành Ubuntu, ứng dụng web có tồn tại lỗ hổng File upload nguy hiểm. Máy chủ Defacement monitoring được triển khai theo dõi ứng dụng web. Máy tính attacker chạy hệ điều hành Windows 11, sử dụng công cụ Burp Suite để khai thác lỗ hổng Upload file trên ứng dụng web. Thông qua lỗ hổng File upload, attacker sẽ cố gắng lấy được quyền thực thi mã từ xa trên máy chủ web và thay đổi giao diện ứng dụng.

Các bước thực hiện:

Bước 1: Đăng kí theo dõi cảnh báo

Hình 3.22. Kịch bản 1: Đăng kí cấu hình theo dõi trang web

Hệ thống cảnh báo đã tự động lưu lại dữ liệu và lập lịch theo dõi.

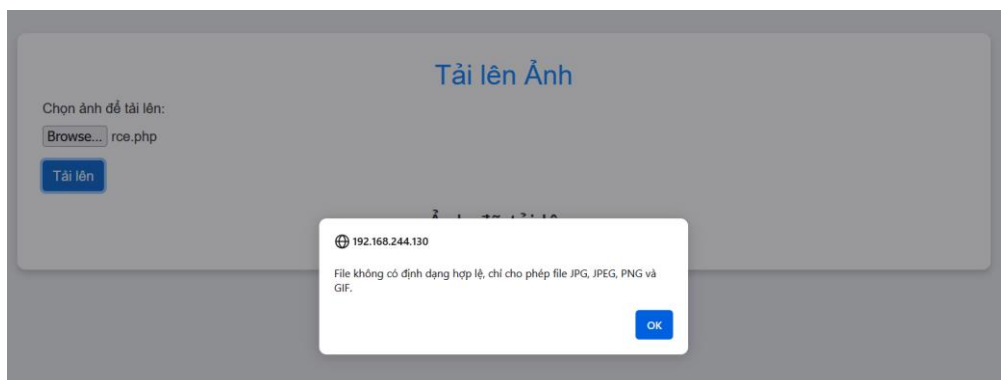
```
hoang@In0ri:~/snap$ crontab -l
* * * * * python3 /opt/In0ri/main.py http://192.168.244.130/ htnguyenbg@gmail.com # e
54e12250a522c9be02c06819c299582
```

Bước 2: Khai thác lỗ hổng ứng dụng

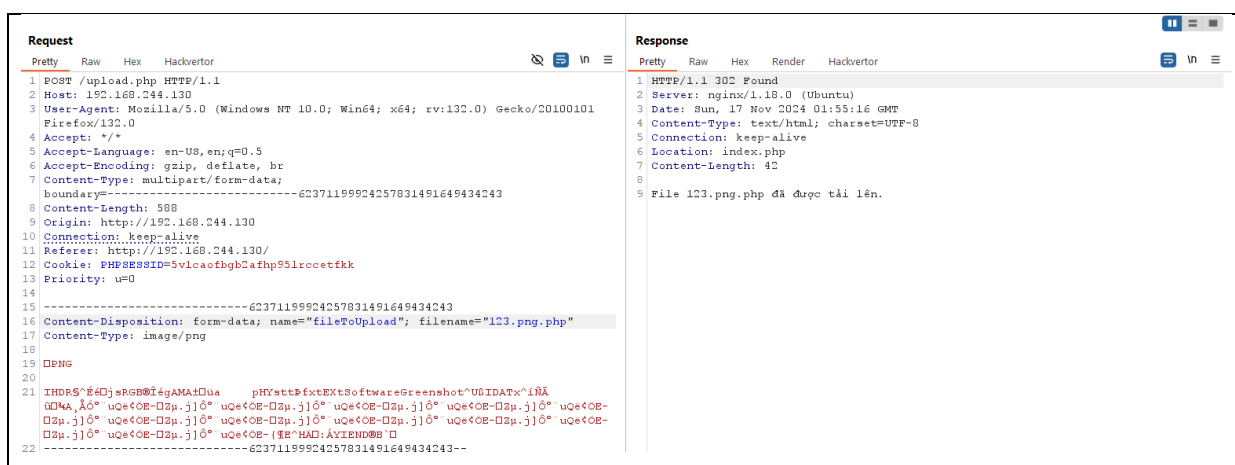
Nghiên cứu có trang tải lên và hiển thị tệp tin ảnh như sau:

Hình 3.23. Kịch bản 1: Giao diện ứng dụng web chứa lỗ hổng khai thác

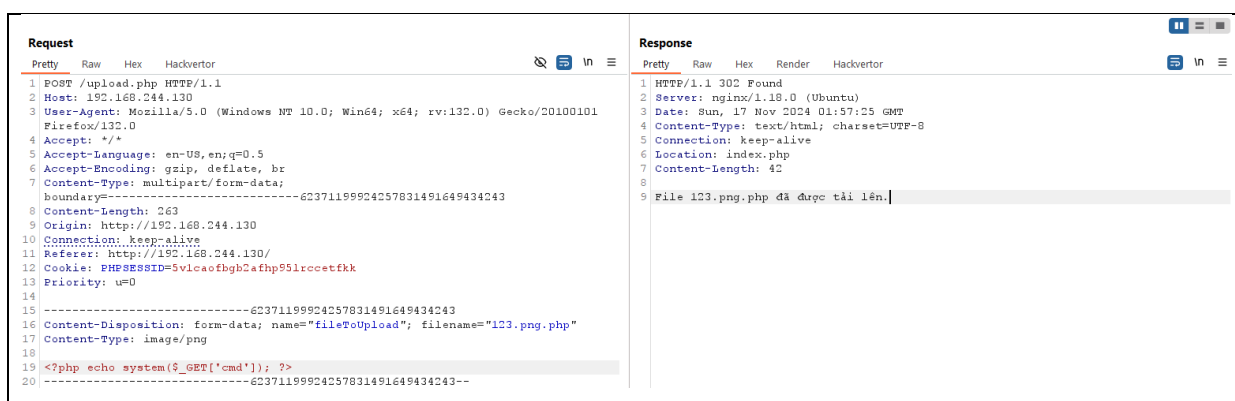
Khi tải lên một tệp tin có định dạng khác với định dạng ảnh, có một thông báo mã “File không có định dạng hợp lệ, chỉ cho phép file JPG, JPEG, PNG và GIF”.



Tuy nhiên, khi kiểm tra luồng gói tin gửi đi trong công cụ Burp Suite, không hề thấy có mã thông báo lỗi phản hồi hay tệp tin được gửi đi, do vậy chắc chắn việc kiểm tra định dạng tệp tin chỉ được thực hiện ở phía trình duyệt. Do vậy có thể dễ dàng khai thác lỗ hổng này bằng cách tải lên một tệp tin ảnh bình thường bất kì, rồi đổi nội dung của nó là thành công.

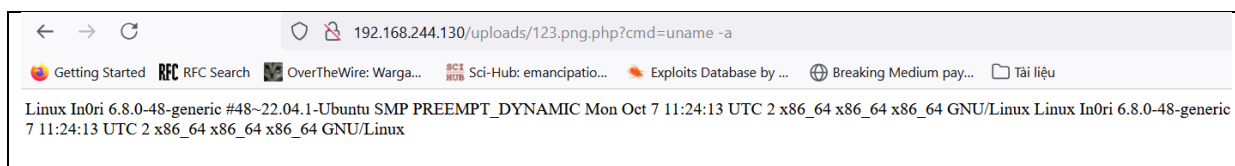


Thay đổi nội dung thành câu lệnh PHP, thực hiện thành công tải lên một webshell có thể thực thi mã từ xa trên máy chủ.



Ứng dụng web đã không trả về thông báo lỗi, chứng tỏ câu lệnh đã được thực thi thành công. Như vậy ứng dụng đã mắc hai lỗi bảo mật nghiêm trọng

trong trường hợp này dẫn tới máy chủ bị thực thi mã từ xa đó là lỗ hổng File upload và lỗi phân quyền cho phép các tệp tin trong thư mục uploads được phép thực thi. Kết quả ứng dụng bị kẻ tấn công thực thi mã từ xa:

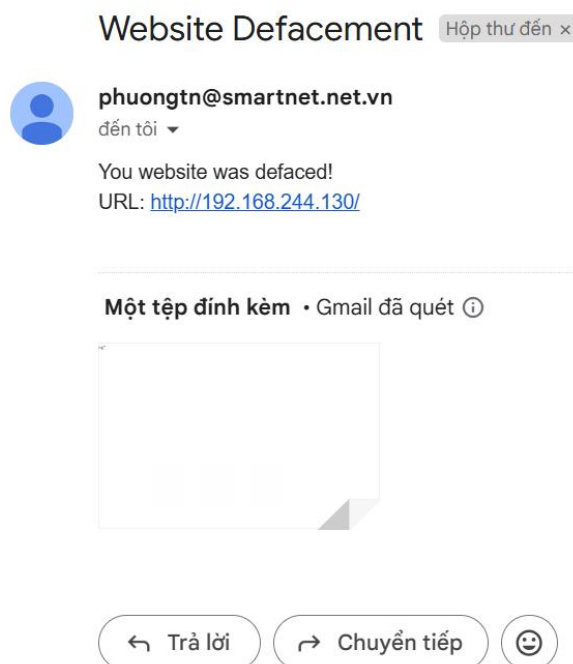


Sử dụng câu lệnh:

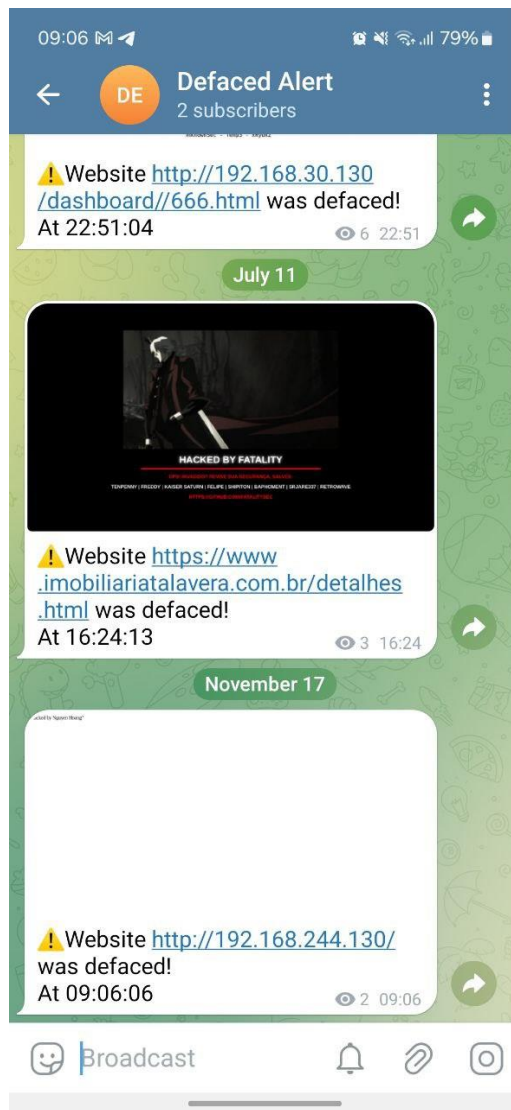
"http://192.168.244.130/uploads/123.png.php?cmd=echo%20%E2%80%9Chacked%20by%20Nguyen%20Hoang%E2%80%9D%20%E2%0A/var/www/html/index.php" đã có thể thay đổi giao diện được trang web. Kết quả:



Ngay sau đó, hệ thống liên cảnh báo về Email và Telegram rằng trang web đã bị tấn công thay đổi giao diện. Kết quả:



Hình 3.24. Kịch bản 1: cảnh báo về Email



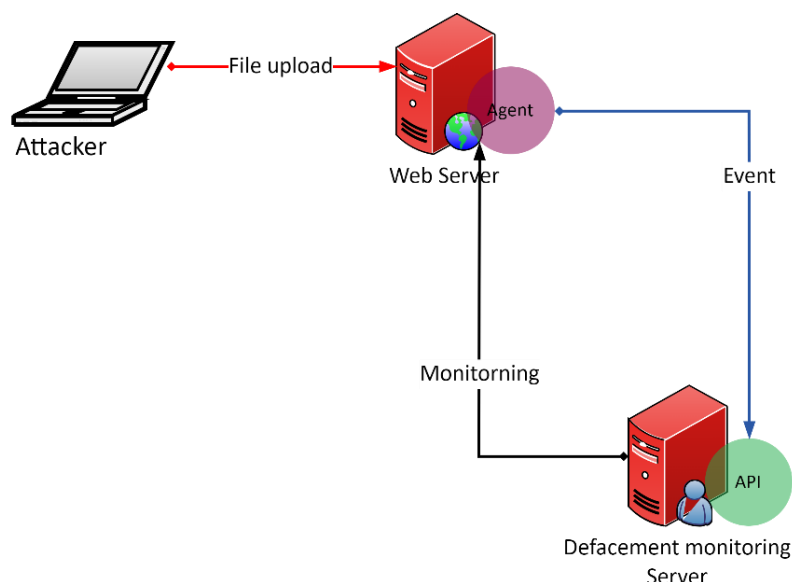
Hình 3.25. Kịch bản 1: cảnh báo về Telegram

Kết luận: Hệ thống đã phát hiện và cảnh báo rất chính xác

Đặt vấn đề: Nếu hiện tại kẻ tấn công thay đổi giao diện ở tệp tin login.php hay bất kì tệp tin chức năng nào khác hoặc kẻ tấn công tự tạo ra tệp tin có tên riêng, nó trong suốt với người dùng thì hệ thống không thể cảnh báo được những trường hợp này, do lúc đầu hệ thống mới chỉ đang kí theo dõi đối với url *http://192.168.244.130/* tức là chỉ theo dõi duy nhất với tệp tin index.php. Trong kịch bản 2, nghiên cứu sẽ giải quyết vấn đề này bằng cách cài Agent lên máy chủ web.

3.4.2. Kịch bản 2

Mục tiêu: Trong kịch bản này, nghiên cứu sẽ thực hiện tấn công thay đổi giao diện một trang web (được cài agent), trong đó sẽ đánh giá ưu điểm của hệ thống phát hiện cảnh báo khi máy chủ web được cài agent.



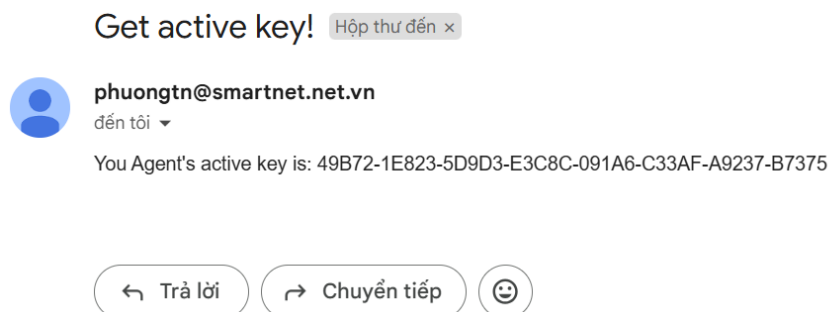
Hình 3.26. Sơ đồ kịch bản 2

Trong sơ đồ này, có một máy chủ web chạy hệ điều hành Ubuntu được cài agent của máy Defacement monitoring, ứng dụng web có tồn tại lỗ hổng Upload file nguy hiểm. Máy chủ Defacement monitoring được triển khai lắng nghe các sự kiện từ agent trên máy chủ web. Máy tính attacker chạy hệ điều hành Windows 10, sử dụng công cụ Burp Suite để khai thác lỗ hổng Upload file trên ứng dụng web. Thông qua lỗ hổng Upload file, attacker sẽ cố gắng lấy được quyền thực thi mã từ xa trên máy chủ web và thay đổi giao diện ứng dụng.

Bước 1: Đăng kí lấy khóa kích hoạt Agent

Trên trang chủ hệ thống cảnh báo, chọn “Create Agent” sau đó nhập chính xác thông tin URL đã đăng kí và email nhận tài key.

Sau khi nhấn “Submit”, key sẽ được gửi về email. Kết quả:



Hình 3.27. Kịch bản 2: active key được gửi về Email

Bước 2: Cấu hình Agent

Trong tệp tin config.json, trường “id” là phần để định danh một trang web, giả sử có theo dõi nhiều trang web trên một máy chủ thì mỗi id đại diện cho trang web đó, id phải là số nguyên dương. Trường “key” là chuỗi key ở email vừa được gửi, “rootPath” là thư mục gốc của trang web, “excludePath” là thư mục được phép bỏ qua không theo dõi.

```
hoang@In0ri:/opt/In0ri/Agent$ cat config.json
{
  "id": "01",
  "key": "49B72-1E823-5D9D3-E3C8C-091A6-C33AF-A9237-B7375",
  "rootPath": "/var/www/html",
  "excludePath": "",
  "apiServer": "http://192.168.244.130:8088/checkdeface"
}
```

Sau khi cấu hình xong, chạy câu lệnh sau để khởi chạy agent:

→ python3 agent.py

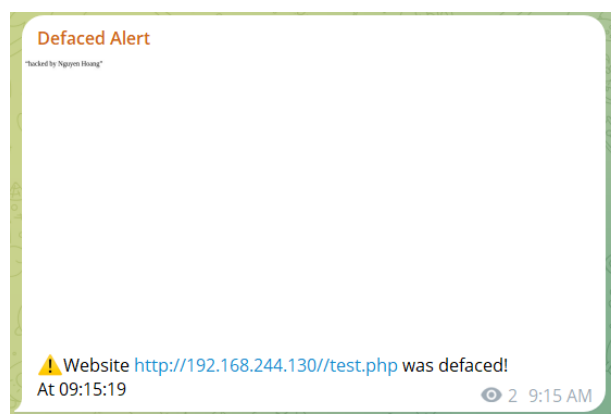
Bước 3: Tấn công thay đổi giao diện web

Trở lại công cụ Burp Suite, tải lại web shell lên máy chủ web.



Ngay lập tức agent đã phát hiện có một tệp bị thay đổi trên máy chủ web và gửi cảnh báo.

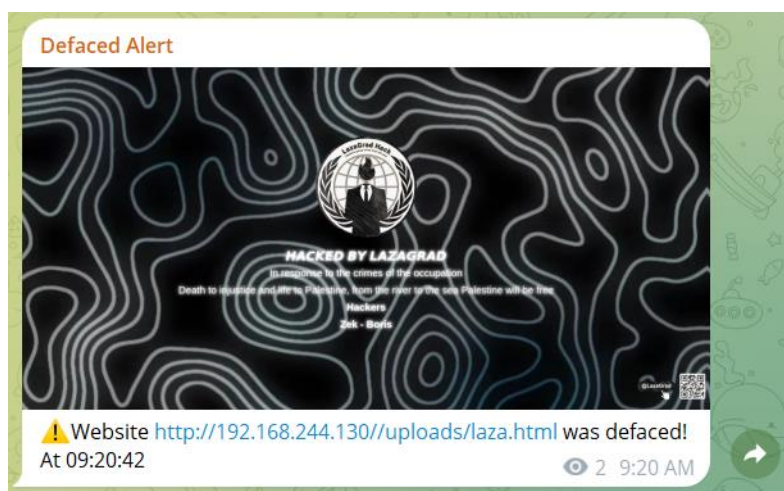
```
hoang@In0ri:/opt/In0ri/Agent$ python3 agent.py
Notification, /var/www/html/test.php has been modified
{'status': 'Website was defaced!'}
```



Hình 3.28. Kịch bản 2: cảnh báo một tệp tin bị thay đổi giao diện bất thường

Trong trường hợp kẻ tấn công chỉ đơn thuần tải một tệp tin hiển thị với chữ kí của chúng, không thay đổi gì đến các tệp tin chức năng trong hệ thống cả, agent cũng vẫn có thể phát hiện chính xác hành vi này. Nghiên cứu gọi lệnh sau để tải một tệp tin chữ kí của kẻ tấn công mới nhất được lấy từ trang mirror-h.org: `http://192.168.244.130/123.png.php?cmd=wget%20http://lmrolim.com.br/wp-content/uploads/2024/11/laza.html`

Kết quả hệ thống cũng đã phát hiện ra và gửi cảnh báo:



Hình 3.29. Kịch bản 2: cảnh báo về telegram

Giả sử quản trị viên thay đổi giao diện trang web một cách hợp lệ, ở đây nghiên cứu phục hồi lại tệp tin login.php vừa thay đổi. Ngay lập tức hệ thống cũng đã phát hiện sự thay đổi tệp tin và kiểm tra có phải bị tấn công hay không, nhưng không có cảnh báo do hành vi này là hợp lệ.

```
hoang@In0ri:/opt/In0ri/Agent$ python3 agent.py
Notification, File /var/www/html/login.php.bka has been moved to /var/www/html/login.php
{'status': 'Everything oke!'}
]
```

```
hoang@In0ri:/var/www/html$ mv login.php.bka login.php
hoang@In0ri:/var/www/html$
```

Log phía máy chủ:

```
192.168.244.130 - - [17/Nov/2024 09:22:26] "POST /checkdeface HTTP/1.1" 200 -  
Screenshoting...http://192.168.244.130//login.php  
  
image 1/1 /opt/In0ri/FlaskApp/static/images/639cc6e31c7141995ca1f97c2cff30b0.png: 256  
x256 clean 1.00, defaced 0.00, 10.9ms  
Speed: 11.5ms preprocess, 10.9ms inference, 0.2ms postprocess per image at shape (1,  
3, 256, 256)  
clean  
Everything oke!
```

Đặc biệt, nếu không tính thời gian chụp ảnh trang web, với mỗi bức ảnh, hệ thống chỉ mất khoảng 10.5ms để phân loại một bức ảnh. Cho thấy tốc độ phát hiện cực kì nhanh.

Kết luận: Hệ thống cảnh báo đã phát hiện rất chính xác tất cả các bài kiểm tra và không có cảnh báo giả. Nhờ Agent cài trên máy chủ, hệ thống đã có thể phát hiện được các đường dẫn chưa được đăng kí và cả các tệp tin lạ gây thay đổi giao diện trang web với tốc độ rất nhanh. Giúp cho người quản trị nắm được tình hình sớm nhất về tình trạng an toàn hệ thống của mình, để đưa ra các hướng điều tra ứng phó tiếp theo.

3.5. Kết luận chương 3

Chương này đã xây dựng thành công một hệ thống cảnh báo phát hiện tấn công thay đổi giao diện web, ứng dụng mô hình YOLOv10 đã luyện tại chương 2 để phát triển hệ thống. Hệ thống có tốc độ phát hiện nhanh và chính xác về email, hệ thống có giao diện đơn giản giúp đăng kí theo dõi trang web của mình, đồng thời có các agent để giám sát các trường hợp tấn công đặc biệt, giúp tránh bỏ sót các trường hợp tấn công khác nhau.

KẾT LUẬN

Kết quả đạt được

Qua quá trình tìm hiểu, nghiên cứu tham khảo các nguồn tài liệu cùng với sự hướng dẫn tận tình cả thầy giáo. Nghiên cứu đã đạt được một số kết quả như sau:

1. Xây dựng được một mô hình hiệu quả phát hiện được các cuộc tấn công thay đổi giao diện web dựa trên học sâu.
2. Xây dựng được một hệ thống cảnh báo với tốc độ phát hiện rất nhanh và có giao diện tương tác với người dùng đơn giản.
3. Hệ thống có agent giám sát trên các máy chủ web theo thời gian thực, giúp tránh bỏ sót các trường hợp tấn công đặc biệt.
4. Khắc phục được các điểm yếu của các giải pháp phát hiện trước đó và đạt được độ chính xác cao.

Hạn chế

Bên cạnh những kết quả đạt được, nghiên cứu vẫn còn một số hạn chế về tập dữ liệu huấn luyện chưa được đa dạng, tỉ lệ phát hiện chưa đạt được tuyệt đối, vẫn còn bỏ sót mẫu.

Hướng phát triển

Để có thể giải quyết được một số mặt hạn chế của đề tài thì cần có những hướng phát triển như sau:

- Cần thu thập thêm nhiều tập dữ liệu huấn luyện
- Xây dựng, tích hợp thêm hệ thống phát hiện dựa trên chữ ký, tăng khả năng phát hiện các trường hợp khó phát hiện khi sử dụng học máy để phân tích ảnh giúp giảm cảnh báo sai và tỉ lệ bỏ sót

PHỤ LỤC

Phụ lục 1: Đóng gói vào Docker

Nghiên cứu là một dự án phức tạp, có nhiều thành phần và thư viện khác nhau của Python được sử dụng trong hệ thống. Để dễ dàng triển khai và cập nhật khi có phiên bản mới trên một hệ thống mới, nghiên cứu sẽ đóng gói hệ thống gồm tất cả mã nguồn phía máy chủ và các cấu hình kết nối vào các container trong Docker, và khởi chạy chúng thông qua Docker Compose. Trong Docker Compose có 2 thành phần chính đó là ứng dụng flask_app là hệ thống cảnh báo đã phát triển ở phần trước và cơ sở dữ liệu mongoDB.

Đầu tiên cần viết một Dockerfile cho ứng dụng flask_app trước.

```
1 FROM ultralytics/ultralytics:latest-python
2
3 LABEL maintainer="Nguyen Hoang <htnguyen98.act@gmail.com>"
4
5
6 RUN apt-get update
7 RUN apt install -y \
8     cron \
9     python3-requests \
10    python3-flask \
11    chromium-chromedriver \
12    && pip install selenium \
13    && pip install Werkzeug==2.0.1 \
14    && pip install webdriver-manager \
15    && pip install Pillow \
16    && pip install python-telegram-bot==13.13 \
17    && pip install urllib3==1.26.6 \
18    && pip install chardet==4.0.0 \
19    && pip install MarkupSafe==2.0.1 \
20    && pip install requests==2.26.0 \
21    && pip install flask \
22    && pip install pymongo \
23    && pip install python-crontab \
24    && pip install pyOpenSSL
25
26 COPY . /opt/In0ri
27 ADD start.sh /start.sh
28 RUN chmod 755 /start.sh
29 EXPOSE 8080 8088
30 WORKDIR /opt/In0ri/FlaskApp
31 CMD ["/start.sh"]
```

Sau khi viết Dockerfile xong nghiên cứu sẽ dựng docker-compose để kết nối các thành phần lại.

```

1  version: "3.5"
2  services:
3    mongodb:
4      image: mongo:latest
5      container_name: mongodb
6      restart: unless-stopped
7      environment:
8        MONGO_INITDB_ROOT_USERNAME: admin
9        MONGO_INITDB_ROOT_PASSWORD: admin
10     ports:
11       - "27017:27017"
12     networks:
13       - app-network
14     volumes:
15       - type: volume
16         source: MONGO_DATA
17         target: /data/db
18       - type: volume
19         source: MONGO_CONFIG
20         target: /data/configdb
21     flask:
22       image: in0ri/defaced:latest
23       container_name: flask_app
24       restart: unless-stopped
25       environment:
26         MONGODB_USERNAME: admin
27         MONGODB_PASSWORD: admin
28         MONGODB_HOSTNAME: mongodb
29       ports:
30         - "8080:8080"
31         - "8088:8088"
32       depends_on:
33         - mongodb
34       networks:
35         - app-network
36
37     volumes:
38       - /etc/localtime:/etc/localtime:ro
39       - /etc/timezone:/etc/timezone:ro
40       - ./Alert:/opt/In0ri/Alert
41
42     networks:
43       app-network:
44         driver: bridge
45
46     volumes:
47       MONGO_DATA:
48         name: MONGO_DATA
49       MONGO_CONFIG:
50         name: MONGO_CONFIG

```

Để cài đặt hệ thống cảnh báo này, máy chủ chỉ cần cài đặt Docker Compose và dùng câu lệnh `docker-compose up`, hệ thống sẽ tự động cài đặt toàn bộ. Nên dựng một máy chủ Reverse Proxy để thiết lập HTTPs cho hệ thống nếu muốn công khai ra mạng Internet.

TÀI LIỆU THAM KHẢO

- [1] Xuan Dau Hoang and Ngoc Tuong Nguyen, “*Detecting Website Defacements Based on Machine Learning Techniques and Attack Signatures*”, 8 May 2019.
- [2] Kevin Borgolte, Christopher Kruegel, Giovanni Vigna, University of California, Santa Barbara, “*Meerkat: Detecting Website Defacements through Image-based Object Recognition*”, August 12–14, 2015, Washington, D.C.
- [3] Ao Wang, Hui Chen, Lihao Liu, Kai Chen, Zijia Lin, Jungong Han, Guiguang Ding of Tsinghua University. “*YOLOv10: Real-Time End-to-End Object Detection*”, 23 May 2024
- [4] Mujadded Al Rabbani Alif* and Muhammad Hussain of Huddersfield University. “*YOLOv1 to YOLOv10: A comprehensive review of YOLO variants and their application in the agricultural domain*”, 14 Jun 2024
- [5] Fei Tony Liu and Kai Ming Ting Gippsland School of Information Technology Monash University, Zhi-Hua Zhou National Key Laboratory for Novel Software Technology Nanjing University, “*Isolation-based Anomaly Detection*”, March 2012
- [6] Pascal Vincent, et al. ICML, “*Extracting and composing robust features with denoising autoencoders*”, 2008.
- [7] Pascal Vincent, et al. Journal of machine learning research, “*Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion*”, 11. Dec (2010): 3371-3408.
- [8] Bruce Schneier, “*Applied Cryptography, Second Edition: Protocols, Algorithms, and Source Code in C (cloth)*”, October 18, 1996.
- [12] Fei-Fei Li & Andrej Karpathy, “*Stanford class: Convolutional Neural Networks for Visual Recognition*”, Jan. 2015. [Online]. Available: <http://cs231n.stanford.edu/syllabus.html>

- [13] Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). *“Imagenet classification with deep convolutional neural networks. In Advances in neural information processing systems”*, (pp. 1097-1105).
- [14] Zed A. Shaw, *“Learn Python 3 the Hard Way Third Edition”*, October 2013.
- [16] Zone-H.org. [Online]. Available: <http://www.zone-h.org/stats/ynd/>.
- [17] Zone.kurd-h.org. [Online]. Available: <https://zone.kurd-h.org/>.
- [18] Mirror-h.org. [Online]. Available: <https://mirror-h.org/>.
- [20] The Moz Top 500 Websites. [Online]. Available: <https://moz.com/top500>
- [21] Jan Hosang, Rodrigo Benenson, Bernt Schiele of Max Planck Institut für Informatik Saarbrücken, Germany. *“Learning non-maximum suppression”*, 9 May 2017
- [22] FileSystemWatcher. [Online]. Available: <https://learn.microsoft.com/en-us/dotnet/api/system.io.filesystemwatcher?view=net-8.0>
- [23] Joseph Redmon* , Santosh Divvala* , Ross Girshick , Ali Farhadi* , University of Washington* , Allen Institute for AI, Facebook AI Research, *“You Only Look Once: Unified, Real-Time Object Detection”*, 9 May 2016